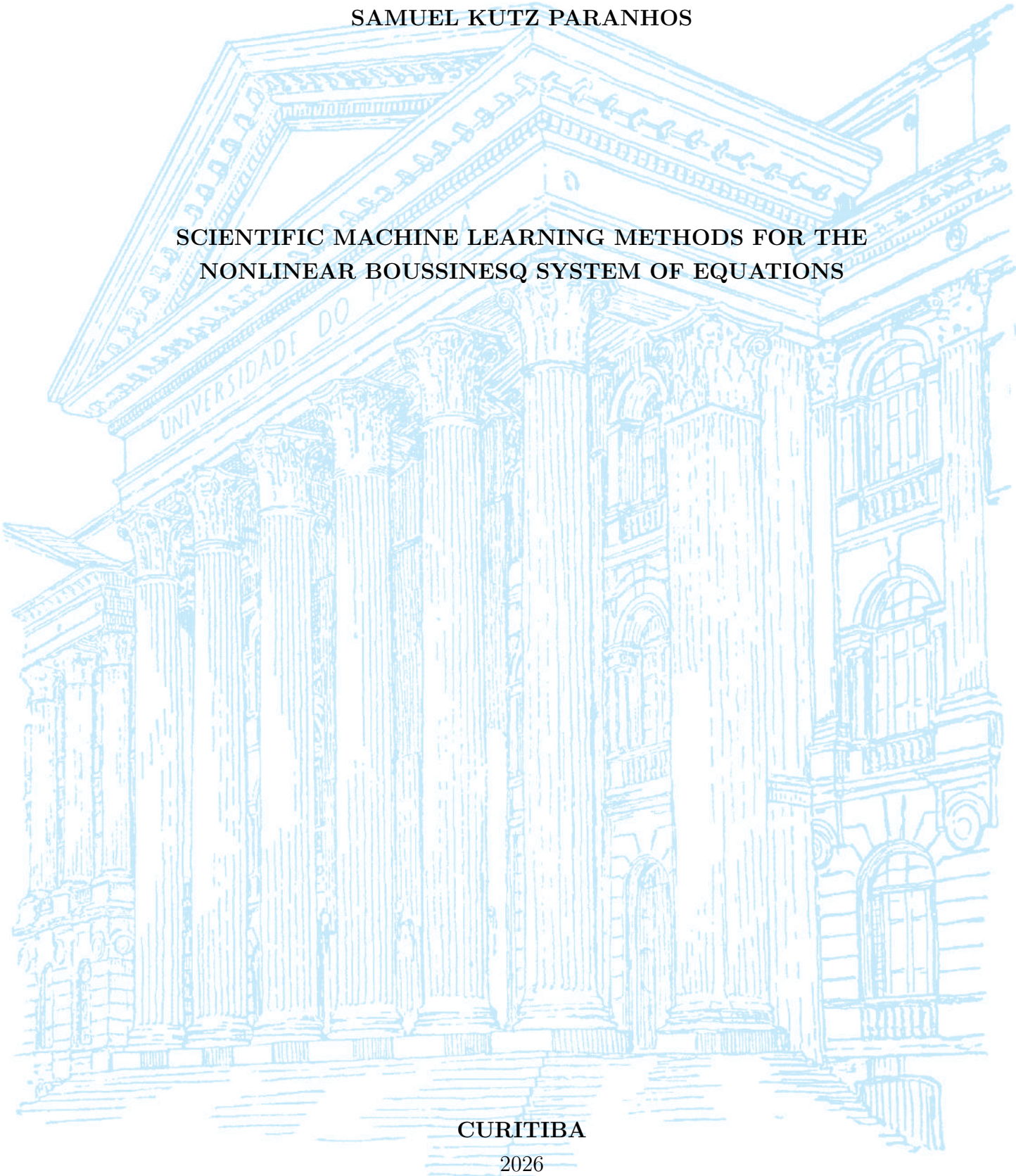
UNIVERSIDADE FEDERAL DO PARANÁ

SAMUEL KUTZ PARANHOS

SCIENTIFIC MACHINE LEARNING METHODS FOR THE NONLINEAR BOUSSINESQ SYSTEM OF EQUATIONS



CURITIBA

2026

SAMUEL KUTZ PARANHOS

TRABALHO DE CONCLUSÃO DE CURSO

SCIENTIFIC MACHINE LEARNING METHODS FOR THE NONLINEAR BOUSSINESQ SYSTEM OF EQUATIONS

Trabalho apresentado como requisito parcial à obtenção do grau de Bacharel em Matemática Industrial no Departamento de Matemática da Universidade Federal do Paraná.

Área de concentração: Matemática Industrial.

Orientador: Prof. Roberto Ribeiro.

Título do Projeto: Scientific Machine Learning Methods for the Nonlinear Boussinesq System of Equations.

CURITIBA

2026

RESUMO

Este trabalho investiga a eficácia de métodos de Scientific Machine Learning (SciML) na resolução e aproximação do sistema não linear de Boussinesq, um sistema de Equações Diferenciais Parciais (EDPs) utilizado para modelar a dinâmica de ondas aquáticas. Neste estudo, o sistema de Boussinesq será tratado como uma EDP dependente de dois parâmetros: os parâmetros de não linearidade e de dispersão, aqui mantidos iguais entre si, de modo que uma única coordenada controle a dificuldade do problema. O objetivo é analisar o uso do ecossistema contemporâneo de ferramentas desenvolvido em torno do recente crescimento de Machine Learning como complemento aos métodos numéricos clássicos empregados na solução do sistema de Boussinesq. Foi utilizado o método pseudoespectral para gerar um conjunto de dados consistindo de soluções de referência do sistema de Boussinesq para diversos parâmetros, empregado tanto como base de treinamento quanto como referência de acurácia para quatro arquiteturas de SciML investigadas neste trabalho: um Perceptron Multicamadas (MLP), treinado apenas com dados e utilizado como linha de base; Redes Neurais Informadas pela Física (PINNs), treinadas a um único parâmetro tanto com restrições de dados quanto de forma puramente física; Operadores Neurais de Fourier (FNO), operando em regime puramente supervisionado a vários parâmetros com uso exclusivo de dados sem nenhuma informação do operador diferencial da EDP; e Operadores Neurais Informados pela Física (PINO), que consiste em uma FNO com a inclusão do operador diferencial da EDP na função de perda da arquitetura, treinada com e sem dados. Dessas arquiteturas resultam seis modelos, organizados em duas famílias, a pontual e a de operadores, e em três regimes de treinamento: puramente supervisionado, dados e física, e puramente físico. Como métricas de eficiência dessas arquiteturas, utilizamos a evolução do erro espectral e do erro relativo ao longo do espaço-tempo com relação a solução de referência, avaliadas ao longo da faixa de parâmetros e também em resoluções distintas daquela de treinamento, de modo a verificar a invariância à discretização própria dos operadores. O mecanismo por trás do viés espectral é ainda examinado em um experimento mínimo, no qual redes de larguras distintas ajustam um único perfil de elevação do sistema de Boussinesq e têm seu núcleo tangente neural (NTK) medido ao longo do treinamento. Como resultados, percebe-se que as arquiteturas pontuais, especialmente quando treinadas sem o auxílio de dados, sofrem de um fenômeno conhecido como viés espectral, apresentando tendência a aprender primeiramente as componentes de baixa frequência da solução e grandes dificuldades em capturar detalhes essenciais de alta frequência. Em contrapartida, por aprenderem diretamente no espaço das frequências, os modelos de aprendizado de operadores, como o FNO e o PINO treinado com dados, atenuam essa limitação nas baixas e médias frequências, embora nenhum método a elimine por completo: um erro residual de alta frequência

persiste em todas as arquiteturas, e o viés reaparece de forma mais acentuada quando o PINO é treinado de maneira puramente física. Observa-se ainda que a inclusão do resíduo físico confere estabilidade temporal, suprimindo o artefato de Gibbs que degrada o FNO no instante final, e que a invariância à discretização se sustenta no aspecto geométrico, mas não em acurácia. O experimento mínimo, por sua vez, mostra que o decaimento do erro segue a previsão do núcleo inicial nas direções alcançáveis e que os autovalores do núcleo decaem por várias ordens de grandeza, o que permite atribuir o desequilíbrio entre frequências a uma causa mensurável em vez de a uma analogia. Assim, concluímos que, entre as intervenções examinadas, a inclusão de dados na função de perda é a que mais atenua — ainda que não anule — o viés espectral comumente presente nessas arquiteturas, enquanto o resíduo físico contribui sobretudo com estabilidade ao longo do tempo.

Palavras-chave: Scientific Machine Learning; Equações Diferenciais Parciais; Sistema de Boussinesq; Redes Neurais Informadas pela Física; Viés Espectral.

ABSTRACT

This work investigates the effectiveness of Scientific Machine Learning (SciML) methods in solving and approximating the nonlinear Boussinesq system, a system of Partial Differential Equations (PDEs) used to model water wave dynamics. In this study, the Boussinesq system will be treated as a PDE dependent on two parameters: the nonlinearity and dispersion parameters, here held equal to one another, so that a single coordinate controls the difficulty of the problem. The objective is to analyze the use of the contemporary ecosystem of tools developed around the recent growth of Machine Learning as a complement to the classical numerical methods employed in solving the Boussinesq system. The pseudospectral method was used to generate a dataset consisting of reference solutions of the Boussinesq system for various parameters, employed both as the training basis and as the accuracy reference for four SciML architectures investigated in this work: a multilayer perceptron (MLP), trained on data alone and used as a baseline; Physics-Informed Neural Networks (PINNs), trained at a single parameter both with data constraints and purely physics-informed; Fourier Neural Operators (FNOs), operating in a purely supervised regime at multiple parameters using only data without any differential operator information from the PDE; and Physics-Informed Neural Operators (PINOs), which consist of an FNO with the inclusion of the PDE differential operator in the architecture loss function, trained with and without data. These architectures give rise to six trained models, arranged into two families, the pointwise one and the operator one, and three training regimes: purely supervised, data and physics, and purely physics-informed. As efficiency metrics for these architectures, we use the evolution of spectral error and relative error over space-time with respect to the reference solution, evaluated across the parameter range and also at resolutions other than the training one, so as to test the discretization invariance operators are supposed to enjoy. The mechanism behind spectral bias is further examined in a minimal experiment, in which networks of different widths fit a single elevation profile of the Boussinesq system and have their Neural Tangent Kernel (NTK) measured along training. As results, it is observed that the pointwise architectures, especially when trained without the aid of data, suffer from a phenomenon known as spectral bias, showing a tendency to learn the low-frequency components of the solution first and great difficulty capturing essential high-frequency details. In contrast, by learning directly in the frequency space, operator learning models such as FNO and PINO trained with data attenuate this limitation in the low and mid frequencies, although no method removes it entirely: a residual high-frequency error persists across all architectures, and the bias re-emerges more strongly when PINO is trained in a purely physics-informed manner. We further observe that the physics residual confers temporal stability, suppressing the Gibbs artifact that degrades the FNO at the final time, and that

discretization invariance holds geometrically but not in accuracy. The minimal experiment, in turn, shows that the error decay follows the initial-kernel prediction along the reachable directions and that the kernel eigenvalues fall by several orders of magnitude, which lets the imbalance between frequencies be traced to a measurable cause rather than to an analogy. Thus, we conclude that, among the interventions examined, including data in the loss function is the one that most attenuates — though it does not eliminate — the spectral bias commonly present in these architectures, while the physics residual contributes mainly stability over time.

Keywords: Scientific Machine Learning; Partial Differential Equations; Boussinesq System; Physics-Informed Neural Networks; Spectral Bias.

ACKNOWLEDGEMENTS

I would like to thank CNPq, the LabFluid laboratory, and the Federal University of Paraná (UFPR) for their support, infrastructure, and guidance during this research.

This work was supported by CNPq and developed with the collaboration of the LabFluid research group at UFPR.

CONTENTS

LIST OF FIGURES

LIST OF ABBREVIATIONS

SciML	Scientific Machine Learning
ML	Machine Learning
PDE	Partial Differential Equation
PINN	Physics-Informed Neural Network
FNO	Fourier Neural Operator
PINO	Physics-Informed Neural Operator
MLP	Multilayer Perceptron
RK4	Runge–Kutta 4th-order method
DFT	Discrete Fourier Transform
IDFT	Inverse Discrete Fourier Transform
FFT	Fast Fourier Transform
NTK	Neural Tangent Kernel
MSE	Mean Squared Error
SGD	Stochastic Gradient Descent
AD	Automatic Differentiation

LIST OF SYMBOLS

Domain and general operators

x	spatial coordinate
t	temporal coordinate
$u(x,t)$	PDE solution (generic)
$\mathcal{D}$	differential operator of the PDE
$\mathcal{L}$	linear differential operator; also a loss functional
∇	gradient (nabla) operator
N_x	number of spatial grid points
N_t	number of time steps
Δx, Δt	spatial and temporal grid spacings

Boussinesq system and spectral quantities

$\eta(x,t)$	surface elevation field
$u(x,t)$	depth-averaged horizontal velocity field
α	nonlinearity parameter of the Boussinesq system
β	dispersion parameter of the Boussinesq system
A	initial wave amplitude
L	domain half-length, $\Omega = [-L, L]$
k	wavenumber (integer mode index)
ξ_k	wavenumber $\pi k/L$ on the domain $[-L, L]$
$\omega(k)$	angular frequency / dispersion relation
$\hat{f}(k)$	Fourier coefficient of f at mode k
$\hat{f}(k,t)$	time-evolving Fourier mode of f at wavenumber k

Neural networks and Neural Tangent Kernel

θ	trainable parameter vector of a neural network
u_θ	neural network approximation of the PDE solution
$h^{(l)}$	hidden-layer activation vector at layer l
σ	activation function
$W^{(l)}$, $b^{(l)}$	weight matrix and bias vector at layer l
N_{MLP}	number of neurons in a hidden layer
L_{MLP}	number of layers in the neural network
$K(\theta)$	empirical Neural Tangent Kernel, $K = JJ^{\mathsf{T}}$
λ_k	NTK eigenvalue associated with mode k
$\hat{e}(k,t)$	Fourier coefficient of the training error at mode k

Neural operators

$G^\dagger$, G_θ	target solution operator and its neural approximation
$\mathcal{K}_\ell$	integral operator of the ℓ-th FNO spectral layer
κ_ℓ	kernel function of the FNO integral operator
P	lifting operator of the FNO
Q	projection operator of the FNO
d_c	channel (width) dimension of the FNO
$R_{\ell,k}$	FNO spectral weight (discretized kernel) at layer ℓ, mode k
$k_{\max}$	FNO mode-truncation threshold

1 INTRODUCTION

Since the start of scientific thinking, mathematics has been one of the key languages in which we can understand the world, and Partial Differential Equations (PDEs) are one of the main dialects of the mathematical language we use to model and to simplify nature's complexity.

When modelling these equations, we cannot always afford them to have simple closed-form solutions (that is, when they have a solution at all), this exposes the need of obtaining approximated solutions coming from numerical methods for these PDEs, also giving the option to circumvent analytical complexity and allowing the extraction of predictions, comparisons with data, the building of simulations, and so on. These classical numerical methods, widely studied for all types of differential equations, usually rely on discretizing the PDE's domain into grids and performing temporal steps.

With the recent ascension of Machine Learning (ML) methods and a whole ecosystem of tools built around it, a natural path is to bridge these two areas and let ML augment the numerical solution of PDEs, taking advantage of the many software frameworks developed. This is one facet of Scientific Machine Learning (SciML), the broader effort to fuse physics-based, mechanistic models with data-driven methods.

Within SciML, this work focuses on the numerical solution of the Boussinesq System. Given the vast literature around application of SciML on PDEs, we restrict our literature review to works on hydrodynamics equations.

1.1 Literature Overview on Scientific Machine Learning for PDEs

The neural network building blocks underlying all of these approaches trace a line beginning with the formal neuron of McCulloch et al. (1943) and Rosenblatt's perceptron (ROSENBLATT, 1958), the first trainable single-layer model. The key algorithmic advance that unlocked multi-layer training was the backpropagation algorithm (RUMELHART et al., 1986), making gradient-based optimization practical for deep architectures. Shortly after, the universal approximation theorem (CYBENKO, 1989; HORNIK et al., 1989) provided the theoretical guarantee for a multilayer perceptron (MLP) with a single hidden layer of sufficient width to be able to approximate any continuous function on

a compact domain to arbitrary precision. This combination of practical and universal approximation established the MLP as the backbone of modern deep learning. The term neural network itself reflects the biological metaphor at the heart of these models, in which McCulloch et al. (1943) drew an explicit analogy to the firing of biological neurons, and this vocabulary propagated through the field, causing MLPs to be routinely and interchangeably referred to as artificial neural networks (ANNs) or feedforward neural networks. The idea of "deep" in deep learning, popularized following the breakthrough results of LeCun et al. (2015), simply denotes an MLP with many hidden layers, a regime in which representational capacity grows substantially and where the backpropagation-based training paradigm proves to be exceptionally efficient.

These ML foundations become the basis of a scientific methodology once the methods are coupled to the physical laws that govern a problem of interest. This is the premise of Scientific Machine Learning, a term consolidated by Baker et al. (2019) to name the discipline at the intersection of scientific computing and machine learning, and surveyed in breadth by Karniadakis et al. (2021). Its unifying goal is to combine the interpretability, generalization, and inductive biases of physics-based models with the flexibility and data-adaptivity of ML, rather than treating the network as a pure black box. The methods span several families. Some recover governing equations directly from data, as in the Sparse Identification of Nonlinear Dynamics (SINDy) (BRUNTON et al., 2016). Others embed differential-equation structure into the network itself, as in Neural Ordinary Differential Equations (neural ODEs) (CHEN, R. T. Q. et al., 2018) and Universal Differential Equations (UDEs), in which a trainable network stands in for the unknown terms of an otherwise mechanistic model (RACKAUCKAS et al., 2020). These tools have been carried across a wide range of domains, from global weather and climate forecasting (BONEV et al., 2023) to molecular dynamics, materials design, and turbulence modelling (KARNIADAKIS et al., 2021). Among all these directions, the one that concerns us here is the use of SciML to obtain numerical solutions of hydrodynamics equations, as cited above.

SciML for PDEs was reshaped by one idea in particular, the Physics-Informed Neural Networks (PINNs) (RAISSI et al., 2019), that is, the embedding of the governing PDE residual directly into the training loss of a Neural Network, working as regularization term for not only fitting data, but also respecting the underlying physics. In hydrodynamics, a growing body of work has applied PINNs to water flow (DE PINA et al., 2021, 2023), shallow-water dynamics on the sphere (BIHLO et al., 2022), and inverse problems in strongly nonlinear wave systems (JAGTAP et al., 2022). The consistent result that we can address from reviews (CUOMO et al., 2023) is that PINNs are most effective when observational data are sparse and the task is parameter identification, namely inverse problems, which are known for usually being not well-posed problems. But, for a reliable

precise forward simulation at scale, they do not yet rival classical solvers. In our vision, these approaches serve best to complement them.

As a natural extension from neural networks, there is a recent development in learning maps between infinite-dimensional function spaces, in which the theoretical foundation was popularized by T. Chen et al. (1995), who extended the universal approximation theorem to nonlinear operator, in other words, a shallow network can approximate any continuous operator between Banach spaces to arbitrary precision. Bringing this idea into practical terms, Lu et al. (2021) introduced DeepONet, the first deep architecture expressly designed for learning operators coming from differential equations, demonstrating it across a range of ODEs and PDEs. Building on this line of work, operator learning puts aside the single-instance limitation of PINNs by training a model to approximate the full mapping from parameters and initial data to the solution, so that at inference time a new solution costs only a forward pass for the network. Parallel to DeepONet, Fourier Neural Operator (FNO) (LI et al., 2021) is one of the most widely adopted instances of this idea: it parametrizes an integral kernel in Fourier space, achieving speed-ups over other neural operators, and it is discretization-independent, the so-called zero-shot superresolution. Subsequent works explored alternative bases like wavelets (GUPTA et al., 2021), but FNO was validated on many types of equations such as dispersive and soliton wave equations (PEI et al., 2022; ZHONG et al., 2023), and was also scaled to geophysical problems spanning regional ocean modeling, shallow-water emulation, and global weather forecasting (BONEV et al., 2023; LKHAGVASUREN et al., 2024; UCHIDA et al., 2025; PATEL et al., 2025; YU et al., 2025).

Following the idea for PINNs, the Physics-Informed Neural Operator (PINO) (LI et al., 2023a) combines operator learning with PDE-residual constraints, cutting the labelled-data requirement while preserving physical consistency. Applications to shallow-water systems (ROSOFSKY et al., 2023) confirmed that this hybrid regime inherits the generalization of FNO without its data hunger. Broad surveys of the field (TODO, 2023) identify these physics-data approaches working together as a promising SciML direction, while naming turbulence modeling, scalability, and noisy-data robustness as open challenges.

A known limitation that cuts across all SciML architectures is the so-called spectral bias, the tendency of gradient-based optimizers to learn the low-frequency components of a solution faster than the high-frequency ones (RAHAMAN et al., 2019; XU et al., 2020). The theoretical account of this behavior came with the Neural Tangent Kernel (NTK) (JACOT et al., 2018), which showed that a sufficiently wide network trains, to leading order, as a linear model governed by a kernel that stays essentially fixed during optimization, the so-called lazy-training regime. In this picture each mode of the

solution is learned at a rate set by its associated NTK eigenvalue, so the wide disparity between the largest and smallest eigenvalues translates directly into the frequency imbalance seen during training (ARORA et al., 2019; CAO et al., 2021; BORDELON et al., 2020). Wang, Yu, et al. (2022) carried this analysis to PINNs, showing that the eigenvalue disparity between the PDE-residual and initial-condition loss components is what drives their spectral bias. This pathology is especially consequential for dispersive wave problems, where the solution energy is broadly distributed across wavenumbers. A recent systematic study (KHODAKARAMI et al., 2026) showed that second-order optimizers combined with spectrum-aware loss formulations substantially mitigate it.

As far as this review reveals, SciML methods have been applied to a broad class of shallow-water and dispersive wave equations. To the best of our knowledge, however, no study has systematically evaluated FNO, PINO, or PINNs on the parametric 1D Boussinesq system, where both the nonlinearity coefficient α and the dispersion coefficient β vary simultaneously. This gap motivates our effort to implement and evaluate these ScieML methods with the goal of exposing their details for the user aiming to bring together this new set of techniques.

1.2 The Boussinesq System of Equations

For this work, we choose to implement SciML methods for a specific PDE originating from J. Boussinesq's 1872 effort to give a theoretical account of the solitary wave of translation observed by J. Scott Russell in a canal in 1834 (BOUSSINESQ, 1872). Russell had noticed that a hump of water, set in motion by a stopping barge, could travel for miles without changing shape, a phenomenon the linear wave theory of the time could not explain. Boussinesq showed that the stability of such a wave is the result of a precise balance between nonlinear steepening and frequency dispersion, two effects that are separately destabilizing but mutually compensating in the shallow-water, long-wave regime. The mathematical structure of this balance, and of dispersive wave theory in general, are treated in depth by Whitham (1974) and Debnath (2012).

A modern two-field form of the system, for the free-surface elevation $\eta(x,t)$ and the depth-averaged horizontal velocity $u(x,t)$, was systematically derived and classified by Bona et al. (2002), who identified a four-parameter family of equivalent Boussinesq systems obtainable from the incompressible, irrotational Euler equations by asymptotic simplifications. Earlier, Peregrine (1967) had already written down the standard form of these equations for water of variable depth, establishing the notation and scaling that remain in widespread use. What makes the Boussinesq system particularly suitable as a parametric testbed for this work is that changing the nonlinearity and dispersion coefficients continuously deforms the solution from the near-linear, wave-packet regime to

the strongly nonlinear soliton regime, giving rise to a more interesting range of behaviors within a single mathematical framework.

The specific one-dimensional form of the system used in this work, with its parameterization of the nonlinearity coefficient α and the dispersion coefficient β, was adopted from Martins (2023) and Martins et al. (2024), which is:

$$\begin{cases} \eta_t + u_x + \alpha\,(\eta u)_x = 0, \\ u_t - \dfrac{\beta}{3}\,u_{xxt} + \eta_x + \alpha\,u u_x = 0. \end{cases} \tag{1.1}$$

For this work, the parameters α and β are held equal, defining a one-parameter family of problems whose difficulty grows. Namely, for small $\alpha = \beta$ the dynamics are nearly linear and the solution is well-captured by low-frequency modes, while for large $\alpha = \beta$ the strong coupling between nonlinearity and dispersion produces a more rich dynamics that are fundamentally harder to learn. This controlled variation is precisely what makes the setting useful for comparing SciML architectures, with the goal of observing how each method learns the target solution's frequency.

Classical numerical methods for the Boussinesq system rely on pseudospectral and finite-difference discretizations that exploit the periodic or nearly periodic spatial structure of the waves. Pseudospectral (TREFETHEN, 2000) schemes achieve spectral accuracy on the spatial derivatives and are therefore well suited to the dispersive character of the equations (STEINMOELLER et al., 2012); this is the approach used to generate the reference dataset in this work. On the application side, operational coastal-engineering codes such as FUNWAVE (TODO, 2010) implement Boussinesq-type solvers for harbor and nearshore wave propagation, demonstrating the practical relevance of the model beyond the academic setting.

SciML methods have recently only been applied to the Boussinesq equation, not the specific Boussinesq System. Liu et al. (2023) applied parameter-integrated PINNs with phase-domain decomposition to reconstruct two-dimensional Boussinesq dynamics including phase transitions and rogue-wave formation, identifying spectral bias as a key obstacle at high nonlinearity. Wang et al. (2024) trained a physics-informed network to infer velocity and pressure fields from surface-elevation data alone on a Boussinesq model, demonstrating the inverse-problem capabilities of the approach. Complementary studies on related nonlinear wave equations (ZHANG et al., 2024; KUMAR et al., 2025) have confirmed that purely data-driven and purely physics-driven networks each display characteristic failure modes at strong nonlinearity. Taken together, these results suggest that a systematic, parametric evaluation of FNO/PINO/PINN on the 1D Boussinesq system, which is the focus of this work, can be fruitful to understand better the interactions

between the method's ease of training and nonlinearities and dispersiveness.

1.3 Research Goals

This work implements and evaluates four SciML architectures on the parametric 1D Boussinesq system, arranged into two families of three trained models each. The neural networks family represents a single parameter: a plain multilayer perceptron (MLP) trained on data alone, and a Physics-Informed Neural Network (PINN) trained both with and without supervised data. The operator family learns the whole parameter range in a single training and answers a new parameter with one forward pass: a Fourier Neural Operator (FNO) trained on data alone, and a Physics-Informed Neural Operator (PINO) trained both with and without supervised data. Crossing the two families with the three training regimes, pure data, data and physics, and pure physics, gives the six models compared throughout. A separate strand runs beside them: a minimal network fitting a fixed time Boussinesq elevation profile, whose Neural Tangent Kernel (NTK) is measured directly to isolate the spectral-bias mechanism, exposing results derived . All results are measured against a pseudospectral reference solver, which sets a high-accuracy baseline. The study is guided by the following questions.

- Accuracy relative to the pseudospectral baseline: how closely can each family reproduce the reference solutions as nonlinearity and dispersion grow? Where does each model start to fail, and how does its error scale with the difficulty of the regime? For the operators, does the discretization invariance their spectral layers promise survive a query off the training grid?

- Data-driven, physics-informed, and hybrid training: how does the presence or absence of supervised reference data affect the accuracy and training behavior of each method? Does adding a physics residual term improve generalization, or does it introduce new difficulties?

- Spectral bias across training regimes: how does spectral bias manifest in each architecture when the error spectrum is tracked band by band during training, and to what extent does the inclusion of data or physics constraints alter the frequency at which errors concentrate?

- The mechanism behind spectral bias: does Neural Tangent Kernel (NTK) theory genuinely predict the frequency imbalance seen on the Boussinesq system, or only describe it after the fact? We put the theory to two quantitative tests on a minimal network fitting a single Boussinesq elevation profile at a fixed time, across a range of network widths. The first targets the initial-kernel (lazy-training) assumption the

analysis depends on: we measure how far the network parameters and the empirical kernel drift over training, and whether the kernel's eigenvalue spectrum is essentially unchanged from initialization to final iteration. The second targets the predicted driver of the bias: modes carrying the high-frequency detail are learned far more slowly than low-frequency ones. Testing both, where they hold and where they break, ties the frequency-band errors seen during training to a concrete mechanism instead of a loose analogy.

Together, these questions locate where each family and each training regime holds up on dispersive, nonlinear wave problems and where it fails, and what that implies for choosing among them.

1.4 Work Structure

This work is written with the goal of introducing SciML to senior undergraduates in mathematics, physics, or engineering, that aim to apply these methods in their future research, presenting its pros and cons. We assume the reader has good knowledge of scientific computing and linear algebra. The remaining chapters are organized as follows.

- Chapter 2 develops the mathematical background. It covers the Boussinesq system and its analytical properties, the discrete Fourier transform and the pseudospectral solver built on it, and then the theoretical foundations of the two families, the MLP and the PINN on one side and the FNO and the PINO on the other. It closes with the Neural Tangent Kernel and a derivation of spectral bias mechanism that the later chapters put to the test.

- Chapter 3 describes the experimental methodology. It details the computational setup, the pseudospectral dataset that serves as both training target and ground truth, the error metrics that score every model on one scale, the six trained models on the two-family by three-regime grid, and the minimal experiment that isolates the spectral-bias mechanism.

- Chapter 4 presents the numerical results. It accounts for what each model costs to train, then takes them along one axis at a time: the nonlinearity tests, the multi-resolution tests, and the spectrum compared mode by mode and closes with the NTK measurements and the frequency-band tracking that tie the observed errors to spectral bias for all methods.

- Chapter 5 assesses each family against the questions posed above, states what the study cannot claim, and outlines directions for future work.

2 MATHEMATICAL BACKGROUND

This chapter introduces the mathematical foundations of the methods studied in this work. Section 2.1 presents the Boussinesq system and the limiting wave-equation regime. Section 2.2 develops the pseudospectral numerical method used to generate reference solutions. Sections 2.3 and 2.4 introduce, respectively, the neural network and neural operator architectures studied in this work.

2.1 The Boussinesq System

The Boussinesq system is a weakly nonlinear, weakly dispersive model governing the coupled evolution of two scalar fields in shallow water: the free surface elevation $\eta(x,t)$ and the depth-averaged horizontal velocity $u(x,t)$, both depending on position x and time t. In the one-dimensional, spatially periodic setting studied in this work, the system reads

$$\eta_t + u_x + \alpha\,(\eta u)_x = 0, \tag{2.1a}$$

$$u_t - \frac{\beta}{3}\,u_{xxt} + \eta_x + \alpha\,u u_x = 0, \tag{2.1b}$$

on the spatial domain $x \in [-L, L]$, where $\alpha \geq 0$ is the nonlinearity parameter, $\beta > 0$ is the dispersion parameter, and $L > 0$ is the domain half-length. The two coefficients weigh the competing effects the model balances: α scales the nonlinear terms $(\eta u)_x$ and uu_x, which steepen the wave, while β scales the dispersive term u_{xxt}, which spreads it. This nondimensional form follows equation (3.1) of Martins (2023) (see also Martins et al. (2024)).

The system (2.1) is solved throughout this work from a single solitary profile at rest,

$$\eta(x,0) = A\,\operatorname{sech}^2(x), \qquad u(x,0) = 0, \tag{2.2}$$

where $A > 0$ is the initial amplitude and $\operatorname{sech}(z) = 2/(e^z + e^{-z})$ is the hyperbolic secant. This places one crest at the centre $x = 0$ of an otherwise flat surface, with zero initial velocity, as drawn in Figure 2.1.

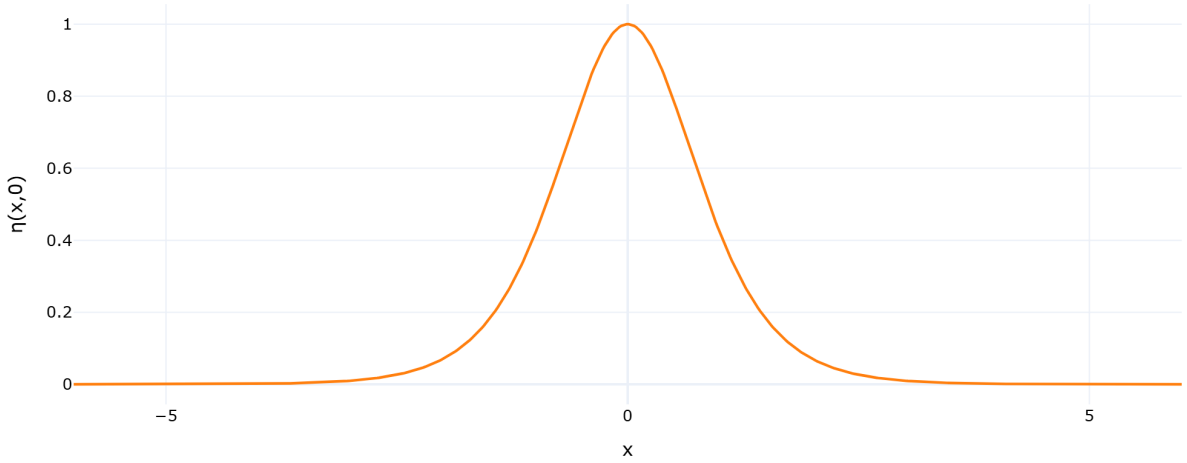


Figure 2.1 – Initial condition $\eta(x,0) = A\,\mathrm{sech}^2(x)$ for $A = 1$.

Source: The author (2026).

2.1.1 Recovering the Wave Equation

When both coefficients vanish, $\alpha = \beta = 0$, the nonlinear and the dispersive terms disappear together and the Boussinesq system reduces to:

$$\eta_t + u_x = 0, \tag{2.3}$$

$$u_t + \eta_x = 0. \tag{2.4}$$

The two fields decouple into a single scalar equation. Differentiating (2.3) with respect to t gives

$$\eta_{tt} + u_{xt} = 0, \tag{2.5}$$

and differentiating (2.4) with respect to x gives

$$u_{tx} + \eta_{xx} = 0. \tag{2.6}$$

Since $u_{xt} = u_{tx}$, subtracting (2.6) from (2.5) cancels the mixed derivative and leaves

$$\eta_{tt} - \eta_{xx} = 0,$$

that is, the wave equation for η,

$$\eta_{tt} = \eta_{xx}. \tag{2.7}$$

In an analogous manner, differentiating (2.4) with respect to t and (2.3) with respect to x, the same cancellation leaves $u_{tt} - u_{xx} = 0$, so u obeys the same equation.

D'Alembert's formula (DEBNATH, 2012) provides the general solution to (2.7).

Given initial data $\eta(x,0) = f(x)$ and $\eta_t(x,0) = g(x)$, the solution is

$$\eta(x,t) = \frac{f(x+t) + f(x-t)}{2} + \frac{1}{2}\int_{x-t}^{x+t} g(s)\,ds. \tag{2.8}$$

We now apply this to the initial condition (2.2). From (2.3), $\eta_t(x,0) = -u_x(x,0) = 0$, so $f(x) = A\,\mathrm{sech}^2(x)$ and $g \equiv 0$. The integral in (2.8) vanishes and the surface elevation evolves as

$$\eta(x,t) = \frac{A}{2}\big[\mathrm{sech}^2(x+t) + \mathrm{sech}^2(x-t)\big]. \tag{2.9}$$

The initial bump of amplitude A therefore splits into two counter-propagating pulses of amplitude $A/2$, travelling at speed ± 1 without changing shape.

For u, from (2.4) we have $u_t(x,0) = -\eta_x(x,0)$, so $f \equiv 0$ and $g(x) = -\eta_x(x,0)$. The f term vanishes in (2.8), leaving

$$u(x,t) = -\frac{1}{2}\int_{x-t}^{x+t} \eta_x(s,0)\,ds = -\frac{1}{2}\big[\eta(x+t,0) - \eta(x-t,0)\big],$$

and substituting $\eta(x,0) = A\,\mathrm{sech}^2(x)$,

$$u(x,t) = \frac{A}{2}\big[\mathrm{sech}^2(x-t) - \mathrm{sech}^2(x+t)\big]. \tag{2.10}$$

Any deviation from this wave-equation baseline in the full system (2.1) is attributable either to dispersion ($\beta > 0$) or to nonlinearity ($\alpha > 0$).

2.2 Pseudospectral Method

The pseudospectral method is applied here to the Boussinesq system (2.1) on the periodic spatial domain $\Omega = [-L, L]$. It transforms the two fields in space alone and leaves time as a separate evolution direction, which is the treatment an evolution equation asks for: taking the DFT along x turns each spatial derivative into an algebraic multiplication, so the PDE collapses into a system of ordinary differential equations in time, one for every spatial frequency. The nonlinearity is what keeps the method from being purely spectral. A product of two fields is not the product of their spectra, so each nonlinear term is instead assembled by returning to the grid, multiplying pointwise there and transforming back. Alternating between the two representations in this way preserves the accuracy of the spatial derivatives while keeping the nonlinear coupling cheap to evaluate.

2.2.1 Spatial Discretization and the Discrete Fourier Transform

The spatial domain $\Omega = [-L, L]$ of total length $2L$ is discretized into N_x equispaced points:

$$x_j = -L + j\,\Delta x, \qquad \Delta x = \frac{2L}{N_x}, \qquad j = 0, \ldots, N_x - 1.$$

The Discrete Fourier Transform (DFT) of a function f sampled at these points is (TREFETHEN, 2000):

$$\hat{f}(k) = \sum_{j=0}^{N_x-1} f(x_j)\, e^{-2\pi i k j / N_x}, \qquad k = -\tfrac{N_x}{2}, \ldots, \tfrac{N_x}{2} - 1, \tag{2.11}$$

with its inverse, the Inverse Discrete Fourier Transform (IDFT), recovering the physical values:

$$f(x_j) = \frac{1}{N_x} \sum_{k} \hat{f}(k)\, e^{2\pi i k j / N_x}. \tag{2.12}$$

The fundamental identity of the DFT with respect to differentiation is that, for a periodic function, the spatial derivative transforms into a pointwise multiplication (TREFETHEN, 2000; STEINMOELLER et al., 2012):

$$\widehat{(f_x)}(k) = i\frac{\pi k}{L}\,\hat{f}(k), \quad \widehat{(f_{xx})}(k) = -\frac{\pi^2 k^2}{L^2}\,\hat{f}(k). \tag{2.13}$$

This is in contrast to finite-difference methods, where the derivative carries a truncation error that shrinks with the grid spacing. Here the error enters at a different point. The identity (2.13) is an exact statement about the trigonometric interpolant of the sampled values, so what remains is the discrepancy between that interpolant and the function itself. For a smooth periodic function this discrepancy decays faster than any fixed power of Δx, which is the spectral accuracy the method is named for (TREFETHEN, 2000).

We define the wavenumber of mode k on $[-L, L]$ as:

$$\xi_k \;:=\; \frac{\pi k}{L}. \tag{2.14}$$

With this definition, the derivative identities read:

$$\widehat{(f_x)}(k) = i\,\xi_k\,\hat{f}(k), \qquad \widehat{(f_{xx})}(k) = -\xi_k^2\,\hat{f}(k). \tag{2.15}$$

2.2.2 Spectral Formulation of the Boussinesq System

To bring the Boussinesq system (2.1) into the Fourier domain, following the spectral formulation of Martins (2023), we apply the DFT to each equation and use the derivative

identities (2.15). Equation (2.1a), treating $\widehat{(\eta u)}(k,t)$ as a given nonlinear term, becomes:

$$\hat{\eta}_t(k,t) = -i\,\xi_k\,\hat{u}(k,t) - \alpha\,i\,\xi_k\,\widehat{(\eta u)}(k,t). \tag{2.16}$$

Equation (2.1b) requires more care because of the u_{xxt} term. Since spatial derivatives commute with the DFT:

$$\widehat{(u_{xxt})}(k,t) = -\xi_k^2\,\hat{u}_t(k,t).$$

Substituting into the DFT of (2.1b), with its nonlinear term written in the equivalent form $uu_x = \frac{1}{2}(u^2)_x$:

$$\hat{u}_t(k,t) + \frac{\beta}{3}\,\xi_k^2\,\hat{u}_t(k,t) + i\,\xi_k\,\hat{\eta}(k,t) + \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t) = 0.$$

Collecting the $\hat{u}_t(k,t)$ terms on the left and solving algebraically:

$$\hat{u}_t(k,t) = \frac{-i\,\xi_k\,\hat{\eta}(k,t) - \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t)}{1 + \dfrac{\beta\,\xi_k^2}{3}}. \tag{2.17}$$

The division in (2.17) is where the transform pays off. Isolating u_t in (2.1b) means inverting the operator $1 - \frac{\beta}{3}\frac{\partial^2}{\partial x^2}$, and on the grid that operator couples neighbouring nodes, so inverting it amounts to solving a linear system for all N_x nodal values at every evaluation of the right-hand side. Under the DFT the same operator is diagonal, acting on mode k as multiplication by $1 + \frac{\beta\,\xi_k^2}{3}$, and the inversion reduces to one division per mode.

Equations (2.16) and (2.17) can be written jointly as a first-order ODE system in time for each fixed wavenumber $k \in \mathbb{Z}$:

$$\begin{cases} \hat{\eta}_t(k,t) = -i\,\xi_k\,\hat{u}(k,t) - \alpha\,i\,\xi_k\,\widehat{(\eta u)}(k,t), \\[2ex] \hat{u}_t(k,t) = \dfrac{-i\,\xi_k\,\hat{\eta}(k,t) - \alpha\,i\,\xi_k\widehat{\left(\tfrac{1}{2}u^2\right)}(k,t)}{1 + \dfrac{\beta\,\xi_k^2}{3}}. \end{cases} \tag{2.18}$$

The PDE system (2.1), two equations in (x,t), is thus equivalent to an infinite family of two-dimensional ODE systems (2.18), one for each $k \in \mathbb{Z}$, coupled through the nonlinear terms.

The nonlinear terms $\widehat{(\eta u)}(k,t)$ and $\widehat{(\frac{1}{2}u^2)}(k,t)$ cannot be computed directly in Fourier space without a convolution sum, which costs $O(N_x^2)$. The pseudospectral approach avoids this by computing products in physical space. For $\widehat{(\eta u)}(k,t)$, the procedure

is:

1. recover $\eta(x_j)$ and $u(x_j)$ by applying the inverse DFT (2.12);
2. compute the product $(\eta u)_j = \eta(x_j) \cdot u(x_j)$ pointwise on the grid;
3. apply the DFT (2.11) to obtain $\widehat{(\eta u)}(k,t)$.

Similarly, $\widehat{(\frac{1}{2}u^2)}(k,t)$ is obtained by the same three-step procedure with $u^2(x_j) = u(x_j)^2$, where $u(x_j) = \mathrm{IDFT}(\hat{u})(x_j)$; and $(\eta u)(x_j) = \mathrm{IDFT}(\hat{\eta})(x_j) \cdot \mathrm{IDFT}(\hat{u})(x_j)$. Since each DFT/IDFT costs $O(N_x \log N_x)$ via the Fast Fourier Transform (FFT), this pseudospectral strategy assembles each nonlinear term in $O(N_x \log N_x)$ instead of $O(N_x^2)$.

The nonlinear term of (2.1b) fits this procedure only in the form it was given in (2.17). Since $uu_x = \frac{1}{2}(u^2)_x$, the two forms are equivalent, but only the second is a product of a field with itself: recovering u on the grid, squaring it pointwise and transforming back costs one inverse and one forward transform, whereas uu_x would require recovering u and u_x separately before multiplying them. Both forms appear in this work. The solver uses $\frac{1}{2}(u^2)_x$ for exactly this reason, while the physics-informed network of Section 2.3.5 differentiates uu_x directly by automatic differentiation, where the products are never assembled on a grid and the distinction carries no cost.

2.2.3 Time Evolution via Runge-Kutta 4

Collecting the two spectral fields into a single state vector $\hat{Y}(t) = \big(\hat{\eta}(k,t),\, \hat{u}(k,t)\big)$, the system (2.18) takes the standard form of an initial value problem for a system of ordinary differential equations,

$$\hat{Y}_t(t) = F\big(\hat{Y}(t)\big), \tag{2.19}$$

where the pseudospectral right-hand side operator F is the map

$$F(\hat{\eta}, \hat{u})(k) \;=\; \begin{pmatrix} -i\,\xi_k\,\hat{u}(k) - \alpha\, i\,\xi_k\,\widehat{(\eta u)}(k) \\ \dfrac{-i\,\xi_k\,\hat{\eta}(k) - \alpha\, i\,\xi_k\,\widehat{\big(\frac{1}{2}u^2\big)}(k)}{1 + \dfrac{\beta\,\xi_k^2}{3}} \end{pmatrix},$$

whose two components are $\hat{\eta}_t$ and $\hat{u}_t$ as given by equations (2.16)–(2.17), with the nonlinear terms assembled by the pseudospectral procedure above. In the form (2.19) the problem is what any explicit one-step integrator expects, and time evolution uses the classical fourth-order Runge-Kutta (RK4) scheme (SÜLI et al., 2003; BURDEN et al., 2011). Given the state $\hat{Y}_n = (\hat{\eta}^n, \hat{u}^n)$ at time t_n, the four stage evaluations $\kappa_1, \kappa_2, \kappa_3, \kappa_4$ and the update to

$t_{n+1} = t_n + \Delta t$ are:

$$\begin{aligned}
\kappa_1 &= F(\hat{Y}_n), \\
\kappa_2 &= F\left(\hat{Y}_n + \tfrac{\Delta t}{2}\,\kappa_1\right), \\
\kappa_3 &= F\left(\hat{Y}_n + \tfrac{\Delta t}{2}\,\kappa_2\right), \\
\kappa_4 &= F\left(\hat{Y}_n + \Delta t\,\kappa_3\right), \\
\hat{Y}_{n+1} &= \hat{Y}_n + \frac{\Delta t}{6}\,(\kappa_1 + 2\,\kappa_2 + 2\,\kappa_3 + \kappa_4).
\end{aligned}$$

At each stage, the intermediate state is decoded to physical space to assemble the nonlinear products, then encoded back to Fourier space to evaluate the spectral derivatives. This split between physical-space products and Fourier-space derivatives is what characterizes the pseudospectral method.

2.2.4 Training Dataset

The numerical experiments in this work vary only the PDE parameters (α, β) while keeping the initial condition fixed at (2.2). The models are therefore trained and evaluated on the dataset

$$\mathcal{S} = \left\{ \big((\alpha_i, \beta_i),\ (\eta_i, u_i)\big) \right\}_{i=1}^{M}, \tag{2.20}$$

where (α_i, β_i) are the nonlinearity and dispersion parameters for the i-th sample, and (η_i, u_i) is the corresponding space-time solution obtained from the pseudospectral solver of Section 2.2 with those parameters and the shared initial condition (2.2).

2.3 Neural Networks

This section introduces the neural network architectures central to this work, together with the theory that explains how they train. We begin with the Multilayer Perceptron, the base model underlying every architecture studied here. We then develop the Neural Tangent Kernel. That kernel is the tool we use to analyse the spectral bias a network produces when it is fitted to data sampled on a regular grid. Finally we turn to Physics-Informed Neural Networks. There we show that the same kernel analysis carries over to the physics-informed setting, where spectral bias is a well-documented obstacle.

2.3.1 Multilayer Perceptrons

In this work, we focus on Multilayer Perceptrons (MLPs), popularized by Rumelhart et al. (1986). These networks are compositions of affine transformations parametrized

by tunable weights and biases, separated by nonlinear activations, where an optimization process fits the network to a dataset.

Given a labeled dataset $\{(x_i, u_i)\}_{i=1}^{N}$, the objective is to fit the function $u_\theta(x)$ by minimizing a loss function, here the Mean Squared Error (MSE). With $\|\cdot\|$ denoting the Euclidean norm, the unconstrained optimization problem is:

$$\mathcal{L}_{\text{MLP}}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \|u_\theta(x_i) - u_i\|^2. \tag{2.21}$$

The MLP $u_\theta : \mathbb{R}^{d_{\text{in}}} \to \mathbb{R}^{d_{\text{out}}}$ is defined by the composition:

$$h^{(0)} = x \in \mathbb{R}^{d_{\text{in}}}, \tag{2.22}$$

$$h^{(l)} = \sigma\left(W^{(l)}h^{(l-1)} + b^{(l)}\right) \in \mathbb{R}^{N_{\text{MLP}}}, \quad l = 1, \ldots, L_{\text{MLP}} - 1, \tag{2.23}$$

$$u_\theta(x) = W^{(L_{\text{MLP}})}h^{(L_{\text{MLP}}-1)} + b^{(L_{\text{MLP}})} \in \mathbb{R}^{d_{\text{out}}}. \tag{2.24}$$

where:

- $L_{\text{MLP}} \in \mathbb{N}$ is the number of layers and $N_{\text{MLP}} \in \mathbb{N}$ the number of neurons per hidden layer;
- $W^{(l)} \in \mathbb{R}^{N_{\text{MLP}} \times N_{\text{MLP}}}$ and $b^{(l)} \in \mathbb{R}^{N_{\text{MLP}}}$ are the weight matrix and bias vector at layer l, with $W^{(1)} \in \mathbb{R}^{d_{\text{in}} \times N_{\text{MLP}}}$ and $W^{(L_{\text{MLP}})} \in \mathbb{R}^{N_{\text{MLP}} \times d_{\text{out}}}$ adapted at the boundaries;
- $\sigma : \mathbb{R} \to \mathbb{R}$ is a sigmoidal activation function applied elementwise, satisfying $\sigma(t) \to 1$ as $t \to +\infty$ and $\sigma(t) \to 0$ as $t \to -\infty$;
- $\theta = \{W^{(l)}, b^{(l)}\}_{l=1}^{L_{\text{MLP}}}$ denotes all trainable parameters.

If the activation function is sigmoidal, MLPs are universal approximators: they can approximate any continuous function on a compact domain to arbitrary precision (CYBENKO, 1989).

In Figure 2.2, we show a common depiction of the neural network. Here the input vector is the pair $(x, t) \in \mathbb{R}^2$ and the output is $(\eta_\theta(x, t), u_\theta(x, t)) \in \mathbb{R}^2$.

2.3.2 Gradient Flow

The analysis of the following sections is carried out on the continuous-time limit of gradient descent, the form in which the neural-tangent-kernel results this work relies on are stated (JACOT et al., 2018; WANG; YU, et al., 2022). This subsection derives that limit.

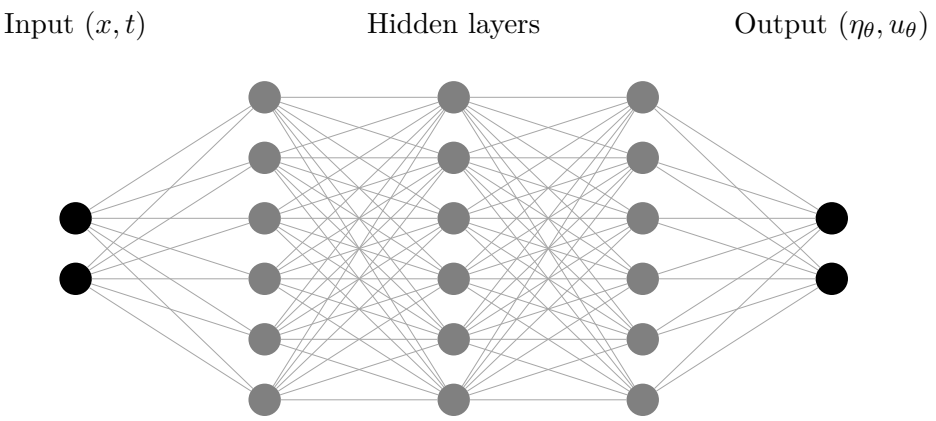


Figure 2.2 – Example of MLP architecture of input (x, t), with $L_{\text{MLP}} = 3$ hidden layers and $N_{\text{MLP}} = 6$ neurons per layer.

Source: The author (2026).

To understand how an MLP minimizes its data-fitting loss (2.21), consider gradient descent with learning rate $\mu > 0$. Starting from an initialization $\theta^{(0)}$, each iteration updates the parameters by

$$\theta^{(k+1)} = \theta^{(k)} - \mu \nabla_\theta \mathcal{L}(\theta^{(k)}), \qquad k = 0, 1, 2, \ldots, \tag{2.25}$$

where $\theta^{(k)}$ denotes the parameter vector after k steps. Rearranging (2.25) and dividing by the step size μ isolates the per-step increment as a difference quotient:

$$\frac{\theta^{(k+1)} - \theta^{(k)}}{\mu} = -\nabla_\theta \mathcal{L}(\theta^{(k)}). \tag{2.26}$$

The left-hand side of (2.26) already has the shape of a derivative: a difference of two parameter vectors divided by a spacing. It is not one yet. The index k counts iterations and carries no unit of time, so there is nothing for the difference to be taken with respect to. Reading (2.26) as the discretization of an ordinary differential equation therefore requires a new, continuous parameter along which the iterates can be laid out.

The idea is to regard the discrete sequence $\{\theta^{(k)}\}_{k=0}^{\infty}$ as samples of a smooth curve $\theta(\tau)$, read at a set of instants $\tau^{(k)}$,

$$\theta^{(k)} = \theta(\tau^{(k)}), \tag{2.27}$$

so that the recursion becomes a statement about $\theta(\tau)$ rather than about the index k. What remains is to fix the instants. The step size μ supplies their spacing: we assign to

the k-th iterate the training-time stamp

$$\tau^{(k)} = k\mu, \qquad \text{so that} \qquad \tau^{(k+1)} - \tau^{(k)} = \mu, \tag{2.28}$$

which places successive iterates μ apart along a time axis τ and gives $\theta^{(k+1)} = \theta(\tau^{(k)} + \mu)$. Substituting into the left-hand side of (2.26) recasts it as a forward finite-difference quotient of θ in τ:

$$\frac{\theta(\tau^{(k)} + \mu) - \theta(\tau^{(k)})}{\mu} = -\nabla_\theta \mathcal{L}\big(\theta(\tau^{(k)})\big). \tag{2.29}$$

As the step size $\mu \to 0$, the spacing in (2.28) shrinks to zero, the discrete stamps $\tau^{(k)}$ fill out a continuous variable τ, and the left-hand side of (2.29) converges to the derivative $d\theta/d\tau$ by definition of the derivative. The recursion thus becomes the gradient flow ODE:

$$\frac{d\theta}{d\tau} = -\nabla_\theta \mathcal{L}(\theta), \qquad \theta(0) = \theta^{(0)}. \tag{2.30}$$

The gradient flow (2.30) is a central object in optimization, and many standard first-order methods can be recovered as different time-discretization schemes of this same continuous trajectory. The discrete update (2.25) is precisely the forward (explicit) Euler discretization of (2.30) with step size μ (SÜLI et al., 2003; BURDEN et al., 2011), so gradient descent is just one numerical integrator for the flow; applying instead the backward (implicit) Euler scheme (SÜLI et al., 2003) recovers the proximal gradient method (PARIKH et al., 2013). Reading optimizers as discretizations of (2.30) thus organizes them into a single family, and it is the continuous flow that we analyze in what follows.

2.3.3 Neural Tangent Kernel

The kernel derived in this subsection was introduced by Jacot et al. (2018) and carried to the physics-informed setting by Wang, Yu, et al. (2022); the derivation below follows their treatment, specialized to the data-fitting loss (2.21). Consider the MLP fit to the dataset $\{(x_i, u_i)\}_{i=1}^N$ of Section 2.3.1. Group the pointwise fitting errors into a single error vector:

$$e(\theta) = \begin{bmatrix} u_\theta(x_1) - u_1 \\ \vdots \\ u_\theta(x_N) - u_N \end{bmatrix} \in \mathbb{R}^N. \tag{2.31}$$

Up to the constant factor $1/N$, the loss (2.21) is the squared norm of this vector, which we use in the form

$$\mathcal{L}(\theta) = \frac{1}{2}\,\|e(\theta)\|^2 = \frac{1}{2}\sum_{i=1}^{N} e_i(\theta)^2, \tag{2.32}$$

where $e_i(\theta) = u_\theta(x_i) - u_i$ is the i-th component of the error vector (2.31). Define the Jacobian of the error vector with respect to the parameters:

$$J(\theta) = \frac{\partial e}{\partial \theta} \in \mathbb{R}^{N \times N_\theta}, \qquad J_{ij} = \frac{\partial e_i}{\partial \theta_j}, \quad i = 1, \dots, N,\ j = 1, \dots, N_\theta, \tag{2.33}$$

where N_θ is the total number of trainable parameters. Each row $J_i = (J_{i1}, \dots, J_{iN_\theta}) \in \mathbb{R}^{N_\theta}$ is the parameter-gradient of the i-th residual. Since $\mathcal{L}$ is the quadratic form (2.32), the j-th component of the loss gradient is:

$$\frac{\partial \mathcal{L}}{\partial \theta_j} = \sum_{i=1}^{N} e_i(\theta)\, \frac{\partial e_i}{\partial \theta_j} = \sum_{i=1}^{N} J_{ij}\, e_i(\theta) = \big(J(\theta)^T e(\theta)\big)_j. \tag{2.34}$$

Collecting all $j = 1, \dots, N_\theta$ into a column vector:

$$\nabla_\theta \mathcal{L} = J(\theta)^T\, e(\theta) \in \mathbb{R}^{N_\theta}. \tag{2.35}$$

Substituting (2.35) into the gradient flow (2.30):

$$\frac{d\theta}{d\tau} = -J(\theta)^T\, e(\tau). \tag{2.36}$$

The error is written here as a function of τ rather than of θ. The two notations denote the same object: along the flow the parameters are themselves a function of training time, so $e(\tau) := e(\theta(\tau))$, and it is this composition whose evolution we now track. To find how the error vector itself evolves, differentiate each component $e_i(\theta(\tau))$ with respect to τ by the chain rule:

$$\frac{de_i}{d\tau} = \sum_{j=1}^{N_\theta} \frac{\partial e_i}{\partial \theta_j}\, \frac{d\theta_j}{d\tau} = J_i(\theta) \cdot \frac{d\theta}{d\tau}. \tag{2.37}$$

Stacking this identity for all $i = 1, \dots, N$ into a matrix equation:

$$\frac{de}{d\tau} = J(\theta)\, \frac{d\theta}{d\tau}. \tag{2.38}$$

Substituting (2.36):

$$\frac{de}{d\tau} = J(\theta)\big(-J(\theta)^T e(\tau)\big) = -\underbrace{J(\theta)\, J(\theta)^T}_{K(\theta) \in \mathbb{R}^{N \times N}}\, e(\tau). \tag{2.39}$$

The matrix $K(\theta) = J(\theta)J(\theta)^T$ is the empirical Neural Tangent Kernel (NTK) (JACOT et al., 2018). It is symmetric positive semi-definite by construction: for any $v \in \mathbb{R}^N$,

$$v^T K(\theta)\, v = v^T J(\theta)\, J(\theta)^T v = \left\| J(\theta)^T v \right\|^2 \geq 0. \tag{2.40}$$

The (i, j) entry of $K(\theta)$ follows directly from expanding the matrix product $J(\theta)J(\theta)^T$:

$$K_{ij}(\theta) = \sum_{l=1}^{N_\theta} J_{il}(\theta)\, J_{jl}(\theta) = \sum_{l=1}^{N_\theta} \frac{\partial e_i}{\partial \theta_l}(\theta)\, \frac{\partial e_j}{\partial \theta_l}(\theta) = \nabla_\theta e_i(\theta)^T\, \nabla_\theta e_j(\theta). \tag{2.41}$$

This is the inner product in $\mathbb{R}^{N_\theta}$ of the parameter gradients of the network output at the i-th and j-th training points, where $e_i = u_\theta(x_i) - u_i$. Hence $K(\theta) \in \mathbb{R}^{N \times N}$ is the Gram matrix of the network's sensitivities across the whole dataset, and equation (2.39) shows that it alone governs how the fitting error evolves under gradient flow.

2.3.4 Spectral Bias

The spectral bias (or frequency principle) is the empirical observation that neural networks trained by gradient-based methods reduce the low-frequency components of the error faster than the high-frequency ones (RAHAMAN et al., 2019). This section derives that phenomenon directly from the NTK dynamics established above. When the training points lie on a regular grid and the target function is periodic, the error admits a discrete Fourier representation, and one can ask how its frequency content is approximated along the training iterations. We first establish the decay in the eigenbasis of the NTK and then, by transforming to Fourier space, show that it is precisely the low frequencies that are resolved first.

For a network with a sufficiently large number of parameters, the Jacobian $J(\theta)$ changes negligibly during training, a regime known as lazy training (JACOT et al., 2018; CHIZAT et al., 2019; WANG; YU, et al., 2022). The motivation for it is a scaling argument. In the wide-network limit the output at any point is an aggregate of many hidden units, each contributing a vanishing share of it as the width grows. An $O(1)$ change in the function is then reached by many parameters each moving very little, instead of a few moving a lot. Since the Jacobian is itself a function of the parameters, it barely moves either, and in the infinite-width limit it stays frozen at its value at initialization (JACOT et al., 2018; CHIZAT et al., 2019). The networks trained here are finite, so this is an approximation, not an identity, and how far it holds is measured directly in Chapter 4. Writing $J_0 = J(\theta(0))$ and $K_0 = J_0 J_0^T$, the approximation $K(\theta(\tau)) \approx K_0$ turns

equation (2.39) into a linear ODE with constant coefficients:

$$\frac{de}{d\tau} = -K_0\, e(\tau), \qquad e(0) = e_0. \tag{2.42}$$

Written component-wise, the i-th row of this system is

$$\frac{de_i}{d\tau} = -\sum_{j=1}^{N} K_0^{ij}\, e_j(\tau), \qquad i = 1, \dots, N, \tag{2.43}$$

where $K_0^{ij} = \nabla_{\theta_0} e_i^T \nabla_{\theta_0} e_j$ is the inner product, at initialization, of the parameter gradients of the network output at training points x_i and x_j. The rate of change of the error at point i is therefore a weighted sum of all current errors, with weights given by these pairwise kernel values. Through this coupling, the geometry of the sample grid and the architecture of the network jointly determine the convergence of every component.

Since $K_0 \in \mathbb{R}^{N\times N}$ is symmetric and positive semi-definite, the spectral theorem for symmetric matrices (STRANG, 2006; GOLUB et al., 2013) guarantees a decomposition

$$K_0 = V\, \Lambda\, V^T, \tag{2.44}$$

where $V = [v_1 \mid \cdots \mid v_N]$ is an orthogonal matrix ($V^T V = I$) and

$$\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_N), \qquad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0. \tag{2.45}$$

Introduce the change of coordinates

$$c(\tau) = V^T e(\tau), \tag{2.46}$$

so that, since V is orthogonal, $e(\tau) = V c(\tau)$. Differentiating (2.46) with respect to τ and substituting the ODE (2.42):

$$\frac{dc}{d\tau} = V^T \frac{de}{d\tau} = -V^T K_0\, e(\tau). \tag{2.47}$$

Inserting $K_0 = V\Lambda V^T$ and $e(\tau) = Vc(\tau)$:

$$\frac{dc}{d\tau} = -V^T\,(V\Lambda V^T)\,(Vc(\tau)) = -(V^T V)\,\Lambda\,(V^T V)\,c(\tau) = -\Lambda\, c(\tau), \tag{2.48}$$

using $V^T V = I$ at each step. Since Λ is diagonal, this decouples into N independent scalar equations:

$$\frac{dc_k}{d\tau} = -\lambda_k\, c_k(\tau), \qquad k = 1, \dots, N. \tag{2.49}$$

Each equation in (2.49) is a first-order linear ODE with constant coefficient (BOYCE et al., 2012). Separating the variables and integrating both sides from the initial state at $\tau = 0$ up to time τ, with s and r as integration variables for the amplitude and the time,

$$\int_{c_k(0)}^{c_k(\tau)} \frac{ds}{s} = -\lambda_k \int_0^{\tau} dr \qquad \Longrightarrow \qquad \ln \frac{|c_k(\tau)|}{|c_k(0)|} = -\lambda_k \, \tau. \tag{2.50}$$

Exponentiating both sides yields the solution

$$c_k(\tau) = c_k(0)\, e^{-\lambda_k \tau}, \tag{2.51}$$

where $c_k(0) = v_k^T e_0$ is the projection of the initial error e_0 onto the k-th eigenvector of K_0. For $\lambda_k = 0$ the equation gives $c_k(\tau) = c_k(0)$ for all τ: that eigendirection never decays.

Returning to the original coordinates via $e(\tau) = Vc(\tau)$:

$$e(\tau) = \sum_{k=1}^{N} \big(v_k^T e_0\big)\, e^{-\lambda_k \tau}\, v_k. \tag{2.52}$$

The error decomposes into N independent eigendirections. Each eigenvector v_k carries initial amplitude $v_k^T e_0$ and decays exponentially at rate λ_k: eigenvectors with large λ_k are suppressed quickly, while those with small λ_k persist throughout training. This per-eigendirection decay, each at the rate set by its eigenvalue, is the rigorous statement of spectral bias established by Cao et al. (2021); the same spectral ordering governs generalization as the training set grows (BORDELON et al., 2020).

Equation (2.52) is the continuous flow; gradient descent takes finite steps, and its error evolution inherits the same eigenstructure in discrete form. The link between the two is the training-time stamp of (2.28): iterate n sits at continuous time $\tau^{(n)} = n\mu$, so the iteration count and the training time are read off one another through

$$\tau = n\mu, \qquad n = \tau/\mu. \tag{2.53}$$

Section 2.3.2 read the update (2.25) as forward Euler applied to the gradient flow in the parameters. The same statement is needed for the error, since that is what the analysis follows. There are two ways to obtain it: discretize the parameter update and then linearize, or linearize first, into the error equation (2.42), and discretize that. Both give the same law.

Take the first. Gradient descent on the loss is

$$\theta^{(n+1)} = \theta^{(n)} - \mu\, \nabla_\theta \mathcal{L}\big(\theta^{(n)}\big).$$

Under lazy training the Jacobian is frozen at J_0, so (2.35) gives $\nabla_\theta \mathcal{L}(\theta^{(n)}) = J_0^T e^{(n)}$ and the step is

$$\Delta\theta^{(n)} := \theta^{(n+1)} - \theta^{(n)} = -\mu\, J_0^T e^{(n)}.$$

Expanding the error to first order in that displacement,

$$e^{(n+1)} = e^{(n)} + J_0\, \Delta\theta^{(n)} + O\big(\|\Delta\theta^{(n)}\|^2\big),$$

and substituting the step,

$$e^{(n+1)} = e^{(n)} - \mu\, J_0 J_0^T e^{(n)}.$$

Now the second. Equation (2.42) is already linear in e. The forward, or explicit, Euler scheme (SÜLI et al., 2003; LEVEQUE, 2007) replaces its derivative at $\tau^{(n)}$ by a forward difference over one step of length μ,

$$\left.\frac{de}{d\tau}\right|_{\tau^{(n)}} \approx \frac{e^{(n+1)} - e^{(n)}}{\mu},$$

and evaluates the right-hand side at the current iterate,

$$\frac{e^{(n+1)} - e^{(n)}}{\mu} = -K_0\, e^{(n)}.$$

Multiplying through by μ,

$$e^{(n+1)} = e^{(n)} - \mu\, K_0\, e^{(n)}.$$

Since $K_0 = J_0 J_0^T$, the two results are the same equation. Collecting terms gives the single-step law

$$e^{(n+1)} = (I - \mu K_0)\, e^{(n)}. \tag{2.54}$$

One gradient-descent step of size μ is one forward-Euler step of length μ on the error. Iterating from $e^{(0)} = e_0$ and diagonalizing with $K_0 = V \Lambda V^T$,

$$e^{(n)} = (I - \mu K_0)^n e_0 = \sum_{k=1}^{N} \big(v_k^T e_0\big)\, (1 - \mu\lambda_k)^n\, v_k. \tag{2.55}$$

The continuous solution reaches the same expression, one mode at a time. Evaluate the decay factor of (2.52) at $\tau = n\mu$ and split it across the n steps,

$$e^{-\lambda_k \tau} = e^{-\lambda_k n \mu} = \big(e^{-\lambda_k \mu}\big)^n,$$

then expand the per-step factor and truncate at first order in $\mu\lambda_k$ (BURDEN et al., 2011),

$$e^{-\lambda_k \mu} = 1 - \mu\lambda_k + O\big((\mu\lambda_k)^2\big) \approx 1 - \mu\lambda_k. \tag{2.56}$$

Substituting term by term carries (2.52) into (2.55), so $(1 - \mu\lambda_k)^n$ is the forward-Euler counterpart of $e^{-\lambda_k \tau}$. The two agree as the step vanishes with $\tau = n\mu$ held fixed,

$$\lim_{\mu \to 0} \big(1 - \mu\lambda_k\big)^{\tau/\mu} = e^{-\lambda_k \tau}.$$

The same truncation constrains the step. Under the continuous flow every mode with $\lambda_k > 0$ decays for any $\mu > 0$. The discrete mode contracts only when its amplification factor is smaller than one in magnitude,

$$\big|1 - \mu\lambda_k\big| < 1 \qquad \Longleftrightarrow \qquad 0 < \mu\lambda_k < 2, \tag{2.57}$$

which is the absolute-stability condition of the explicit Euler method (LEVEQUE, 2007; SÜLI et al., 2003). The product $\mu\lambda_k$ alone fixes how the k-th mode behaves, and it does so in four regimes. Below one, the factor lies in $(0, 1)$ and the mode decays monotonically toward zero; at exactly one it vanishes and the mode is removed in a single step. Between one and two the factor is negative, so the mode still decays but flips sign at every step. Past two it exceeds one in magnitude and the mode grows without bound. A single step size therefore acts differently on every eigendirection of K_0.

Since $\lambda_1 = \lambda_{\max}$ is the largest eigenvalue, every mode stays stable only when the largest one does, which caps the learning rate at

$$\mu < \frac{2}{\lambda_{\max}}. \tag{2.58}$$

The minimal experiment of Section 3.5 sets its step from this limit: rather than a nominal value, μ is fixed at a constant fraction of $2/\lambda_{\max}$ at every width, small enough to keep every mode in the monotone regime $\mu\lambda_k < 1$ while staying comparable across widths whose $\lambda_{\max}$ differ.

The same argument gives the time each mode needs. From (2.51), the amplitude of the k-th eigendirection at time τ is

$$\big|c_k(\tau)\big| = \big|v_k^T e_0\big|\, e^{-\lambda_k \tau}. \tag{2.59}$$

Require it to fall below a target precision $\varepsilon > 0$,

$$\big|v_k^T e_0\big|\, e^{-\lambda_k \tau} < \varepsilon,$$

and divide by $\left|v_k^T e_0\right| > 0$,

$$e^{-\lambda_k \tau} < \frac{\varepsilon}{\left|v_k^T e_0\right|}.$$

Taking logarithms preserves the inequality,

$$-\lambda_k \, \tau < \ln \frac{\varepsilon}{\left|v_k^T e_0\right|},$$

and dividing by $-\lambda_k < 0$ reverses it,

$$\tau > \frac{1}{\lambda_k} \, \ln \frac{\left|v_k^T e_0\right|}{\varepsilon}.$$

The minimum training time to resolve the k-th eigendirection to precision ε is therefore

$$\tau_k = \frac{1}{\lambda_k} \, \ln \frac{\left|v_k^T e_0\right|}{\varepsilon}. \tag{2.60}$$

The projection enters (2.60) through a logarithm and the eigenvalue through $1/\lambda_k$, so λ_k sets the scale of τ_k while the projection only shifts it. Since $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_N \geq 0$, the times are ordered $\tau_1 \leq \tau_2 \leq \cdots \leq \tau_N$: higher-indexed eigendirections require more training time, in inverse proportion to their eigenvalue. Arora et al. (2019) reached similar conclusions from a discrete-time analysis of gradient descent, showing that the convergence speed of each eigenvector is governed by its eigenvalue and by the projection of the target onto that eigenvector.

The decomposition (2.52) is expressed in the eigenbasis $\{v_k\}_{k=1}^N$ of the NTK, which need not coincide with any physically meaningful basis. When the N training points lie on a regular grid and the target is periodic, however, the natural basis is the Fourier one, and recasting the dynamics there shows that the decay is genuinely organized by frequency. Let $F \in \mathbb{C}^{N \times N}$ be the Fourier matrix, that is, the discrete Fourier transform (DFT) matrix with entries

$$F_{jl} = \frac{1}{\sqrt{N}} \, \omega^{jl}, \qquad \omega = e^{-2\pi \mathrm{i}/N}, \qquad j, l = 0, \ldots, N-1. \tag{2.61}$$

With this normalization F is unitary, $F^* F = I$, so that $F^{-1} = F^*$, where F^* is the conjugate transpose (GOLUB et al., 2013; STRANG, 2006). Applying F to the error vector yields its spectrum, with inverse transform F^{-1}:

$$\hat{e}(\tau) = F \, e(\tau), \qquad e(\tau) = F^{-1} \hat{e}(\tau) = F^* \hat{e}(\tau). \tag{2.62}$$

In particular $\hat{e}_0 = F e_0$ is the spectrum of the initial error, the difference between the

spectra of the initial network output and the target. Since F is constant in τ, applying it to the linearized dynamics (2.42) carries the error evolution into the frequency domain:

$$\frac{d\hat{e}}{d\tau} = F\,\frac{de}{d\tau} = -FK_0\,e(\tau) = -FK_0F^{-1}\,\hat{e}(\tau). \tag{2.63}$$

Inserting the eigendecomposition $K_0 = V\Lambda V^T$ from (2.44),

$$FK_0F^{-1} = FV\,\Lambda\,V^TF^{-1} = (FV)\,\Lambda\,(FV)^* = (FV)\,\Lambda\,(FV)^{-1}, \tag{2.64}$$

where $V^TF^{-1} = V^*F^* = (FV)^*$ uses that V is real orthogonal, and $(FV)^* = (FV)^{-1}$ uses that a product of unitary matrices is unitary. The columns of FV, namely the Fourier transforms $\{Fv_k\}_{k=1}^N$ of the NTK eigenvectors, therefore diagonalize the frequency-domain dynamics. Defining the modal amplitudes $\tilde{c}(\tau) = (FV)^{-1}\hat{e}(\tau)$ and repeating the steps (2.46)–(2.51) gives

$$\frac{d\tilde{c}_k}{d\tau} = -\lambda_k\,\tilde{c}_k(\tau) \qquad \Longrightarrow \qquad \tilde{c}_k(\tau) = \tilde{c}_k(0)\,e^{-\lambda_k\tau}. \tag{2.65}$$

These are the same amplitudes as before: $\tilde{c} = (FV)^{-1}Fe = V^{-1}e = V^Te = c$, the projection of the error onto the NTK eigenvectors, now read in Fourier space. Transforming back through (2.62) yields the evolution of the error spectrum,

$$\hat{e}(\tau) = \sum_{k=1}^{N}\big[(Fv_k)^*\hat{e}_0\big]\,e^{-\lambda_k\tau}\,(Fv_k). \tag{2.66}$$

Equation (2.66) is the spectral-bias statement in the literal sense of the frequency spectrum: the error spectrum is a superposition of the transformed eigenvectors Fv_k, each damped at its own rate λ_k. The frequencies resolved first are those carried by the eigenvectors with the largest eigenvalues; empirically these eigenvectors are smooth and concentrated in the low Fourier modes (RAHAMAN et al., 2019; XU et al., 2020), so the low-frequency content of the error decays first while the high-frequency content persists.

To close, we specialize the initial condition. The weights of an MLP are drawn at random from a distribution centred at zero, so at small initialization scale $\theta_0 \approx 0$ and the network output u_{θ_0} is close to zero across the whole grid. Set against a target u of order one, it is negligible. This is a common simplification in spectral-bias analyses, and under

it the initial error is the target with opposite sign, as is its spectrum:

$$\hat{e}_0 = F\big(u_{\theta_0} - u\big) = \hat{u}_{\theta_0} - \hat{u} \approx -\,\hat{u} = -\begin{bmatrix} \hat{u}(0) \\ \vdots \\ \hat{u}(N-1) \end{bmatrix}. \tag{2.67}$$

Substituting (2.67) into (2.66) casts the dynamics as a reconstruction of the target frequency by frequency: each Fourier mode of u is acquired at a rate set by the NTK eigenvalues, the smooth low-frequency content emerging first and the fine high-frequency detail last. This is the regime anticipated at the start of the section, a periodic target on a regular grid, in which the quality of the approximation can be tracked mode by mode along the training iterations. It is also the setting we implement: the minimal experiment of Chapter 3 fits a single Boussinesq elevation profile on exactly such a grid, so that (2.67) holds by construction and the mode-by-mode reconstruction can be measured against the prediction.

This last observation can be made quantitative. Taking the modulus of the modal amplitude (2.65) isolates the magnitude of the k-th component along training,

$$|\tilde{c}_k(\tau)| = |\tilde{c}_k(0)|\, e^{-\lambda_k \tau}, \qquad |\tilde{c}_k(0)| = |(Fv_k)^* \hat{e}_0| \approx |(Fv_k)^* \hat{u}|, \tag{2.68}$$

where the last estimate uses the small-scale initialization (2.67), so that $|(Fv_k)^*\hat{u}|$ measures how strongly the target overlaps with the k-th eigen-mode. Training, however, advances in discrete steps: by the construction of the gradient flow (2.28), the continuous time and the iteration count n are related by $\tau = n\mu$, with μ the learning rate. Substituting $\tau = n\mu$ into (2.68) and requiring the amplitude to fall below a target precision $\varepsilon > 0$,

$$|(Fv_k)^* \hat{u}|\, e^{-\lambda_k n \mu} < \varepsilon \implies n > \frac{1}{\mu\, \lambda_k} \ln \frac{|(Fv_k)^* \hat{u}|}{\varepsilon}, \tag{2.69}$$

gives an estimate of the number of gradient-descent iterations needed to resolve the k-th frequency:

$$n_k = \left\lceil \frac{1}{\mu\, \lambda_k} \ln \frac{|(Fv_k)^* \hat{u}|}{\varepsilon} \right\rceil. \tag{2.70}$$

The count follows just as directly from the discrete recursion, without passing through the continuous flow. The discrete counterpart of (2.68) is the modal amplitude read off (2.55), $|\tilde{c}_k^{(n)}| = |\tilde{c}_k(0)|\,|1 - \mu\lambda_k|^n$, and requiring it below ε gives

$$|\tilde{c}_k(0)|\,|1 - \mu\lambda_k|^n < \varepsilon \implies n > \frac{1}{-\ln|1 - \mu\lambda_k|} \ln \frac{|\tilde{c}_k(0)|}{\varepsilon}, \tag{2.71}$$

the inequality flipping because in the stable regime $|1 - \mu\lambda_k| < 1$, so $\ln|1 - \mu\lambda_k| < 0$.

This exact discrete count reduces to the continuous estimate (2.70): by the first-order truncation (2.56), $-\ln|1-\mu\lambda_k| = \mu\lambda_k + O\big((\mu\lambda_k)^2\big)$, so the denominator collapses to $\mu\lambda_k$ and, with $|\tilde{c}_k(0)| \approx |(Fv_k)^*\hat{u}|$, equation (2.71) becomes (2.70). Substituting $\tau = n\mu$ into the continuous decay and counting steps directly on the discrete recursion therefore agree to leading order in the step, exactly as the forward-Euler link between (2.52) and (2.55) requires.

This estimate is tested numerically in this work. The minimal experiment of Chapter 3 records, for each eigendirection, the iteration at which the measured amplitude first falls below ε, and sets it against the count predicted by (2.71). The discrete form is the one put to the test, since it rests on the same geometric law $|1-\mu\lambda_k|^n$ as the measured decay; the continuous estimate (2.70) would add the truncation of the exponential to whatever discrepancy the measurement shows. Because (2.71) carries no reference to the width of the network, it predicts that networks of different widths fall on a single curve, which Chapter 4 checks at three widths.

2.3.5 Physics-Informed Neural Networks

Introduced by Raissi et al. (2019), Physics-Informed Neural Networks (PINNs) insert the PDE directly into the optimization process, forcing the neural network to satisfy the model equations and approximating the PDE solution.

Consider the general PDE Initial Value Problem:

$$\begin{cases} \mathcal{D}[u](x,t) = 0, \quad \forall (x,t) \in \Omega \times (0,T], \\ \quad u(x,0) = u_0(x), \quad x \in \Omega, \end{cases} \tag{2.72}$$

where $\mathcal{D}$ is a differential operator on u, $\Omega \times (0,T]$ is the spatio-temporal domain, and u_0 is the initial condition.

The PDE residual loss penalizes violation of the governing equation at collocation points $\{(x_i,t_i)\}_{i=1}^{N_\mathcal{D}} \subset \Omega \times (0,T]$:

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{N_\mathcal{D}} \sum_{i=1}^{N_\mathcal{D}} \left\| \mathcal{D}[u_\theta](x_i,t_i) \right\|^2 . \tag{2.73}$$

The initial condition is enforced at collocation points $\{x_i\}_{i=1}^{N_0} \subset \Omega$:

$$\mathcal{L}_{\text{ic}}(\theta) = \frac{1}{N_0} \sum_{i=1}^{N_0} \left\| u_\theta(x_i,0) - u_0(x_i) \right\|^2 . \tag{2.74}$$

The combined PINN loss is:

$$\mathcal{L}_{\text{PINN}}(\theta) = \mathcal{L}_{\text{pde}}(\theta) + \mathcal{L}_{\text{ic}}(\theta). \tag{2.75}$$

Due to the compositional structure of MLPs, PINN implementations evaluate PDE residuals via Automatic Differentiation (AD) (LINNAINMAA, 1970; PASZKE et al., 2019), using ADAM (KINGMA et al., 2014) or L-BFGS (NOCEDAL, 1980) as the optimizer.

Application to the Boussinesq Dataset

For the Boussinesq system, a separate PINN is trained for each sample $(\alpha_i, \beta_i) \in \mathcal{S}$ (2.20). The differential operator $\mathcal{D}$ in the generic problem (2.72) is instantiated as the pair of Boussinesq residuals. Each acts on both predicted fields and carries the coefficients of the sample, so we write them with all four arguments displayed:

$$\mathcal{D}_1[\eta_\theta, u_\theta, \alpha_i, \beta_i] = (\eta_\theta)_t + (u_\theta)_x + \alpha_i\,(\eta_\theta\, u_\theta)_x, \tag{2.76}$$

$$\mathcal{D}_2[\eta_\theta, u_\theta, \alpha_i, \beta_i] = (u_\theta)_t - \tfrac{\beta_i}{3}(u_\theta)_{xxt} + (\eta_\theta)_x + \alpha_i\, u_\theta (u_\theta)_x. \tag{2.77}$$

The combined loss (2.75) for sample i then reads:

$$\begin{aligned}\mathcal{L}_{\text{PINN}}(\theta) = \frac{1}{N_{\mathcal{D}}}\sum_{l=1}^{N_{\mathcal{D}}}\Big[\mathcal{D}_1[\eta_\theta, u_\theta, \alpha_i, \beta_i](x_l,t_l)^2 + \mathcal{D}_2[\eta_\theta, u_\theta, \alpha_i, \beta_i](x_l,t_l)^2\Big] \\ + \frac{1}{N_0}\sum_{l=1}^{N_0}\Big[(\eta_\theta(x_l,0) - \eta_0(x_l))^2 + (u_\theta(x_l,0) - u_0(x_l))^2\Big],\end{aligned} \tag{2.78}$$

where η_0 and u_0 are given by the shared initial condition (2.2). Because the parameters (α_i, β_i) enter the residuals explicitly, each PINN encodes knowledge of a single sample and must be retrained for every new parameter pair.

2.3.6 Spectral Bias in Physics-Informed Neural Networks

The Neural Tangent Kernel analysis of Sections 2.3.3–2.3.4 was carried out for an MLP fitting data on a uniform grid, the setting in which the kernel is approximately circulant and its eigenvectors are genuine Fourier modes. A PINN does not fit data directly: its loss (2.75) combines an initial-condition term with a PDE-residual term. The same machinery nevertheless applies once both constraints are stacked into a single error vector, now indexed over the N_r residual points in place of the N data points of the

MLP,

$$e(\theta) = \begin{bmatrix} u_\theta(x_1, 0) - u_0(x_1) \\ \vdots \\ u_\theta(x_{N_0}, 0) - u_0(x_{N_0}) \\ \mathcal{D}[u_\theta](x_1, t_1) \\ \vdots \\ \mathcal{D}[u_\theta](x_{N_\mathcal{D}}, t_{N_\mathcal{D}}) \end{bmatrix} \in \mathbb{R}^{N_r}, \qquad N_r = N_0 + N_\mathcal{D}, \tag{2.79}$$

so that, up to normalization, $\mathcal{L}_{\text{PINN}}$ reduces to the same quadratic form $\frac{1}{2}\|e\|^2$ analyzed for the MLP. With this definition the gradient-flow derivation leading to (2.39) carries over verbatim, with the Jacobian now also containing the parameter gradients of the PDE residuals, and the error still obeys the linear dynamics $\frac{de}{d\tau} = -K_0\, e$, with convergence of each eigenvector governed by its eigenvalue exactly as in (2.60) (WANG; YU, et al., 2022).

This prediction is borne out empirically. Spectral bias is a frequently reported obstacle in PINN training: networks fit smooth, low-frequency solutions readily but stall on sharp gradients and high-frequency content (RAHAMAN et al., 2019; XU et al., 2020; WANG; TENG, et al., 2021), and the same pathology underlies several documented PINN failure modes (KRISHNAPRIYAN et al., 2021). Wang, Yu, et al. (2022) traced the effect directly to the NTK, showing that its spectrum is concentrated at low eigenvalues, so that high-frequency PDE residuals are the last to be minimized. For nonlinear wave equations of Boussinesq type, Wang et al. (2024) report the same difficulty in resolving the finer wave structure; the two-field Boussinesq system studied here has not been examined in this respect, which is part of what Chapter 4 sets out to do. A range of remedies target this imbalance directly, among them Fourier feature mappings that recondition the kernel spectrum (TANCIK et al., 2020), adaptive loss weighting (WANG; TENG, et al., 2021), and second-order or spectrum-aware optimizers (KHODAKARAMI et al., 2026). Pursuing these mitigations is beyond the scope of this work: our aim is to apply the SciML methods to the Boussinesq system and to expose and understand the spectral bias that emerges there, not to remove it. Understanding the effect nonetheless remains essential background for the neural-operator methods studied in the remainder of this work, whose ability to capture the high-frequency content of the Boussinesq solutions is examined directly in Chapter 4.

2.4 Neural Operators

Neural operators generalize neural networks from learning finite-dimensional mappings to learning operators between infinite-dimensional function spaces. This section introduces the general framework, then the Fourier Neural Operator (FNO) as the principal

architecture, and finally the Physics-Informed Neural Operator (PINO) as the physics-constrained extension.

2.4.1 General Definition

Let $D \subset \mathbb{R}^d$ be a bounded open domain and let $\mathcal{A}(D;\mathbb{R}^{d_a})$ and $\mathcal{U}(D;\mathbb{R}^{d_u})$ be Banach spaces of functions (KREYSZIG, 1978). The target operator is a (potentially nonlinear) map

$$G^\dagger : \mathcal{A} \to \mathcal{U}, \qquad a \mapsto u = G^\dagger(a), \tag{2.80}$$

where both the input a and the output u are functions defined on D.

In this work, $G^\dagger$ is the solution operator of the Boussinesq system: given an input a encoding the PDE coefficients (α, β) together with the initial condition (2.2), the operator returns the full space-time solution (η, u).

A neural operator G_θ approximates $G^\dagger$ by alternating linear layers with pointwise nonlinearities, in the composition

$$G_\theta = Q \circ \sigma\Big(W_L + \mathcal{K}_L\Big) \circ \cdots \circ \sigma\Big(W_1 + \mathcal{K}_1\Big) \circ P, \tag{2.81}$$

where:

- $P : \mathbb{R}^{d_a} \to \mathbb{R}^{d_c}$ is the lifting operator, mapping the input $a(x)$ pointwise to a higher-dimensional channel representation with $d_c \gg d_a$;
- $\sigma\Big(W_\ell + \mathcal{K}_\ell\Big)$, for $\ell = 1, \ldots, L$, are the operator layers, each combining a local linear map W_ℓ with a nonlocal integral operator $\mathcal{K}_\ell$, followed by a pointwise activation σ;
- $Q : \mathbb{R}^{d_c} \to \mathbb{R}^{d_u}$ is the projection operator mapping the final channel representation back to the output dimension.

What makes (2.81) an operator, rather than a map between finite-dimensional vectors, is the nonlocal term $\mathcal{K}_\ell$ in each layer. It is an integral operator,

$$(\mathcal{K}_\ell v)(x) = \int_D \kappa_\ell(x, y)\, v(y)\, dy, \tag{2.82}$$

where $\kappa_\ell : D \times D \to \mathbb{R}^{d_c \times d_c}$ is a learned kernel. Each output point therefore draws on the whole input function, and not only on its value at x, which is what the local map W_ℓ alone would give. Different choices of kernel parametrization for each $\mathcal{K}_\ell$ give rise to different neural operator architectures.

2.4.2 Fourier Neural Operators

Presented by Li et al. (2021), the Fourier Neural Operator (FNO) is the neural operator (2.81) in which each $\mathcal{K}_\ell$ uses a shift-invariant kernel: $\kappa_\ell(x, y) = \kappa_\ell(x - y)$. This assumption turns each integral operator into a convolution, which by the convolution theorem (DEBNATH, 2012) can be evaluated efficiently in the frequency domain.

Each spectral layer updates the channel representation via:

$$v_{\ell+1}(x) = \sigma\Big((W_\ell\, v_\ell)(x) + (\mathcal{K}_\ell v_\ell)(x)\Big). \tag{2.83}$$

The operator $\mathcal{K}_\ell$ acts via convolution with the shift-invariant kernel κ_ℓ:

$$(\mathcal{K}_\ell v_\ell)(x) = \int_D \kappa_\ell(x - y)\, v_\ell(y)\, dy. \tag{2.84}$$

By the convolution theorem (DEBNATH, 2012), this equals:

$$(\mathcal{K}_\ell v_\ell)(x) = \mathcal{F}^{-1}\Big(\mathcal{F}(\kappa_\ell) \cdot \mathcal{F}(v_\ell)\Big)(x). \tag{2.85}$$

Property (2.85) is what makes the layer (2.83) affordable: the nonlocal integral never has to be formed in physical space, since a forward transform, a pointwise multiplication and an inverse transform deliver it. To see how this is parametrized, recall that for a periodic function f on $[0, 2\pi]$:

$$f(x) = \sum_{k\in\mathbb{Z}} \hat{f}(k)\, e^{ikx}, \qquad \hat{f}(k) = \frac{1}{2\pi}\int_0^{2\pi} f(x) e^{-ikx}\, dx. \tag{2.86}$$

The convolution $(\kappa * v)$ simplifies to:

$$(\kappa * v)(x) = \int_0^{2\pi} \kappa(x - y)\, v(y)\, dy = \sum_{k\in\mathbb{Z}} \big(2\pi\, \hat{\kappa}(k)\, \hat{v}(k)\big)\, e^{ikx}. \tag{2.87}$$

Instead of learning κ_ℓ as a function in physical space, the FNO directly parametrizes its Fourier coefficients as learnable complex-valued matrices $R_{\ell,k} \approx 2\pi\, \hat{\kappa}_\ell(k)$. Only the $|k| \le k_{\max}$ low-frequency modes are retained, with all others set to zero; $k_{\max}$ is a hyperparameter.

Training Objective

The FNO is trained as a pure supervised learning problem. Given the dataset $\mathcal{S}$ (2.20), whose i-th element pairs the coefficients (α_i, β_i) with the reference solution (η_i, u_i) computed for them, the training objective minimizes the mean squared error be-

tween the predicted and reference space-time fields:

$$\mathcal{L}_{\mathrm{FNO}}(\theta) = \frac{1}{M}\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[(\eta_\theta^i(x_j,t_n)-\eta_i(x_j,t_n))^2+(u_\theta^i(x_j,t_n)-u_i(x_j,t_n))^2\Big], \tag{2.88}$$

where G_θ is the neural operator (2.81) and $(\eta_\theta^i, u_\theta^i) \coloneqq G_\theta(\alpha_i, \beta_i)$ is its prediction for sample i. Minimizing (2.88) therefore drives $G_\theta(\alpha_i, \beta_i)$ towards (η_i, u_i) for all M samples of $\mathcal{S}$ at once, which is what lets a single trained operator answer the whole parameter family.

Application to Non-Time-Periodic Problems

When the solution is also assumed to be periodic in time with period T (so that $D = [0, 2\pi] \times [0, T]$), shift-invariance extends to the time direction as well: $\kappa_\ell(x, y, t, \tau) = \kappa_\ell(x - y,\, t - \tau)$. The convolution then generalizes to:

$$(\mathcal{K}_\ell v_\ell)(x,t) = \int_D \kappa_\ell(x-y,\, t-\tau)\, v_\ell(y,\tau)\, dy\, d\tau = \mathcal{F}^{-1}\Big(\mathcal{F}(\kappa_\ell)\cdot\mathcal{F}(v_\ell)\Big)(x,t), \tag{2.89}$$

and the FNO parametrizes the two-dimensional Fourier modes (k_x, k_t) with $|k_x| \le k_{x,\max}$ and $|k_t| \le k_{t,\max}$, both hyperparameters.

For problems that are not time-periodic, however, applying the Fourier transform along the temporal axis implicitly extends the signal periodically, potentially introducing a jump discontinuity at the temporal boundary. This can trigger the Gibbs phenomenon (GIBBS, 1899), producing oscillatory artifacts near the boundary that persist regardless of how many temporal modes are retained. Standard signal-processing techniques such as zero-padding and window functions (e.g. Hann, Hamming windows) can partially mitigate these effects, at the cost of altering the temporal structure of the signal. Li et al. (2021) acknowledge this limitation and provide an alternative formulation in which the Fourier layers operate only along the spatial axis while the model advances forward in time, avoiding the periodicity assumption in the temporal direction entirely. In this work we deliberately retain the two-dimensional space-time formulation, in which the Fourier layers act on both the spatial and the temporal axes, so that a single operator emits the entire trajectory in one forward pass. This keeps the architecture simple and uniform across space and time, but it reintroduces the temporal-periodicity assumption for a Boussinesq solution that is not time-periodic, and therefore admits a Gibbs artifact at the temporal boundary $t = T$. Rather than suppress this artifact with windowing, we retain the plain formulation and examine the boundary behaviour directly in Chapter 4, where it turns out to distinguish the purely data-driven operator from its physics-informed variant.

2.4.3 Physics-Informed Neural Operators

Analogously to how PINNs add a physics-informed residual loss to the MLP training objective, the Physics-Informed Neural Operator (PINO) (LI et al., 2023a) incorporates PDE constraints into the FNO training. The FNO backbone remains unchanged; what differs is the loss function, which now combines a data term with residuals evaluated on the predicted space-time fields.

Unlike a PINN, which represents the solution as a single continuous function and differentiates it via automatic differentiation, the FNO outputs (η_θ, u_θ) as a discrete $N_x \times N_t$ array in a single forward pass over the full space-time domain. Evaluating the PDE residuals therefore reduces to numerically differentiating this output array.

Spatial derivatives are computed pseudospectrally using ξ_k (2.14): the DFT of η_θ along the spatial axis at each time t_n yields $\hat{\eta}_\theta(k, t_n)$, from which

$$(\eta_x)_j = \mathcal{F}^{-1}(i\,\xi_k\,\hat{\eta}_\theta(k,t_n))_j\,, \qquad (\eta_{xx})_j = \mathcal{F}^{-1}\big(-\xi_k^2\,\hat{\eta}_\theta(k,t_n)\big)_j\,, \tag{2.90}$$

and identically for u_θ. Time derivatives are computed by finite differences: second-order central in the interior and first-order one-sided at the boundaries,

$$(\eta_\theta)_t^n = \begin{cases} \dfrac{\eta_\theta^{n+1} - \eta_\theta^n}{\Delta t}, & n = 0, \\ \dfrac{\eta_\theta^{n+1} - \eta_\theta^{n-1}}{2\,\Delta t}, & 1 \le n \le N_t - 1, \\ \dfrac{\eta_\theta^n - \eta_\theta^{n-1}}{\Delta t}, & n = N_t, \end{cases} \tag{2.91}$$

and identically for $(u_\theta)_t^n$. The mixed term $(u_\theta)_{xxt}$ is obtained by first computing $(u_\theta)_{xx}$ spectrally and then applying the time-difference stencil to the result. This combination of spectral spatial derivatives and finite-difference temporal derivatives follows the reference implementation of Li et al. (2023a) (LI et al., 2023b).

With all derivatives available, the Boussinesq equations (2.1) define two differential operators acting on the predicted fields:

$$\mathcal{D}_1(\eta_\theta, u_\theta) \coloneqq (\eta_\theta)_t + (u_\theta)_x + \alpha\,(\eta_\theta\,u_\theta)_x, \tag{2.92}$$

$$\mathcal{D}_2(\eta_\theta, u_\theta) \coloneqq (u_\theta)_t - \tfrac{\beta}{3}(u_\theta)_{xxt} + (\eta_\theta)_x + \alpha\,(u_\theta(u_\theta)_x). \tag{2.93}$$

Here $(\eta_\theta^i, u_\theta^i) \coloneqq G_\theta(\alpha_i, \beta_i)$ denotes the network output for sample i, and the subscript $_{j,n}$ indicates evaluation at the grid point (x_j, t_n). The PINO loss is a weighted sum of three terms:

$$\mathcal{L}_{\text{PINO}}(\theta) = \lambda_{\text{pde}}\,\mathcal{L}_{\text{pde}}(\theta) + \lambda_{\text{ic}}\,\mathcal{L}_{\text{ic}}(\theta) + \lambda_{\text{data}}\,\mathcal{L}_{\text{data}}(\theta), \tag{2.94}$$

where:

- $\mathcal{L}_{\text{pde}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[\mathcal{D}_1(\eta_\theta^i, u_\theta^i)_{j,n}^2 + \mathcal{D}_2(\eta_\theta^i, u_\theta^i)_{j,n}^2\Big]$ penalizes violation of the Boussinesq equations at each grid point, with residuals evaluated using the parameters (α_i, β_i) of each sample;
- $\mathcal{L}_{\text{ic}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x}\sum_{j=0}^{N_x-1}\Big[(\eta_\theta(x_j,0) - \eta_0(x_j))^2 + (u_\theta(x_j,0) - u_0(x_j))^2\Big]$ enforces the shared initial condition (2.2);
- $\mathcal{L}_{\text{data}}(\theta) = \dfrac{1}{M}\displaystyle\sum_{i=1}^{M}\frac{1}{N_x N_t}\sum_{j=0}^{N_x-1}\sum_{n=0}^{N_t}\Big[(\eta_\theta^i(x_j,t_n) - \eta_i(x_j,t_n))^2 + (u_\theta^i(x_j,t_n) - u_i(x_j,t_n))^2\Big]$ measures deviation from the reference solutions $(\eta_i, u_i) \in \mathcal{D}$ (2.20);
- λ_{pde}, λ_{ic}, $\lambda_{\text{data}} > 0$ are weighting hyperparameters.

The PINO therefore generalizes the FNO by adding physics supervision: it can be trained with fewer labeled samples by relying on the PDE residual for additional guidance.

3 METHODOLOGY

This chapter shows how the experiments were built, on what data, and under which rules, in enough detail to reproduce them. Three things are common to every experiment, and we settle them first: the computational setup and the reproducibility rules (Section 3.1), the pseudospectral dataset that serves both as training target and as ground truth (Section 3.2), and the error metrics that score every model on one scale (Section 3.3). Section 3.4 then defines the six trained models. Section 3.5 sets up the minimal experiment on spectral bias, the behaviour they all share, run on the same Boussinesq solution and the same kind of network as the rest of the work, and Section 3.6 collects the hyperparameters in one table.

3.1 Computational Setup and Reproducibility

The software behind these experiments was all written for this work. It runs on Python 3.11 with PyTorch 2.12.0 (PASZKE et al., 2019) built against CUDA 13.0, on a single NVIDIA GeForce GTX 1660 SUPER GPU. The static figures were drawn with Plotly and the animations with Matplotlib. We used no pre-trained weights, no third-party operator-learning library, and no external dataset. The pseudospectral solver, the four architectures, and each optimization loop are our own code, which let us instrument them for the spectral analysis and the custom visualization used in upcoming chapters. The full implementation is available at `https://github.com/samuelkutz/TCC`.

Each learned architecture stays close to its source. The Fourier Neural Operator follows the original implementation of Li et al. (2021); the Physics-Informed Neural Operator follows Li et al. (2023a), together with its released code (LI et al., 2023b); and the Physics-Informed Neural Network follows Raissi et al. (2019). All of them are written from scratch in PyTorch, and where an implementation departs from its reference we point that out.

Reproducibility rests on two conventions. A single global seed, 37, initializes Python, NumPy and PyTorch at the start of the pipeline and puts the cuDNN backend in deterministic mode. The stages then run in a fixed order, each inheriting the random state its predecessors left instead of drawing a fresh one, which is how the weights reported

here were produced; a configuration flag reseeds every stage from that same global seed, which lets a single stage be rerun on its own and still reproduce.

We now fix the experimental parameters shared by every model and method. Starting with the physical model, the spatial and temporal domains of the Boussinesq system (2.1) are

$$x \in [-L, L], \quad L = 60, \qquad t \in [0, T], \quad T = 30. \tag{3.1}$$

Every experiment uses the initial condition (2.2) at unit amplitude $A = 1$, that is, $\eta(x, 0) = \operatorname{sech}^2(x)$ and $u(x, 0) = 0$. A unit crest sits at the centre of a domain 120 units wide. That width is deliberate: the two counter-propagating pulses of the wave-equation limit (2.9)–(2.10) travel at unit speed, so over the horizon $T = 30$ they cover half the distance to the imposed periodic boundary and never reach it. Table 3.1 lists the constants that hold everywhere. Each of them is restated below at the point where it first matters.

Table 3.1 – Fixed simulation, dataset, and evaluation constants shared by every experiment. Unless a section says otherwise, these values hold throughout.

Parameter	**Value**
Domain and initial condition	
Spatial half-length L $(\Omega = [-L, L])$	60
Final time T $(t \in [0, T])$	30
Initial amplitude A	1
Initial condition	$\eta(x, 0) = \operatorname{sech}^2(x),\ u(x, 0) = 0$
Dataset solve	
Spatial grid points N_x	128
Time levels (127 RK4 steps)	128
Spatial spacing $\Delta x = 2L/N_x$	0.9375
Temporal spacing $\Delta t = T/127$	≈ 0.2362
Parameter sampling	
Coupled coefficients	$\alpha = \beta$
Training samples M	20
Sampling grid	linspace(0.1, 4.0, 20)
Evaluation and reproducibility	
Evaluation values $\alpha = \beta$	{0.1, 3.21, 4.2}
Evaluation resolutions N_x	{128, 256, 512}
Spectral snapshot resolution	256
Global seed	37

Source: The author (2026).

3.2 The Parametric Dataset

The reference solutions do double duty. They are the supervised targets that the data-driven models learn from, and they are the ground truth against which every model is scored. Both roles ask for a high-fidelity baseline, and the pseudospectral Runge–Kutta solver of Section 2.2 supplies one: the domain is wide enough that the pulses never reach its ends, so the periodicity the solver assumes costs nothing over the simulated horizon.

We keep the parameter family one-dimensional on purpose. Following Section 2.2.4, the initial conditions are fixed and only the PDE coefficients vary, tied together as $\alpha = \beta$ so that the two-parameter Boussinesq family collapses onto a single dimension of difficulty. For small values of the parameters the dynamics stay close to linear; for larger values, nonlinearity and dispersion bring richer behaviour. We take $M = 20$ equally spaced values,

$$\alpha_i = \beta_i \in \text{linspace}(0.1,\, 4.0,\, 20), \qquad i = 1, \ldots, 20, \tag{3.2}$$

running from the near-linear value 0.1 to a more nonlinear regime at 4.0, and pair each with the solution the solver returns for it.

Each value in (3.2) is handed to the solver on $N_x = 128$ spatial points and marched through 127 Runge–Kutta steps over $[0, T]$, giving $N_t = 128$ time levels. The grid spacings are

$$\Delta x = \frac{2L}{N_x} = \frac{120}{128} = 0.9375, \qquad \Delta t = \frac{T}{127} = \frac{30}{127} \approx 0.2362. \tag{3.3}$$

A solve returns the two space-time fields $\eta_i(x_j, t_n)$ and $u_i(x_j, t_n)$ on the $N_x \times N_t$ grid, and we collect them into the solution dataset $\mathcal{S}$ of Equation (2.20). Each sample is stored with four input channels and two output channels. The inputs are the initial conditions $\eta_i(x_j, 0)$ and $u_i(x_j, 0)$ at every spatial point x_j, together with the two coefficients α_i and β_i, each of the four held constant along time so that all of them fill the space-time grid:

$$X \in \mathbb{R}^{M \times 4 \times N_x \times N_t}, \qquad Y \in \mathbb{R}^{M \times 2 \times N_x \times N_t}. \tag{3.4}$$

The two output channels carry the fields η_i and u_i themselves.

From now on the panels report only the surface elevation η, since it carries the wave structure under study and the velocity field behaves the same way.

The quantities a network reads here do not share a scale. The fields η and u are of order one, the coefficients α and β run over more than a decade, and the coordinates span $[-L, L] \times [0, T]$. Practice in the two literatures this work draws on says not to leave it that way. For pointwise networks the recommendation is to non-dimensionalize, scaling the variables so that they become dimensionless and of order one, since disparate scales let one variable dominate another in the gradient and force the optimizer into small steps

for one and large steps for the next (WANG; SANKARAN, et al., 2023); for operators, the reference implementations normalize each channel by an affine map as a matter of course (KOSSAIFI et al., 2024; LI et al., 2021).

We use a single rule for all of it. Writing $[f_{\min}, f_{\max}]$ for the envelope of a quantity f, every network input in this work is carried onto $[-1, 1]$ by

$$\tilde{f} = \frac{2\,(f - f_{\min})}{f_{\max} - f_{\min}} - 1, \qquad f = f_{\min} + \tfrac{1}{2}\,(\tilde{f} + 1)\,(f_{\max} - f_{\min}), \tag{3.5}$$

the second expression being the exact inverse of the first. What changes from one quantity to the next is only which envelope is handed to (3.5): $[-A, A]$ for the fields, set by the known initial amplitude; $[\alpha_{\min}, \alpha_{\max}]$ for the coefficients, the endpoints of the sampling range (3.2); $[-L, L]$ and $[0, T]$ for the coordinates. The target $[-1, 1]$, not $[0, 1]$, suits the hyperbolic tangent of the pointwise backbones, which is odd and centred at the origin, and it is the interval the minimal experiment of Section 3.5 already works on.

The choice of envelope matters more than the algebra. Each is fixed in advance from quantities the problem itself provides, never estimated from the reference solutions, so the physics-only regimes of Section 3.4 borrow nothing from the dataset and the no-data models stay data-free in the output scale as well. And because the envelopes are constants, not sample statistics, the map does not move when the grid is refined. This is deliberate: normalization that averages over a discrete grid is a known route by which a neural operator loses the resolution invariance its spectral layers are built to preserve, since the same continuous field resampled at another resolution yields different statistics (KIM et al., 2026). Fixed envelopes are immune to that, so a trained operator can be queried on the refined grids of (3.7) with the scale it was trained under. Nothing forces a normalized value into $[-1, 1]$: these are reference scales, not bounds, and one of the evaluation coefficients of Section 3.3 sits past $\alpha_{\max}$ on purpose, which makes it a test of extrapolation.

Where each model uses it

The MLP applies (3.5) to its two coordinates and leaves its output in physical units. The PINN does the same, and applies it inside the forward pass instead of at the call site: the residual is then differentiated with respect to the physical x and t, automatic differentiation carries the chain rule through the rescaling on its own, and what is enforced stays the residual of (2.1) and not of a rescaled equation. Applying the same map to both also leaves the two models sharing an input scale as well as a backbone, so the comparison between them isolates the training signal. This matters here in particular, because the scale on which a network reads its coordinates sets the frequencies its tangent kernel is

built from (WANG; SANKARAN, et al., 2023), and that kernel is the object Section 3.5 measures.

The FNO and the PINO apply (3.5) to the four input channels and the two output channels of (3.4), with the envelopes stored one pair per channel and broadcast along space and time. To these four inputs both append two coordinate channels, x and t on the same $[-1, 1]$, so the lifting layer reads six channels instead of four; the spatial one follows the solver's periodic grid, whose right endpoint is dropped, so that the channel keeps a fixed physical meaning as the resolution changes. The two operators differ in the units each loss term is formed in. Both compare prediction against target in the normalized variables, which puts the pure-data and data-and-physics regimes on one objective scale. The PINO then returns its fields to physical units for the residual and initial-condition terms, so that the constraint it enforces is the balance of (2.1) at its true magnitudes. Every prediction is returned to physical units by the inverse in (3.5) before any reported metric is formed.

3.3 Error Metrics

A comparison is only fair if every model is measured against the same reference and on the same scales. Each method is scored against a freshly recomputed pseudospectral reference at three values of the parameter of (2.1),

$$\alpha = \beta \in \{0.1,\ 3.21,\ 4.2\}, \tag{3.6}$$

the first near-linear and itself a sample of the training grid (3.2), the second more nonlinear and falling between two training samples, the third more nonlinear still and past the upper end of the sampling range. The middle value 3.21 is the default wherever a single representative parameter is needed, as it is for the MLP and the two PINNs, which are trained at one parameter.

To test the discretization invariance that operators are supposed to enjoy, each trained operator is queried without retraining on grids of resolution

$$N_x \in \{128,\ 256,\ 512\}, \tag{3.7}$$

the training resolution and its two- and fourfold refinements, all powers of two, the sizes on which the fast Fourier transform of Section 2.2.1 runs most efficiently (TREFETHEN, 2000). For each query the input channels of Section 3.2 are rebuilt on the finer grid from the same initial condition, and the reference is recomputed at the matching resolution, so prediction and reference are always compared at the same points. The pointwise models are queried on the same grids, since a continuous map can be sampled anywhere.

Let η_{true} and η_{pred} be the reference and predicted elevation on a shared space-time

grid. We report three complementary numbers.

Time-resolved relative error

At each time level t_n we take the spatial L^2 error and divide it by the norm of the reference at that instant,

$$\mathbb{E}(t_n) = \frac{\left\|\eta_{\text{pred}}(\cdot, t_n) - \eta_{\text{true}}(\cdot, t_n)\right\|_2}{\left\|\eta_{\text{true}}(\cdot, t_n)\right\|_2 + \varepsilon}, \qquad \varepsilon = 10^{-12}. \tag{3.8}$$

The constant ε keeps the quotient defined at an instant where the reference vanishes. It does not bias the reported values: the reference is a travelling wave of fixed amplitude, so its spatial norm stays of order one along the whole horizon and the shift is a relative 10^{-12}. Read against time, $\mathbb{E}(t_n)$ shows where along the trajectory the error piles up, and its spread over all time levels is summarized by a box plot. This is the quantity behind the parameter and resolution panels of Chapter 4.

Relative spectral error

To see which spatial frequencies are wrong, we take the spatial real DFT at a fixed time and compare amplitudes mode by mode. Writing $\hat{\eta}(k, t_n)$ for the amplitude at wavenumber index k, the per-mode error is

$$\mathbb{E}_{\text{spec}}(k, t_n) = \frac{\left|\, |\hat{\eta}_{\text{pred}}(k, t_n)| - |\hat{\eta}_{\text{true}}(k, t_n)| \,\right|}{|\hat{\eta}_{\text{true}}(k, t_n)| + \delta \max\limits_{k'} |\hat{\eta}_{\text{true}}(k', t_n)|}, \qquad \delta = 10^{-3}. \tag{3.9}$$

The floor here is a fraction of the largest reference amplitude at the same instant, not a fixed constant. A reference spectrum does not decay smoothly: it passes through near-nulls, single modes whose amplitude falls orders of magnitude below their neighbours. Such a mode still sits far above any absolute constant one would write down, so a fixed floor never engages, and an unremarkable absolute error divided by a near-null amplitude reports an enormous relative error that says nothing about the model. Scaling the floor to the spectrum removes that: a mode carrying real amplitude dominates the floor and is unaffected, while a mode below δ of the peak is measured against the peak instead of against itself. Taking the maximum at each time level rather than once for the whole trajectory keeps the floor tied to the amplitude the solution actually carries at that instant, so late times, where the whole spectrum has decayed, are not measured against an early-time peak.

Frequency-band error during training

To watch spectral bias form as the optimizer runs, we split the spatial spectrum into three bands and save each band error every 500 optimizer iterations. The per-mode quantity is exactly the relative spectral error (3.9), and it is reduced to one number per band in two averages. First over the N_t time levels, giving a value for each mode, then over the modes of the band,

$$\overline{\mathbb{E}}_{\mathrm{spec}}(k) = \frac{1}{N_t}\sum_{n=1}^{N_t}\mathbb{E}_{\mathrm{spec}}(k,t_n), \qquad \mathbb{E}_{\mathrm{band}}(B) = \frac{1}{|B|}\sum_{k\in B}\overline{\mathbb{E}}_{\mathrm{spec}}(k), \tag{3.10}$$

so the same metric drives the per-frequency spectra and the band-tracking curves. With $N_k = \lfloor N_x/2 \rfloor + 1$ retained real-DFT modes, and writing $q_1 = \lfloor N_k/4 \rfloor$ and $q_2 = \lfloor N_k/2 \rfloor$ for the two cut points, the three index sets B are

$$B_{\mathrm{low}} = \big\{\, k : 0 \le k < q_1 \,\big\}, \quad B_{\mathrm{mid}} = \big\{\, k : q_1 \le k < q_2 \,\big\}, \quad B_{\mathrm{high}} = \big\{\, k : q_2 \le k < N_k \,\big\}, \tag{3.11}$$

a quarter of the range each for the low and mid bands and the upper half for the high band. These curves feed the band-tracking comparison of Chapter 4. Every model is tracked at one common parameter, $\alpha = \beta = 3.21$: the pointwise MLP and PINN against their own training solution at that value, each operator against the dataset sample nearest it, so the bands are read at a single parameter across all six.

3.4 The Six Models

Four architectures give rise to six trained models, arranged along two axes. The first is the architecture family. The pointwise networks, the multilayer perceptron of Section 2.3.1 and the physics-informed network of Section 2.3.5, represent one solution as a continuous map of the coordinates, $(x,t) \mapsto \eta$ for the MLP and $(x,t) \mapsto (\eta, u)$ for the PINN, and are retrained for each parameter. The operator networks, the Fourier neural operator of Section 2.4.2 and its physics-informed variant of Section 2.4.3, learn the whole parameter family (3.2) at once and return a new parameter's solution in a single forward pass.

The second axis is the training regime, that is, what the loss is allowed to see:

- Pure data fits the reference solutions with a supervised term alone and no equation. The data-only MLP and the FNO sit here.
- Data and physics keeps that supervised term and adds the Boussinesq residual beside it. The PINN and the PINO with data sit here.

- Pure physics switches the supervised term off, leaving the equation residual and the initial condition to drive the network. The PINN and the PINO without data sit here.

The MLP and the FNO carry no residual, so each family contributes a single pure-data model; the PINN and the PINO carry a residual whose data weight can be set on or off, so each contributes one data-and-physics and one pure-physics model. Crossing the two families with the three regimes therefore fills a 2×3 grid, the six models collected in Table 3.2.

Reading across the grid isolates what each ingredient contributes. Within a family, pure data against data and physics measures what the residual adds when the data are present, and data and physics against pure physics measures how far the equation alone carries the network once the data are removed. Across families at a fixed regime, the operator against the pointwise model measures what operator learning contributes. Every model is trained against the pseudospectral references of Section 3.2, on the grid of Table 3.1, and is scored by the same metrics of Section 3.3, so a difference in behaviour traces to the family and the regime rather than to the setup. The operators read all $M = 20$ samples of (3.4) at once, while each pointwise model is tied to a single parameter and fits the one solve at $\alpha = \beta = 3.21$.

3.5 Isolating the Spectral-Bias Mechanism

To some degree every model in this work carries the spectral bias derived in Section 2.3.4. We examine it on the simplest object this work can offer, and we keep it on the Boussinesq system: a multilayer perceptron from $\mathbb{R}$ to $\mathbb{R}$ that learns a single elevation profile at the fixed time $t = 15$, taken from the same pseudospectral solver at the intermediate parameter $\alpha = \beta = 3.21$,

$$x \longmapsto \eta_\theta(x) \approx \eta(x,\, 15), \qquad x \in [-L, L]. \tag{3.12}$$

Nothing is introduced that the rest of the work does not already use: no auxiliary equation, no synthetic target, no second architecture. This leaves the setting the theory of Section 2.3.4 was derived for, a target sampled on a regular periodic grid and fitted by gradient descent.

Fixing the time removes the temporal axis from the problem, so the Fourier analysis of the error is a plain spatial transform on the $N_x = 128$ grid and the wavenumbers of the theory are the wavenumbers of the figure, with no averaging in between. The instant $t = 15$ sits at the middle of the horizon, by which point the initial crest has split into two counter-propagating pulses with dispersive oscillations between them, so the target

carries amplitude across the spectrum rather than concentrating it in the single smooth bump the initial condition would offer.

3.5.1 Setting Up the Experiment

We train three networks, at widths $N_{\mathrm{MLP}} \in \{8, 64, 512\}$, each with four hidden layers and hyperbolic-tangent activations, one input and one output. The coordinate reaches the network on $[-1, 1]$ and the target on the amplitude envelope $[-A, A]$, both carried there by the map (3.5), so these networks read the same scales as the rest of the work; the figures undo the target scaling to display η in physical units. Every figure in this section comes from those same three runs.

Because the target lives on a single grid of $N_x = 128$ points, the Jacobian $J \in \mathbb{R}^{N_x \times N_\theta}$ is cheap enough to assemble explicitly, one backward pass per grid point, at every width including the widest. That matters, because the empirical kernel

$$K_0 = J_0 J_0^{\mathsf{T}} = V \Lambda V^{\mathsf{T}} \tag{3.13}$$

has to be eigendecomposed, not merely applied, and it is only 128×128. It is also cheap enough to rebuild the kernel during training, which lets us see how far it drifts from its initial state. Every run is carried out in double precision so that the small eigenvalues survive the decomposition, and the three widths are trained for the same number of iterations.

Training is full-batch gradient descent on the sum-of-squares loss $\frac{1}{2}\|e\|^2$, so that a single step is exactly

$$\theta \leftarrow \theta - \mu\, J^{\mathsf{T}} e, \tag{3.14}$$

the update behind (2.54), with $e = \eta_\theta - \eta_{\mathrm{true}}$ measured on the grid. Each width takes the same fixed fraction of its own stability bound, $\mu = 0.2\,(2/\lambda_{\max}) = 0.4/\lambda_{\max}$, with $\lambda_{\max}$ the largest eigenvalue of K_0 at that width. Since $\lambda_{\max}$ grows with width, the three step sizes differ; what they share is the product $\mu\lambda_{\max} = 0.4$, which places all three networks at the same point inside the monotone regime $\mu\lambda_k < 1$ of Section 2.3.4. Anchoring μ to the bound $\mu < 2/\lambda_{\max}$ instead of to a nominal value keeps the three widths comparable. We iterate 500,000 times.

The MLP is nonlinear in its parameters, so its Jacobian is not constant: J drifts away from J_0 as training proceeds, and the initial-kernel prediction $e^{(n)} = (I - \mu K_0)^n e_0$ of (2.55), which comes from the forward-Euler discretization of the gradient flow (2.42), is an approximation here, not an identity. Whether it survives contact with a real network is part of what the experiment measures.

We therefore record four measurements along the runs and set each against the

initial-kernel prediction of Section 2.3.4.

- Per-eigendirection error decay. We project the measured error onto each eigenvector v_k of K_0, giving $c_k^{(n)} = v_k^{\mathsf{T}} e^{(n)}$, and compare its amplitude against the predicted $|c_k(0)|\,|1-\mu\lambda_k|^n$ of (2.55). Because the decay times $1/(\mu\lambda_k)$ span several decades, the four eigendirections drawn are chosen so that their decay times spread over the timescales the run can resolve, and the iteration axis is logarithmic.

- Per-mode spectral decay. We take the real DFT of the error profile at each iteration, giving one amplitude per wavenumber, and compare the measured decay against the prediction of (2.66). Four fixed wavenumbers are followed across the whole run rather than four eigendirections, so the panel reads the frequency principle literally.

- Band-averaged relative spectral error. The same comparison over the whole spectrum instead of four sampled wavenumbers: the per-mode error (3.9) is averaged within each of the three bands of (3.11). There is a single time level here, so the time average of (3.10) does not apply. Measured and predicted fields are both scored against the reference profile, which puts this panel on the metric the six trained models are tracked with in Chapter 4.

- Per-mode iteration count n_k. For each eigendirection we record the iteration at which $|c_k^{(n)}|$ first drops below a precision ε and compare it against the discrete crossing time $n_k = \ln(|c_k(0)|/\varepsilon)/(-\ln|1-\mu\lambda_k|)$ of (2.71), the step at which the geometric law $|1-\mu\lambda_k|^n$ of (2.55) falls below ε. Using this discrete count instead of its continuous small-step limit (2.70) keeps the panel on the same $(1-\mu\lambda_k)^n$ law as the decay curves, so a departure from the diagonal reflects kernel drift rather than the truncation of the exponential. We set $\varepsilon = 0.05\,\max_k |c_k(0)|$, a fraction of the largest initial coefficient rather than an absolute number, so the threshold scales with the amplitude of η instead of with the units it is measured in. The closed form carries no width, so the three networks should fall on one diagonal; both axes are logarithmic, since the counts span decades. Only the modes whose predicted n_k falls inside the run are drawn, since the rest are never reached and the measurement cannot test them.

The first three panels are drawn at the widest network, since lazy training is an assumption about wide networks; the fourth collects all three widths. Alongside these we keep what the networks actually produced: the final output at each width, and the widest network's output at logarithmically spaced checkpoints, both set against the reference profile.

3.5.2 Kernel Drift and the Eigenvalue Spectrum

The predictions above assume the kernel stays put, and we measure whether it does along the same three runs. We log the relative parameter drift $\|\theta-\theta_0\|/\|\theta_0\|$ and the relative kernel drift $\|K - K_0\|/\|K_0\|$ at logarithmically spaced iterations, and we record the eigenvalue spectrum of K at initialization and again at the end of training.

The two drift curves ask whether the kernel stays put, as lazy training assumes, and whether it does so more convincingly as the network widens, as the initial-kernel approximation (2.42) predicts. The spectrum exposes the eigenvalue disparity that (2.60) names as the cause of spectral bias. Together with the four checks of Section 3.5.1, these curves open the results of Chapter 4, where they anchor the reading of the spectral-bias curves measured on the full models.

3.6 Configuration Summary

Table 3.2 gathers the configuration of every experiment. The four learned architectures share a common budget, 15,000 iterations of Adam at learning rate 10^{-3}, so the comparison of Chapter 4 reflects the architecture and the training regime, not the effort spent on tuning. The batch size of the operators, 256, exceeds the $M = 20$ samples of the dataset, so each of their steps is in practice a full-batch step. The last row reports the measured training wall-clock on the hardware of Section 3.1, which is indicative and depends on the GPU used.

Table 3.2 – Configuration of the four training architectures and of the minimal spectral-bias network. The FNO and PINO share the operator backbone and its parameter count; the MLP and PINN share the feed-forward backbone and are single-parameter models. All Adam-trained models use 15,000 iterations. The PINN draws a fresh set of 5,000 interior collocation points at every iteration and evaluates its initial-condition term on a further 500 points. The last column is a separate network, deliberately minimal, $\mathbb{R} \to \mathbb{R}$ on the single profile $\eta(x, 15)$, trained once per width.

Setting	**MLP**	**PINN**	**FNO**	**PINO**	**Minimal MLP**
Input $\to$ output	$\mathbb{R}^2 \to \mathbb{R}$	$\mathbb{R}^2 \to \mathbb{R}^2$	operator	operator	$\mathbb{R} \to \mathbb{R}$
Trainable parameters	198,401	198,658	2,106,146	2,106,146	241, 12,673, 789,505
Hidden layers	4	4	4 Fourier	4 Fourier	4
Width / channels	256	256	32	32	$8, 64, 512$
Fourier modes (per axis)	—	—	16	16	—
Optimizer	Adam	Adam	Adam	Adam	GD
Learning rate	10^{-3}	10^{-3}	10^{-3}	10^{-3}	$0.4/\lambda_{\max}$
Iterations	15,000	15,000	15,000	15,000	500,000
Parameters trained	single (3.21)	single (3.21)	all 20	all 20	single (3.21)
Collocation / batch	full batch	5,000 pts	batch 256	batch 256	128 pts (full)
Loss weights pde/ic/data	$-/-/1$	$1/1/\{1, 0\}$	$-/-/1$	$100/1/\{1, 0\}$	$-/-/1$
Physics-residual grid	—	5,000 pts	—	256×256	—
Target	$\eta(x, t)$	η, u	family	family	$\eta(x, 15)$
Data regimes	data only	with / without	data only	with / without	data only
Training wall-clock	≈ 152 s	≈ 880 s	≈ 1178 s	≈ 4440–5460 s	—

Source: The author (2026).

4 NUMERICAL RESULTS

This chapter reports what the experiments of Chapter 3 produced. The experiments fall into two groups: a minimal spectral-bias experiment that isolates the learning mechanism on a single profile, and the six trained models, three pointwise and three operators, evaluated across the three training regimes against the pseudospectral reference. Section 4.1 accounts for what each model costs to train. The next three sections then take the models along one axis at a time: Section 4.2 sweeps the nonlinearity at a fixed resolution, Section 4.3 holds the parameter fixed and queries the operators off their training grid, and Section 4.4 fixes both and compares the spectra mode by mode at three instants. The temporal artifact that the physics term corrects is isolated in Section 4.5. With the behaviour on the table, Section 4.6 turns to the mechanism behind it, establishing spectral bias on the kernel of a minimal MLP fitting the Boussinesq elevation, and Section 4.7 follows the error through the frequency bands as training proceeds. Section 4.8 gathers the findings. The pseudospectral solver is the ground truth throughout.

4.1 Training Cost

Before asking how well the models perform, we ask what they cost. Table 4.1 lists the size, wall-clock training time, and final training loss of the six trained models. Two quantities set the models apart. In capacity, the operators carry on the order of a million parameters, about ten times the pointwise MLP and PINN. In training time, the gap is wide: the MLP finishes in a few minutes, the two PINNs and the plain FNO in under half an hour, and the PINO in well over an hour. The PINO is slow because it assembles its physics residual on the finer 256×256 grid (Table 3.2) at every iteration. The final-loss column ranks nothing, since each objective sums a different subset of data, residual, and initial-condition terms, so every comparison that follows rests instead on the held-out relative-error metrics of Section 3.3.

That extra hour of training is what makes the operators general. A single FNO or PINO covers the entire 20-point family (3.2) from one training, whereas each MLP and PINN is fixed to $\alpha = \beta = 3.21$ and must be retrained from scratch for any other value. Spread over the family, the operators are the cheaper route to a solution at a new

Table 4.1 – Model size, training time on the GTX 1660 SUPER, and final training loss. The final-loss column is not comparable across rows: each objective combines data, residual and initial-condition terms differently, so its magnitude reflects the makeup of the loss rather than solution accuracy. Accuracy is measured instead by the relative-error metrics of the following sections.

Model	Regime	Parameters	Train time (s)	Final loss
MLP	pure data	198,401	151.9	4.3×10^{-5}
FNO	pure data	2,106,146	1177.8	4.8×10^{-7}
PINO	data and physics	2,106,146	5459.4	5.2×10^{-5}
PINO	pure physics	2,106,146	4441.2	4.0×10^{-5}
PINN	data and physics	198,658	898.9	7.1×10^{-5}
PINN	pure physics	198,658	868.4	4.6×10^{-5}

Source: The author (2026).

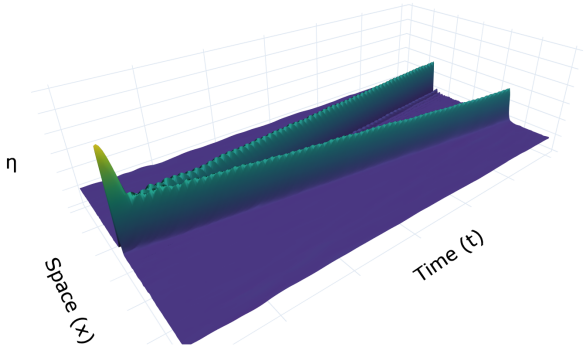
parameter, since each answers a new value in a single forward pass; of the three operator models, the FNO is the cheapest to train, as it carries no physics residual.

4.2 Across the Parameter Range

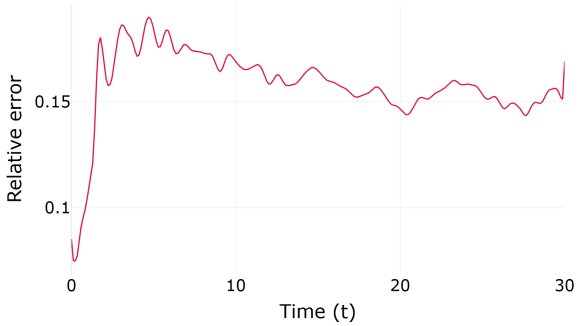
The first comparison holds the resolution fixed and sweeps the nonlinearity. It concerns the operators alone, since each pointwise MLP and PINN is trained at a single parameter and has nothing to say about the sweep.

The box plots at the foot of Figures 4.1 and 4.2 summarize the distribution of the time-resolved error (3.8) over the whole trajectory at each of the three parameters, and the two operators order the parameters differently. For the FNO the near-linear extreme is the hardest, and the error eases as the nonlinearity grows. The PINO with data traces a shallow U across the axis instead: the middle parameter is the easiest and both extremes are harder.

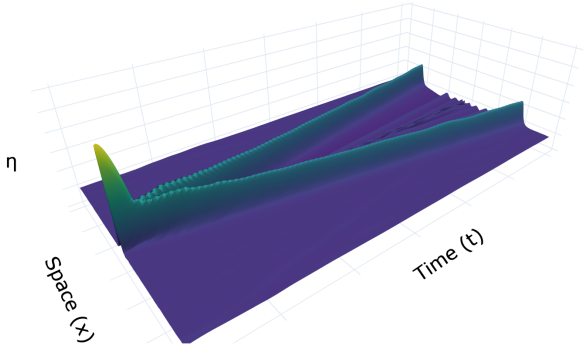
Two effects plausibly combine to make the FNO's near-linear case its worst. First, that parameter sits at the lower edge of the training grid (3.2), where the operator has the least surrounding data to interpolate from, so a boundary-of-range penalty is expected. Second, the relative metric (3.8) divides by the instantaneous norm of the reference. In the near-linear regime the solution is the two clean, slowly deforming pulses of the wave-equation limit, whose norm is modest, so a fixed absolute error reads as a larger relative one. In the PINO with data, the physics residual holds the low and mid modes accurate across the range, and what error remains concentrates where the dynamics are hardest, at the extremes. The two panels agree only that the relative error is non-monotone in the parameter; which end is worst depends on the model. They differ on a second count, one unrelated to the parameter: the FNO error curves end in a sharp terminal spike and the



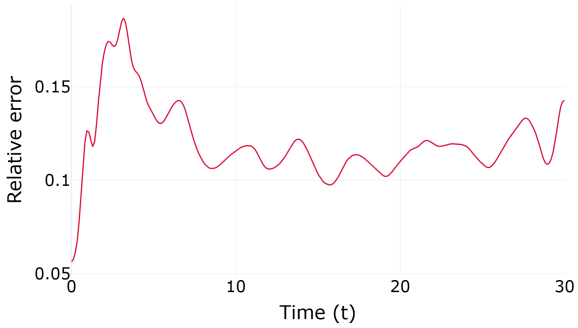
(a) Predicted $\eta(x,t)$, $\alpha = \beta = 0.1$.



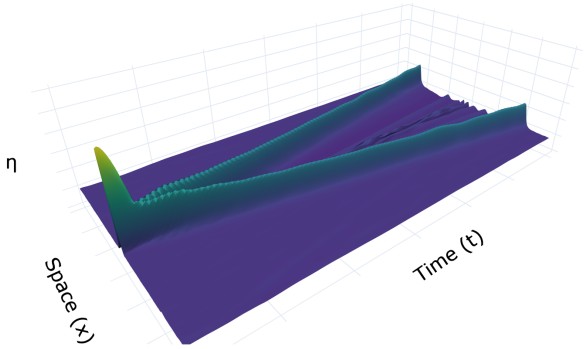
(b) Relative error, $\alpha = \beta = 0.1$.



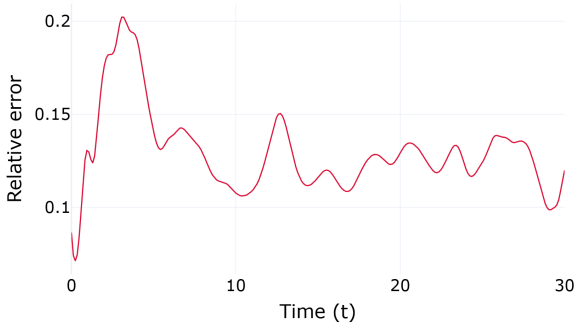
(c) Predicted $\eta(x,t)$, $\alpha = \beta = 3.21$.



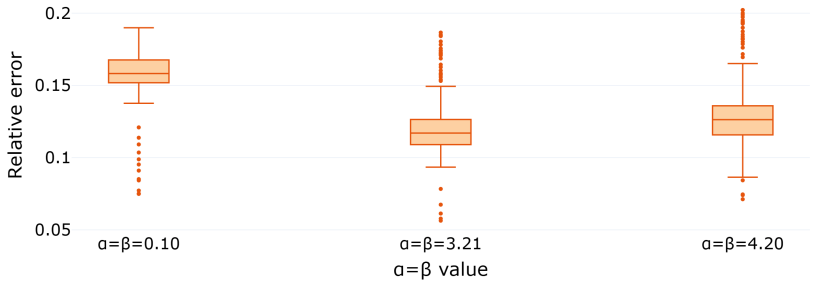
(d) Relative error, $\alpha = \beta = 3.21$.



(e) Predicted $\eta(x,t)$, $\alpha = \beta = 4.2$.



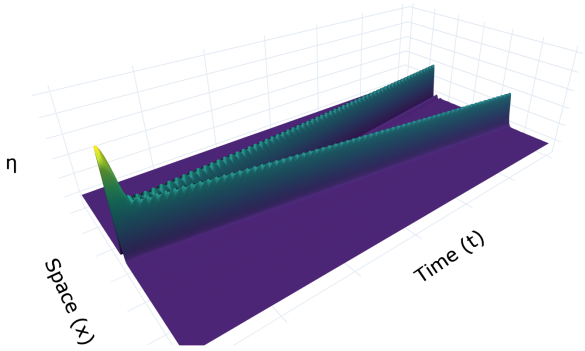
(f) Relative error, $\alpha = \beta = 4.2$.



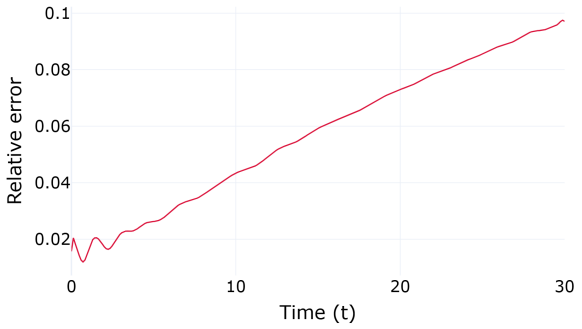
(g) Relative error distribution vs. $\alpha = \beta$.

Figure 4.1 – FNO (pure data) across the nonlinearity axis ($\alpha = \beta \in \{0.1, 3.21, 4.2\}$), evaluated at resolution 256. The predicted surfaces (left) capture the two counter-propagating pulses, but the time-resolved relative error (right) spikes at the final time $t = 30$, the temporal Gibbs artifact of treating time as a periodic axis.

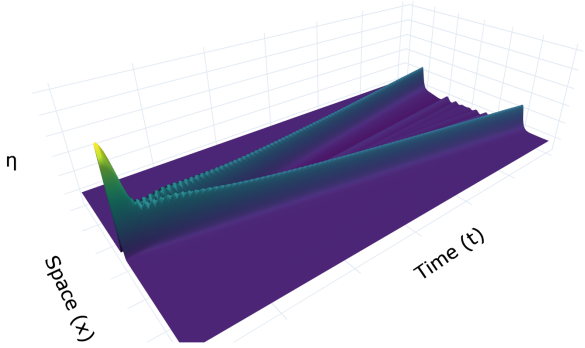
Source: The author (2026).



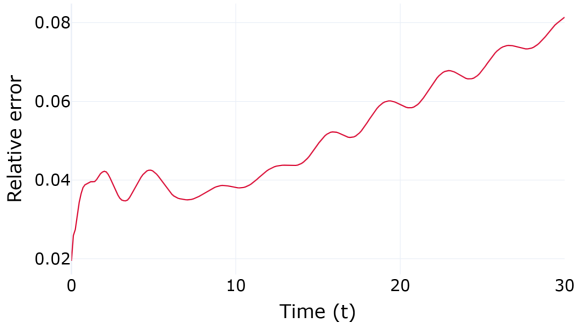
(a) Predicted $\eta(x,t)$, $\alpha = \beta = 0.1$.



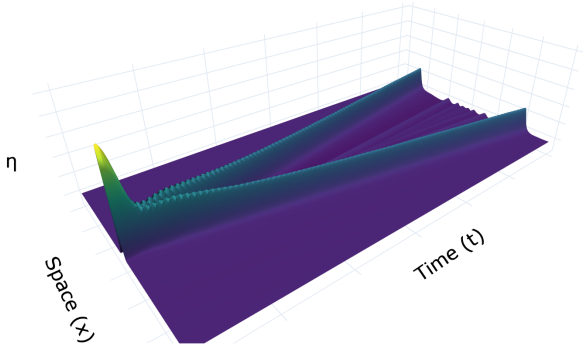
(b) Relative error, $\alpha = \beta = 0.1$.



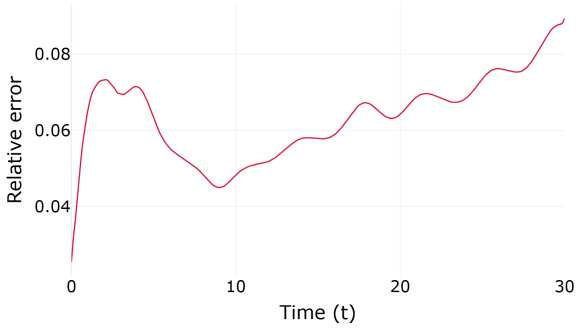
(c) Predicted $\eta(x,t)$, $\alpha = \beta = 3.21$.



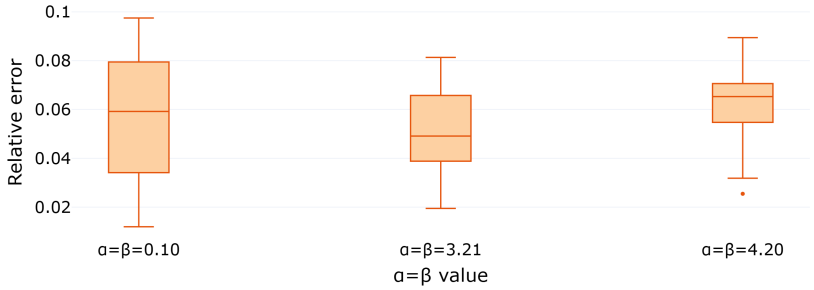
(d) Relative error, $\alpha = \beta = 3.21$.



(e) Predicted $\eta(x,t)$, $\alpha = \beta = 4.2$.



(f) Relative error, $\alpha = \beta = 4.2$.



(g) Relative error distribution vs. $\alpha = \beta$.

Figure 4.2 – PINO (data and physics) across the same nonlinearity axis and resolution. The predicted surfaces are visually indistinguishable from the FNO, but the terminal-time spike of Figure 4.1 is gone: the error grows gently across the trajectory instead of jumping at the boundary. The physics residual and the initial-condition term act as a temporal regularizer.

Source: The author (2026).

PINO curves do not. Section 4.5 takes that up.

4.3 Across Resolutions

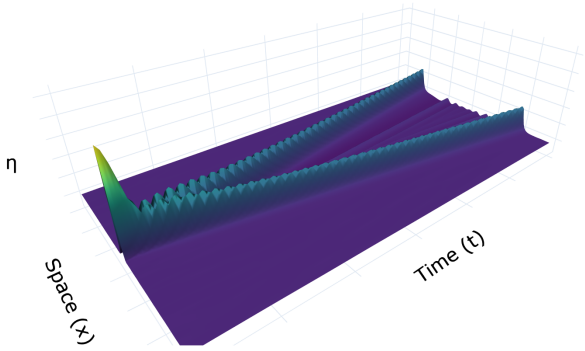
The next comparison queries the operator off its training resolution, probing the discretization invariance operators are meant to enjoy. Figure 4.3 takes the PINO with data, whose supervised term saw the dataset at resolution 128 while its physics residual was formed on a 256×256 grid (Table 3.2), and evaluates it at 128, 256, and 512 at the fixed parameter $\alpha = \beta = 3.21$ with no retraining. Which of those two grids the accuracy follows is the question the panel settles.

Discretization invariance holds in the geometric sense. At every resolution the operator returns a smooth, coherent surface with the correct pulse structure, with no retraining and no architectural change, since the input is simply resampled on the finer grid. The box plot puts the relative error at its lowest on the residual grid and rising away from it in both directions, worst on the coarsest of the three. Queried below that resolution the operator loses the residual-resolved detail; queried above it, it meets high-wavenumber content it never learned to produce, since its spectral layers keep only a fixed number of Fourier modes per axis. The plain FNO, whose only constraint is the data at its training resolution, degrades monotonically over the same sweep, best on the grid it was trained on and worse on every finer one (Figure 4.4). Zero-shot superresolution is thus a geometric property, since the operator accepts and returns fields at any resolution, but not an accuracy property: fidelity stays anchored to the spectral band the training constraints resolved, and it is highest where those constraints were applied.

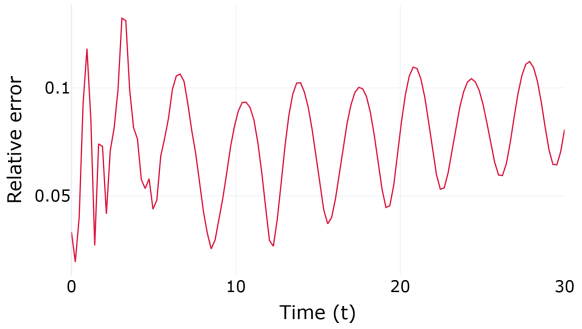
4.4 Reproducing the Spectrum

To compare the families on equal terms we fix $\alpha = \beta = 3.21$ and inspect the full spatial spectrum of the prediction at three times, $t = 0$, $t = T/2 = 15$, and $t = T = 30$. Figures 4.5–4.10 report the six models. In every panel the black curve is the reference spatial spectrum and the dashed orange curve is the prediction, with the per-mode relative error beside each spectrum and the mode-averaged error against time at the foot of each figure. One reading note applies throughout. The per-mode error (3.9) floors its denominator at a fixed fraction of the largest reference amplitude at the same instant, so a mode whose amplitude has fallen to a near-null is measured against the peak instead of against itself, and an unremarkable absolute error there no longer reports an enormous relative one. The mean printed on each panel therefore summarizes the fit across the whole band, and it is against that band, not against any single mode, that the predicted spectrum should be read.

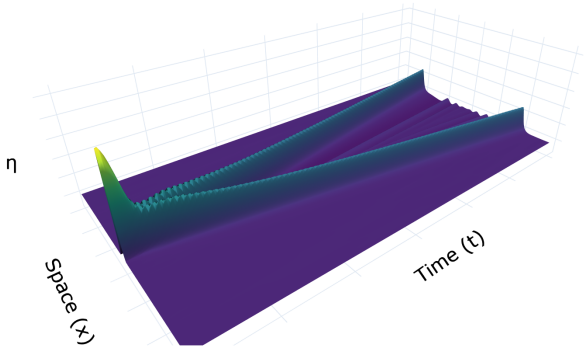
The pointwise panels show one failure, and it is there in the initial state already.



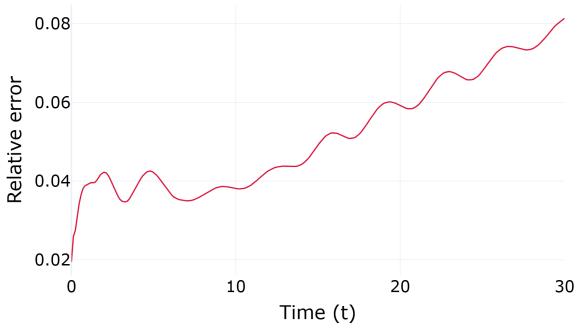
(a) Predicted $\eta(x,t)$, res. 128.



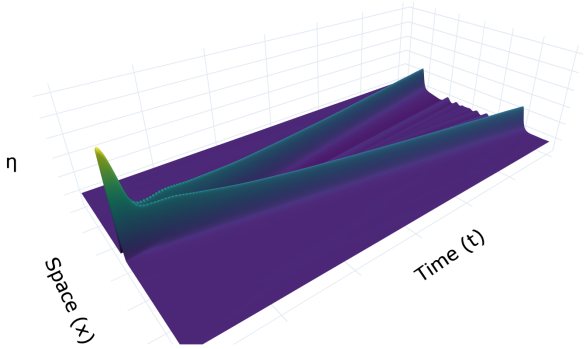
(b) Relative error, res. 128.



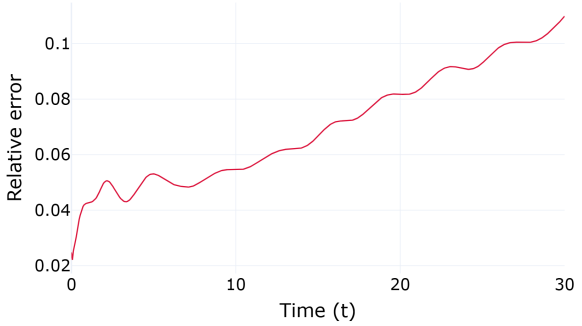
(c) Predicted $\eta(x,t)$, res. 256.



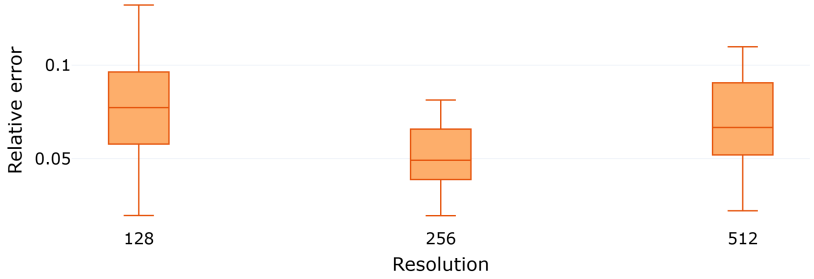
(d) Relative error, res. 256.



(e) Predicted $\eta(x,t)$, res. 512.



(f) Relative error, res. 512.



(g) Relative error distribution vs. resolution.

Figure 4.3 – PINO (data and physics) queried zero-shot at resolutions 128, 256 and 512 at $\alpha = \beta = 3.21$. The operator returns a coherent field at every resolution, and the relative error against the pseudospectral reference is lowest at the resolution on which the physics residual was trained and rises on either side of it, most on the coarsest grid: accuracy is anchored to the residual grid, not the training-data grid.

Source: The author (2026).

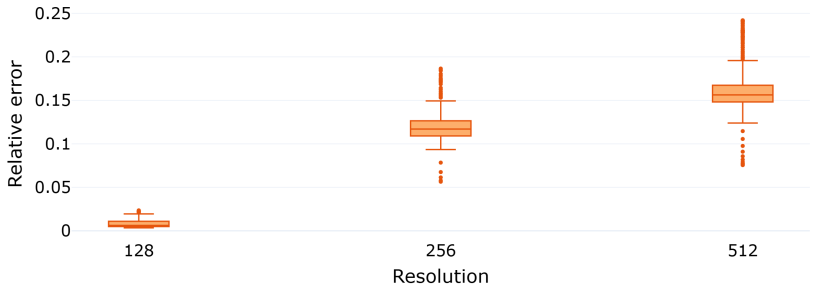


Figure 4.4 – Plain FNO queried zero-shot on the same three grids at $\alpha = \beta = 3.21$. With no physics residual to anchor it, the relative error is lowest on the training grid and grows with every refinement, in contrast to the interior minimum of Figure 4.3.

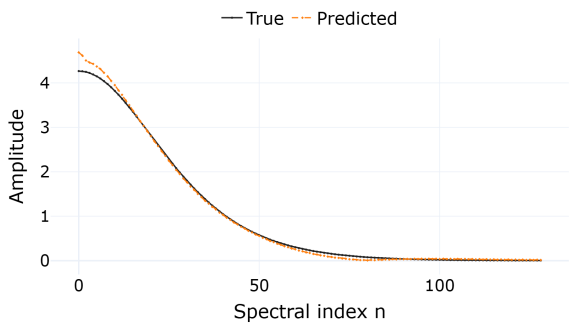
Source: The author (2026).

The MLP (Figure 4.5) fits the reference where the frequencies are low and undershoots where they are high, with no equation and no physics term in the objective to complicate the reading. Both PINN regimes fail to represent the high-wavenumber content of the sech^2 profile at $t = 0$, and their relative error climbs toward and past unity as the mode index grows. Without data (Figure 4.6) the predicted spectrum sheds most of its mid- and high-frequency amplitude by late time; adding data (Figure 4.7) tightens the low-wavenumber crests, in step with the low-band gain of Figure 4.14, yet leaves the mid and high modes largely unrecovered at this parameter. Data helps the pointwise models where the kernel is favourable, and not where it is adverse.

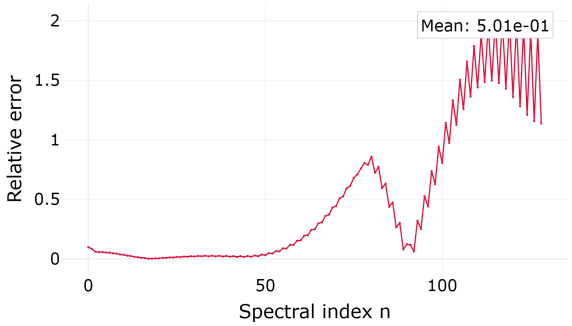
The operators, learning in the frequency domain, invert this picture at low and mid wavenumbers. The FNO (Figure 4.8) tracks the low and mid spectrum closely at early times, at a smaller error than the pointwise models, because those are the modes its spectral layers parametrize. Its weakness is in time: the discrepancy gathers in the high modes and grows as the trajectory advances. The PINO with data (Figure 4.9) inherits the operator's spectral accuracy while holding the late-time behaviour steadier, the benefit of the physics residual that the next section isolates. The PINO without data (Figure 4.10) gives up the advantage, its low- and mid-band undershoot returning to sit between the operators with data and the PINNs.

4.5 Temporal Stability from the Physics Residual

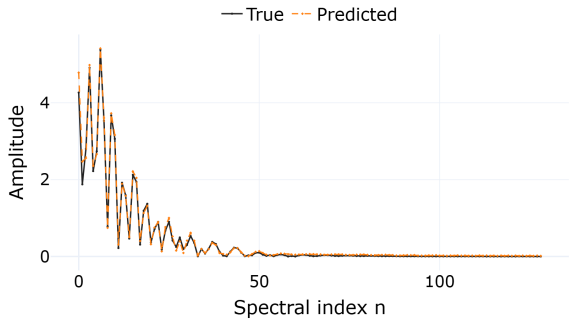
The temporal growth of the FNO error in Figure 4.8 has a specific and instructive cause, and isolating it shows where the physics term makes a visible difference. As Section 2.4.2 explains, the operator applies the Fourier transform along the temporal axis as well as the spatial one, implicitly assuming the trajectory is periodic in time. It is not. The field at $t = T$ does not match the field at $t = 0$, so the assumption manufactures an



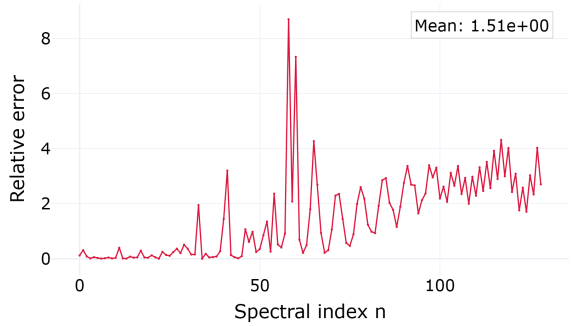
(a) Spectrum, $t = 0$.



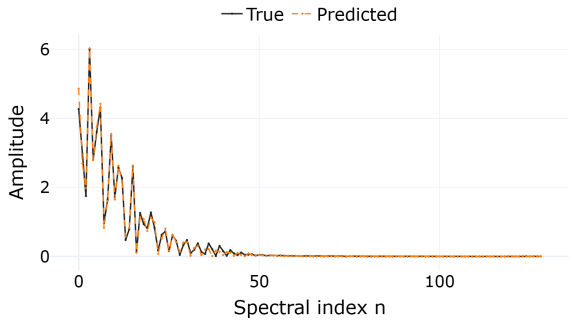
(b) Relative error, $t = 0$.



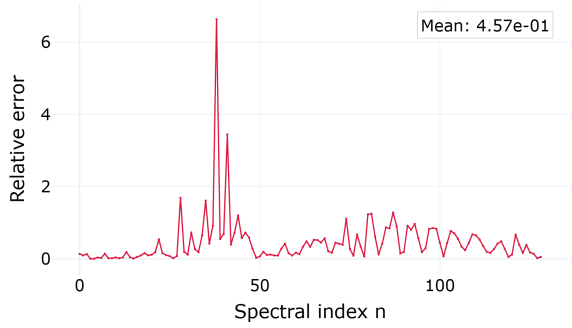
(c) Spectrum, $t = 15$.



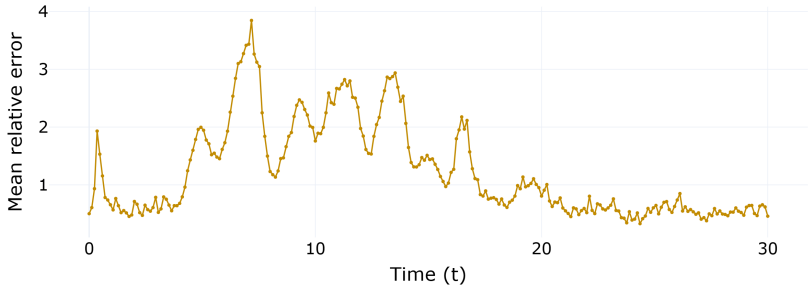
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



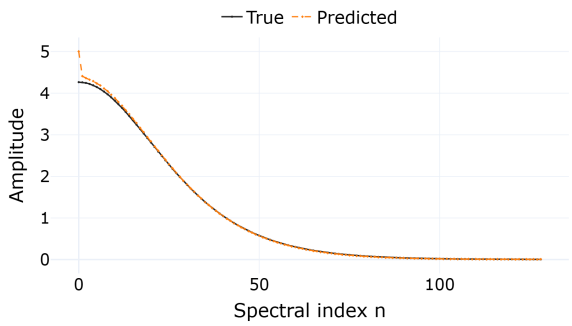
(f) Relative error, $t = 30$.



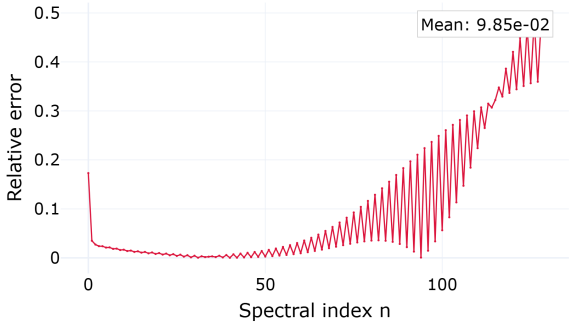
(g) Mean relative spectral error over time.

Figure 4.5 – MLP (pure data) spatial spectrum at $\alpha = \beta = 3.21$. Fitting the reference with no equation and no physics term, the network reproduces the low-wavenumber crests but undershoots the tail. The relative error climbs with wavenumber, a clear case of spectral bias with nothing else in the objective to obscure it.

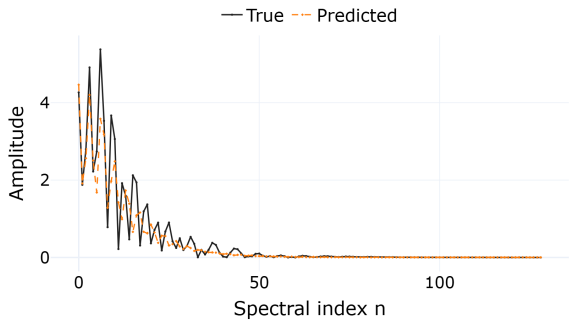
Source: The author (2026).



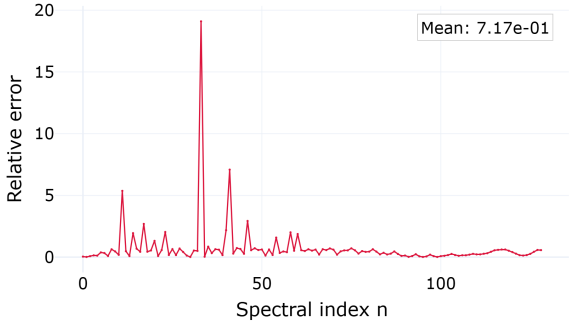
(a) Spectrum, $t = 0$.



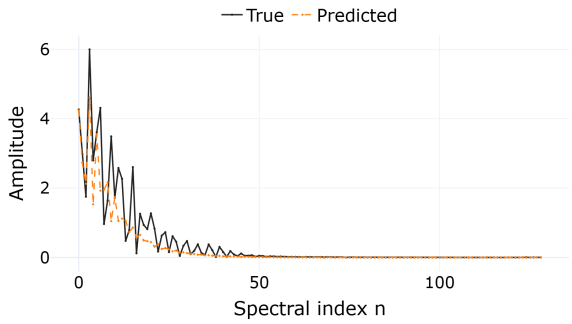
(b) Relative error, $t = 0$.



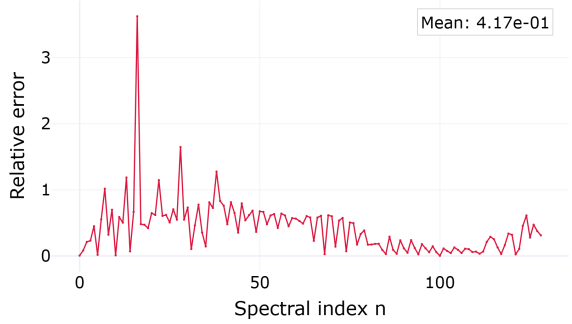
(c) Spectrum, $t = 15$.



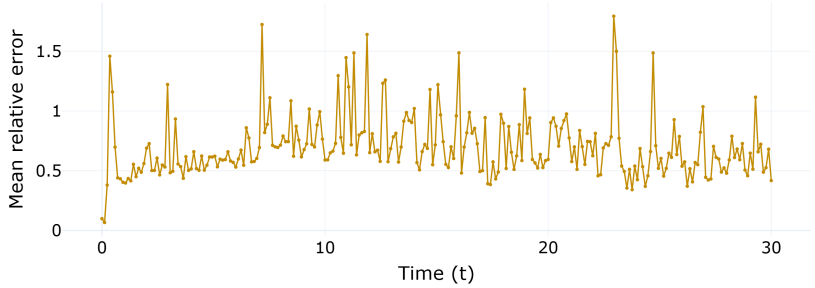
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



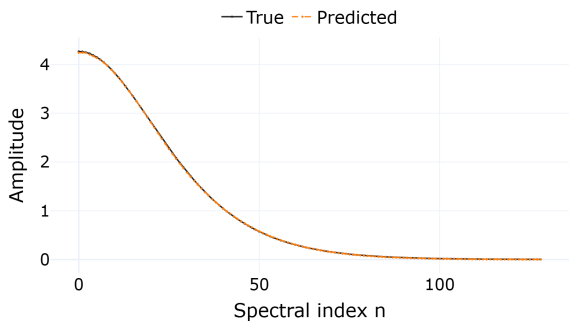
(f) Relative error, $t = 30$.



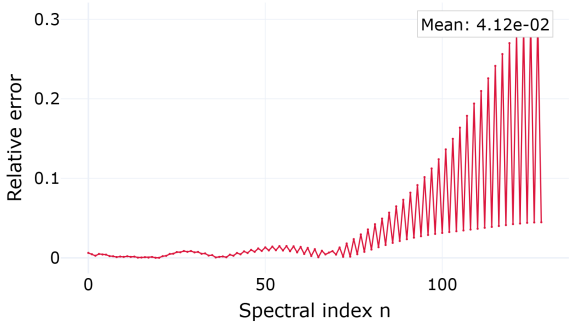
(g) Mean relative spectral error over time.

Figure 4.6 – PINN (pure physics) spatial spectrum at $\alpha = \beta = 3.21$. The network cannot represent the sech^2 tail even at $t = 0$, and the relative error rises with wavenumber, worsening as time advances.

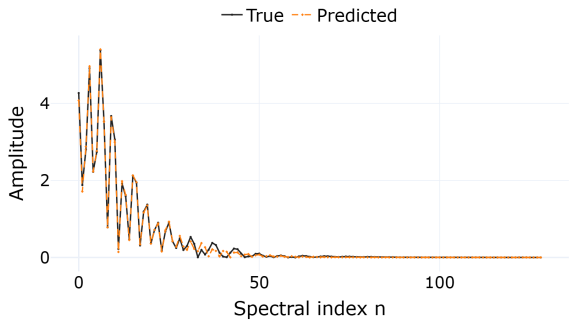
Source: The author (2026).



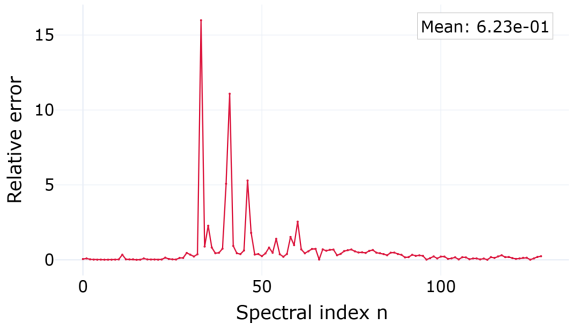
(a) Spectrum, $t = 0$.



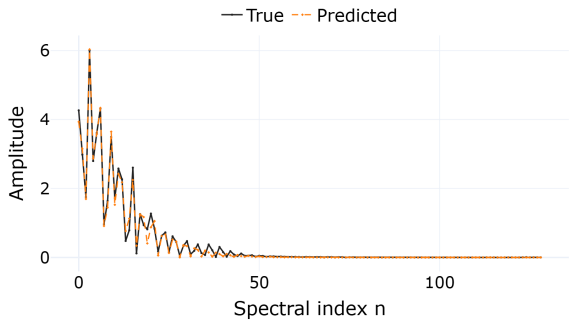
(b) Relative error, $t = 0$.



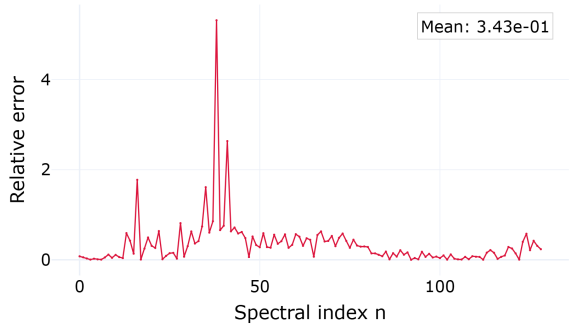
(c) Spectrum, $t = 15$.



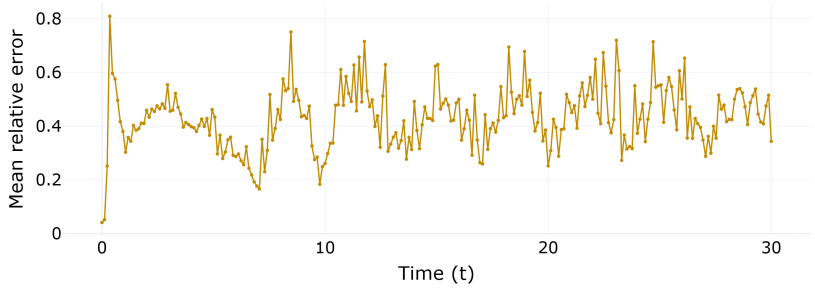
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



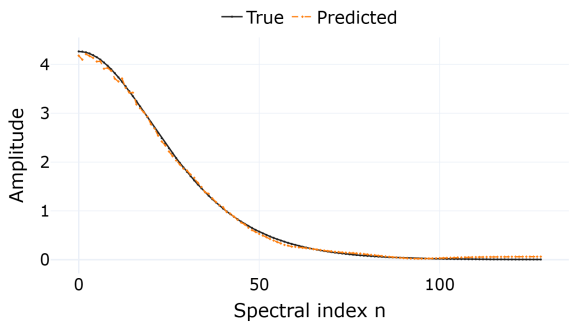
(f) Relative error, $t = 30$.



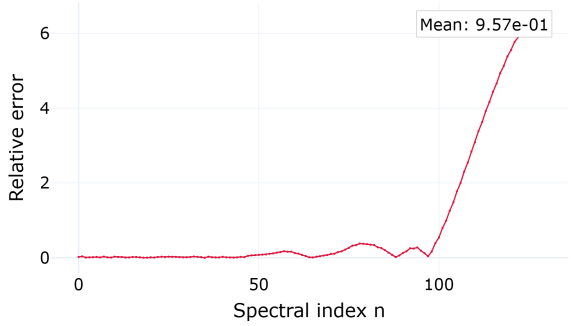
(g) Mean relative spectral error over time.

Figure 4.7 – PINN (data and physics) spatial spectrum at $\alpha = \beta = 3.21$. The low-wavenumber crests are tracked more tightly than in Figure 4.6, but the high-wavenumber undershoot remains and the error still grows in time.

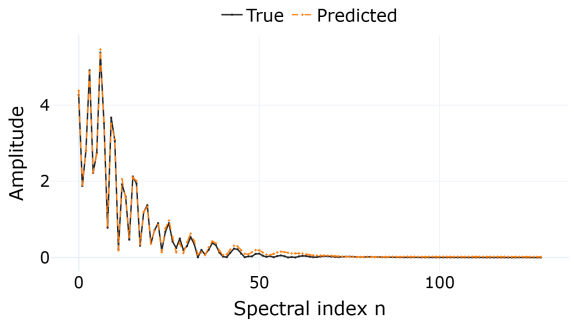
Source: The author (2026).



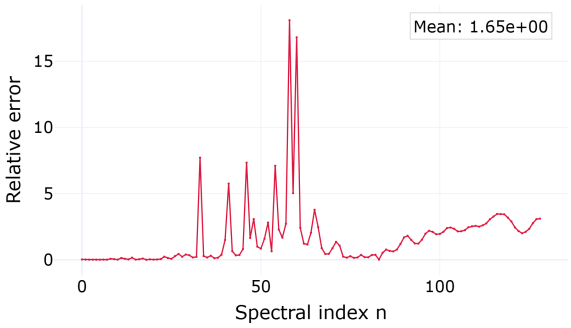
(a) Spectrum, $t = 0$.



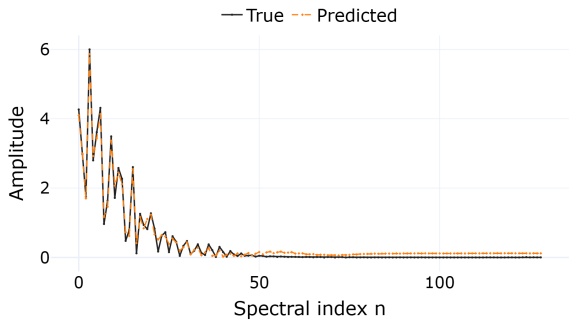
(b) Relative error, $t = 0$.



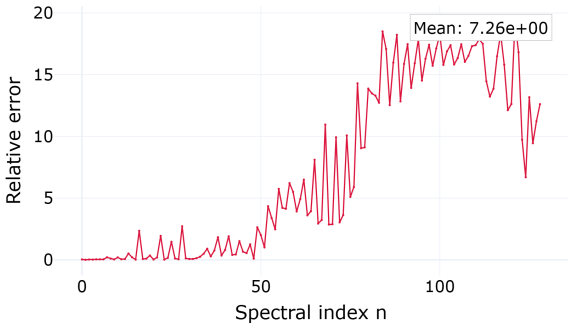
(c) Spectrum, $t = 15$.



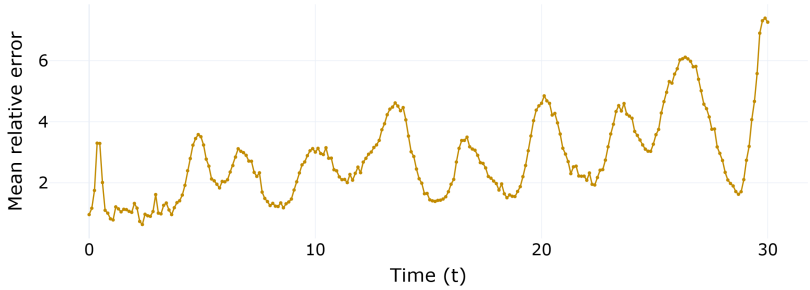
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



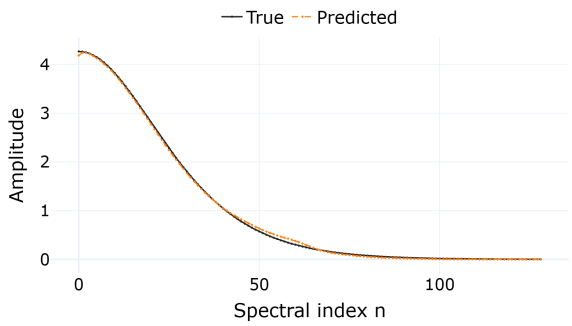
(f) Relative error, $t = 30$.



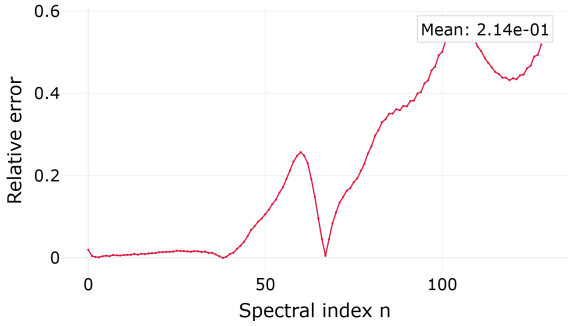
(g) Mean relative spectral error over time.

Figure 4.8 – FNO (pure data) spatial spectrum at $\alpha = \beta = 3.21$. Low and mid wavenumbers are tracked closely, but the error migrates to the high modes and grows in time.

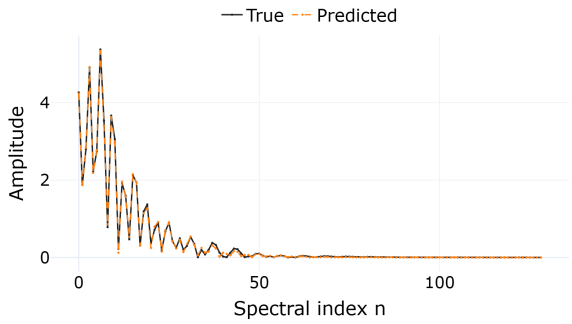
Source: The author (2026).



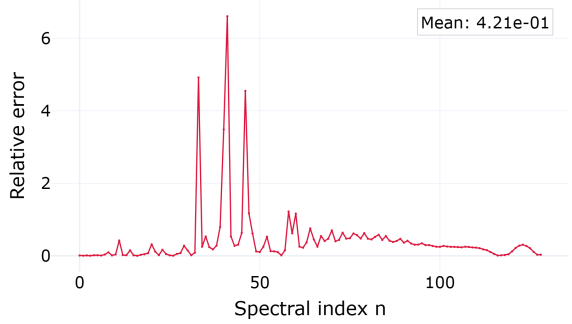
(a) Spectrum, $t = 0$.



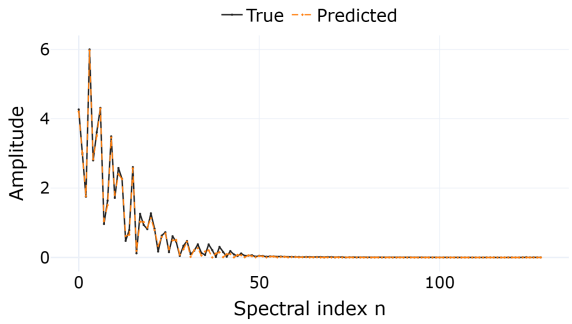
(b) Relative error, $t = 0$.



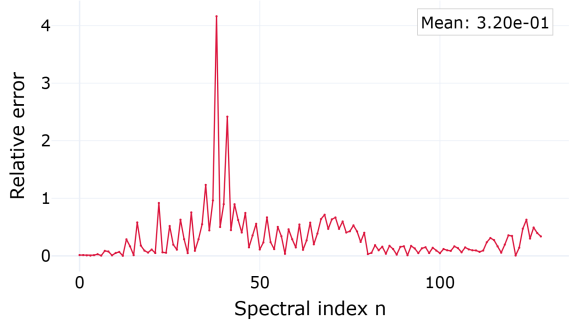
(c) Spectrum, $t = 15$.



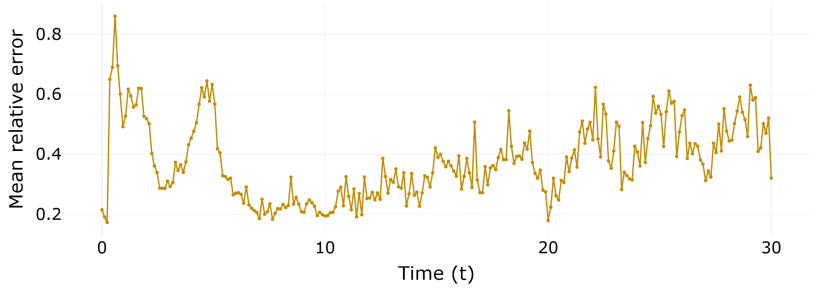
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



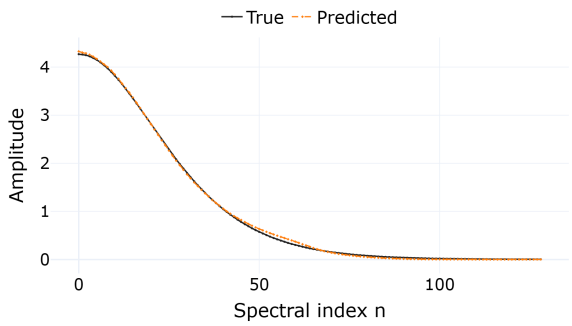
(f) Relative error, $t = 30$.



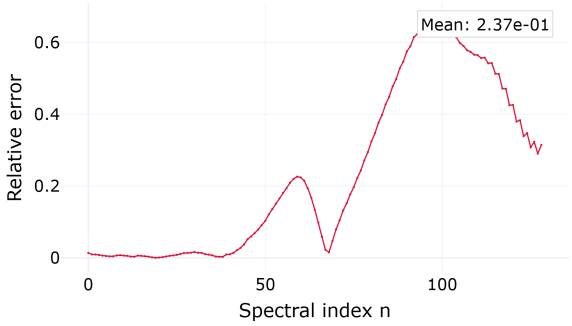
(g) Mean relative spectral error over time.

Figure 4.9 – PINO (data and physics) spatial spectrum at $\alpha = \beta = 3.21$. The prediction tracks the reference across the band, and the physics residual steadies the temporal evolution, so the fit does not degrade toward late time as sharply as the FNO's.

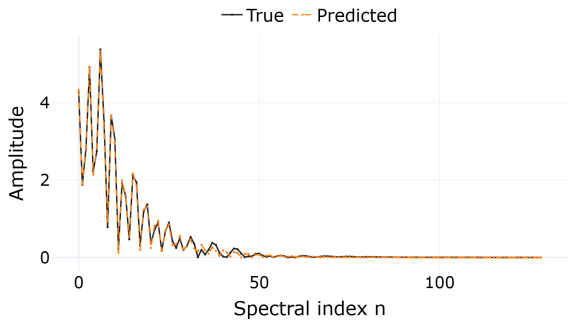
Source: The author (2026).



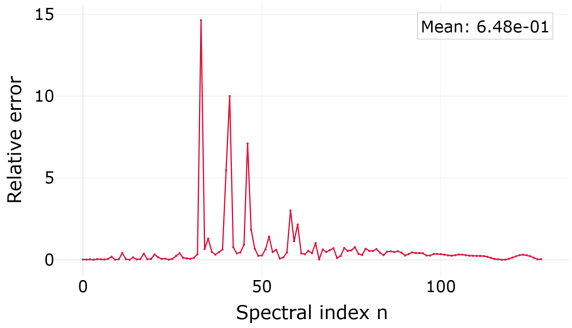
(a) Spectrum, $t = 0$.



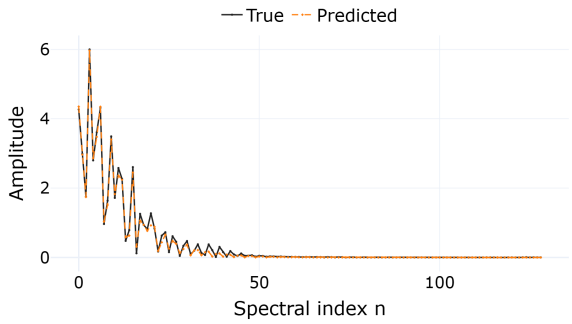
(b) Relative error, $t = 0$.



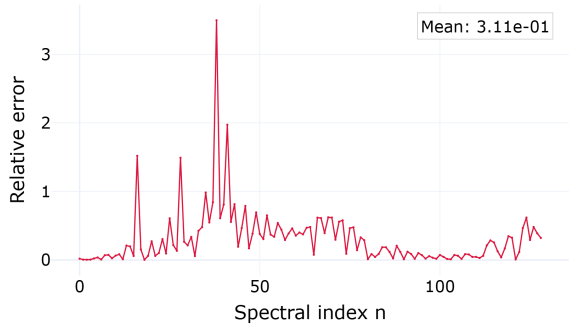
(c) Spectrum, $t = 15$.



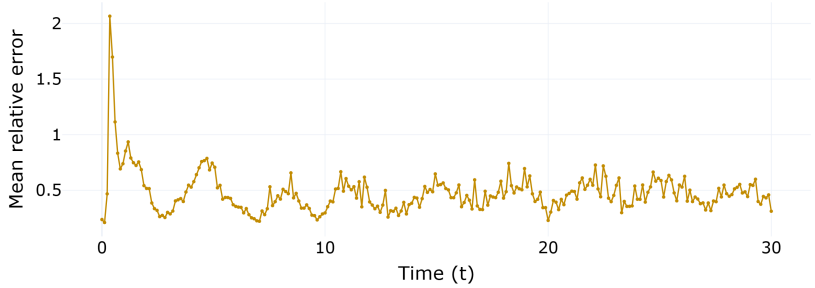
(d) Relative error, $t = 15$.



(e) Spectrum, $t = 30$.



(f) Relative error, $t = 30$.



(g) Mean relative spectral error over time.

Figure 4.10 – PINO (pure physics) spatial spectrum at $\alpha = \beta = 3.21$. Without the supervised term the low- and mid-band undershoot returns, confirming that the physics residual alone does not overcome spectral bias.

Source: The author (2026).

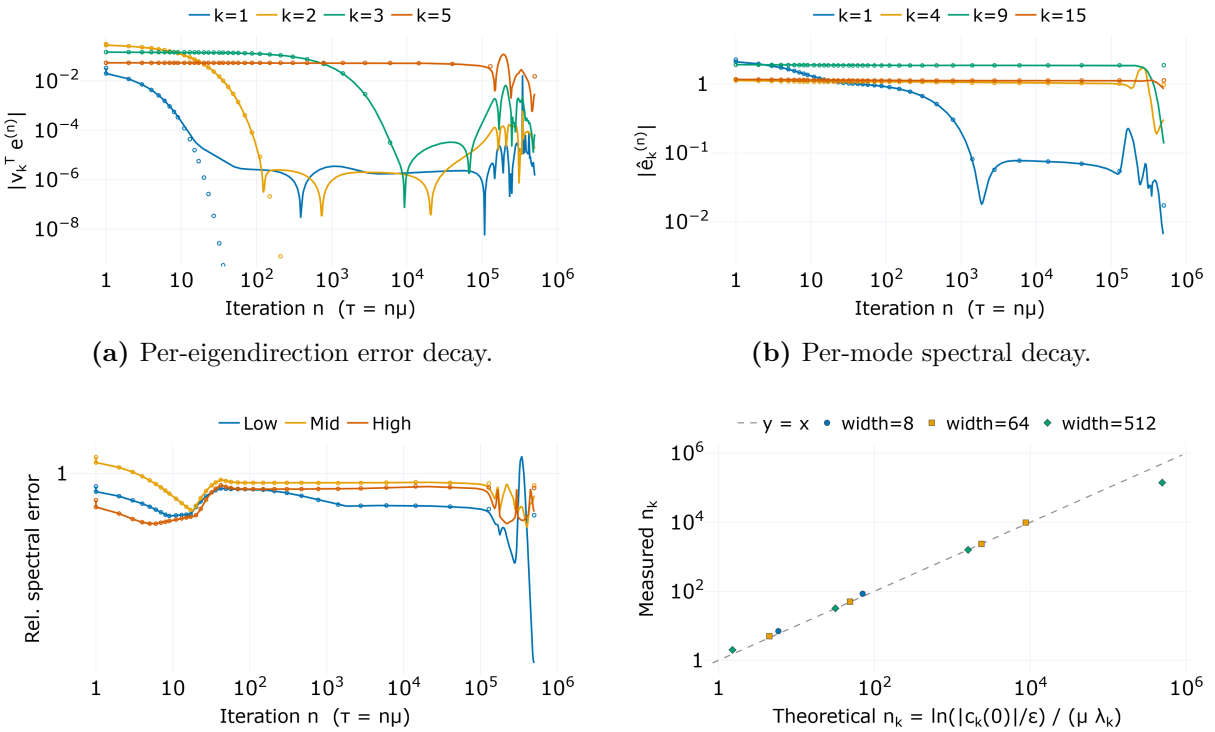
artificial discontinuity at the temporal boundary and excites the Gibbs phenomenon there (GIBBS, 1899). The error curves of Figures 4.1 and 4.2, drawn earlier for the parameter sweep, make the effect and its correction visible once they are read along the time axis rather than across parameters.

In the FNO panel (Figure 4.1) every parameter shows the same feature. After an initial transient the time-resolved error settles to a plateau and then jumps sharply at the final time to several times that plateau. The spike appears at all three nonlinearities and is the dominant part of the FNO's late-time error. In the PINO panel (Figure 4.2) the predicted surfaces are visually identical, yet no such feature appears at the terminal time. The PDE residual and the initial-condition term penalize the temporal inconsistency that a periodic transform would otherwise tolerate, damping the boundary oscillation. Suppressing the FNO's Gibbs artifact is a concrete benefit of the physics term in this study. The correction is a suppression and not a cure, since a small terminal rise remains, and it comes at the modelling cost of the residual machinery.

4.6 Spectral Bias, From Theory to the Network

One behaviour is common to all the models, and we establish its mechanism before reading it off the trained architectures. Spectral bias, derived in Section 2.3.4, predicts that gradient training resolves low frequencies before high ones at a rate set by the eigenvalues of the Neural Tangent Kernel. The minimal experiment of Section 3.5 puts that prediction on the Boussinesq system in its barest form: a network from $\mathbb{R}$ to $\mathbb{R}$ learning the single profile $\eta(x, 15)$ at $\alpha = \beta = 3.21$, at three widths, by gradient descent at the per-width step $\mu = 0.4/\lambda_{\max}$, run for half a million iterations. Because the network is nonlinear in its parameters, nothing here is true by construction. Figure 4.11 reports the four checks.

Each panel tests one prediction. In Figure 4.11a the amplitude of the error along each eigendirection follows the predicted $|1 - \mu\lambda_k|^n$ of (2.55) over several decades, and the four are spent in order of eigenvalue: the leading direction first, the next well after it, the third later still, and the smallest barely moved by the time the run ends. That is the per-eigendirection ordering of (2.52), read off a real network rather than assumed. The measured curves leave the prediction once a coefficient reaches the noise floor and turns back up, where the drifting Jacobian overtakes the initial-kernel approximation. Late in the run they lift further as that drift accumulates. Figure 4.11b reads the same dynamics in Fourier space, where the frequency principle is plain: the lowest sampled wavenumber is driven down while the higher ones lie flat across the run. Figure 4.11c takes the whole spectrum instead of four sampled wavenumbers, averaging the relative error over the low, mid and high bands of (3.11). The low band descends and stays down, dropping further



(a) Per-eigendirection error decay.

(b) Per-mode spectral decay.

(c) Band-averaged relative spectral error.

(d) Measured versus predicted iteration count n_k.

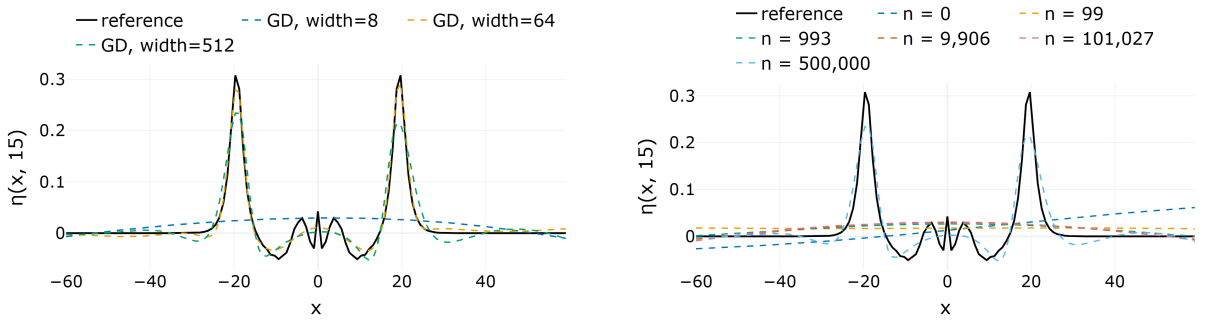
Figure 4.11 – Spectral-bias predictions measured in the minimal experiment of Section 3.5, an $\mathbb{R} \to \mathbb{R}$ network fitting $\eta(x, 15)$ at $\alpha = \beta = 3.21$ by full-batch gradient descent at the per-width step $\mu = 0.4/\lambda_{\max}$, run for 500,000 iterations. Panels (a)–(c) are the widest network, $N_{\mathrm{MLP}} = 512$; panel (d) collects all three widths. Lines are measured, open markers are the initial-kernel prediction.

Source: The author (2026).

in the last stretch of the run; the mid and high bands fall only briefly before levelling off near the magnitude of the reference itself, where they sit for almost the entire run. The error at those wavenumbers stays the size of the signal it was meant to reproduce.

Figure 4.11d makes the agreement quantitative and adds the check the other panels cannot make. The discrete crossing count of (2.71) carries no width, so if it transfers the three networks must fall on one diagonal regardless of their size. For the reachable modes they do, across several decades of iteration count. The departure is as telling as the agreement: the highest modes cross earlier than the initial-kernel count predicts, falling below the diagonal, and they do so by more for the wider networks. That gap is the first sign that the kernel does not stay frozen, and the diagnostic below traces it to its cause.

One measurement qualifies all of this. Figure 4.12 sets what the three runs produced against the profile they were asked to reproduce, and follows the widest network across the run.



(a) Final fit at each width.

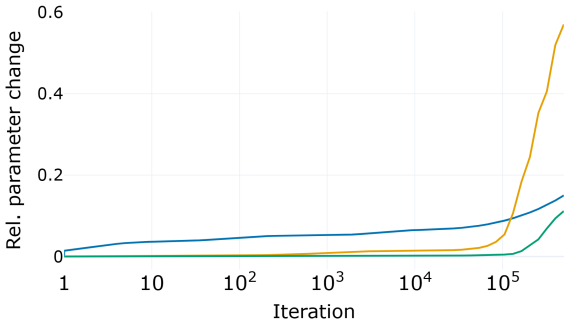
(b) Widest network across the run.

Figure 4.12 – The minimal networks against the reference profile $\eta(x, 15)$ at $\alpha = \beta = 3.21$. Panel (a) shows the final fit at each width; panel (b) follows the widest network across logarithmically spaced checkpoints. The narrowest network never leaves a nearly flat curve, while the wider ones recover the crests, and only at the very end of a half-million-iteration run.

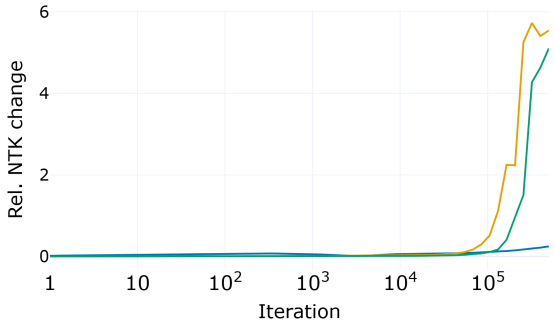
Source: The author (2026).

The reference is two sharp crests with dispersive oscillations between them. The narrowest network never leaves a nearly flat curve, close to a constant offset, and ends at almost the full relative error of its untrained state. The same stall shows in the widest network's own panels, at the wavenumbers the fit never reaches: the mid and high bands of Figure 4.11c level off near the reference, and only the lowest wavenumbers of Figure 4.11b move. The wider networks do eventually escape, but the fit evolution of Figure 4.12b shows the escape arriving only in the last part of the run: for most of the optimization every width sits on the same flat offset, and the recovery is a late event rather than the steady progress the low-frequency modes make from the start.

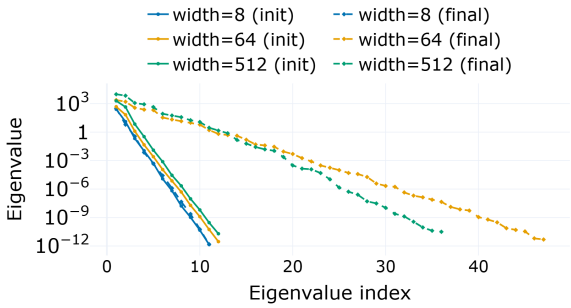
Everything above assumes the kernel stays put. Figure 4.13 reports whether it does, measured along the same three runs.



(a) Relative parameter drift $\|\theta - \theta_0\|/\|\theta_0\|$.



(b) Relative kernel drift $\|K - K_0\|/\|K_0\|$.



(c) Eigenvalue spectrum of K: initialization (solid) versus end of training (dashed), on log–log axes.

Figure 4.13 – Neural Tangent Kernel diagnostic for the minimal experiment at $\alpha = \beta = 3.21$, logged along the same gradient-descent runs as Figure 4.11, for widths 8, 64 and 512. Eigenvalues below the double-precision floor are numerical noise and are not drawn.

Source: The author (2026).

The diagnostic overturns the simplest reading of the theory. The parameters and the kernel do not sit still for all three networks, and they do not settle in order of width. For the narrowest network both move only slightly and then hold, so its kernel is effectively the fixed K_0 the linearization assumes (Figures 4.13a, 4.13b), and it is that network, not the widest, that the initial-kernel approximation (2.42) describes. The wider networks are quiet only at first; late in the run both their parameters and their kernels climb sharply and end far from where they began, which is the movement the linearization rules out. The two wider networks leave the lazy regime and the narrow one never does, the split the fits and the iteration counts already hinted at.

The last panel supplies both the driver and the escape. At initialization the eigenvalue spectrum falls by many orders of magnitude within a handful of modes at every

width, reaching the double-precision floor beyond that. By (2.60) the time to resolve an eigendirection is inversely proportional to its eigenvalue, so a spectrum this steep makes all but the first few smooth directions prohibitively slow to learn and leaves the rest with no usable gradient at all. This is the eigenvalue disparity that Wang, Yu, et al. (2022) identify as the source of spectral bias. The target does not live in the directions that remain: two narrow pulses in a wide domain spread their energy across many wavenumbers, most of them out of reach. For the narrowest network the spectrum barely changes over training, and that is the whole of its story: it stays in the gap between what a fixed kernel offers and what the target requires. The wider networks tell the second half. By the end of training their spectra have flattened, the high modes lifted well clear of the floor, and it is that reshaping which lets them reach frequencies a fixed kernel would deny.

The kernel, not the optimizer, therefore separates the narrowest network from the wider ones, and the route the wider ones take is impractical. It opens only after a run far longer than any spent on the trained models, it depends on the network having the capacity to remake its own kernel, and even then it clears only part of the profile. The minimal experiment exposes the mechanism without supplying a remedy: a fixed kernel imposes the frequency ordering, and escaping it by brute force takes an amount of training no working model would spend. The adaptive and spectrum-aware optimizers of Section 5.3 are meant to provoke the same feature learning deliberately.

That distinction governs how the rest of the chapter reads. All six trained models of Section 3.4 use Adam at learning rate 10^{-3}, so the descent runs above are an instrument for testing (2.55) rather than a forecast of what the MLP achieves; the band-tracking curves of the next section duly show that model driving its low band well down while its high band resists. The shape of the failure survives the change of optimizer, not its magnitude: the ordering of the bands, and the fact that the high wavenumbers are the ones left behind. Two expectations follow for the trained architectures: they inherit the same steep spectrum and so begin biased toward low frequencies, and those with the capacity and the training to reshape their kernels can compress that bias, each to a different degree. The next section puts both to models that carry far more structure than this minimal setting, where a supervised objective, a physics residual and an operator backbone push against the bias with varying success.

4.7 Learning by Frequency Band

We now watch spectral bias develop as the optimizer runs. Figure 4.14 tracks the relative spectral error in the low, mid, and high bands of (3.11) across training for all six models, every one monitored at the common parameter $\alpha = \beta = 3.21$. The panels are therefore directly comparable, and each shows how one architecture and one regime meet

the bias.

The panels share one feature: the high band is the hardest to move. Only the FNO brings it down by more than a modest fraction; in every other architecture and regime it ends within a factor of two of the magnitude of the reference, and in one case above it. The four models carrying a physics residual end with their high band higher than it started, while their low and mid bands come down throughout. The PINN without data is the one panel where the largest error does not sit at high wavenumbers, and for the opposite reason: it is the only model whose low band never falls below the magnitude of the reference, so the worst of its error stays there. The NTK spectrum of Figure 4.13 and the minimal experiment of Figure 4.11 predict exactly this, now measured inside the full architectures. The panels then separate by how far the low and mid bands are pushed down beneath it, and that is where the regimes differ.

We read the MLP panel (Figure 4.14a) first, since with no physics term and no initial-condition term competing against the data fit its ordering carries no confounders: the low band is driven down while the high band barely moves, rising through most of the run before falling back to a little under where it started. This is the prediction of Section 2.3.4 with nothing added, and the other five panels vary it by changing the architecture, the objective, or both.

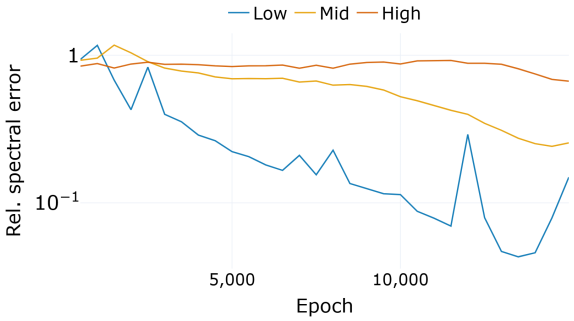
The shared parameter makes one comparison clean: hold the architecture fixed and add or remove the supervised loss. In both the PINN and the PINO families, adding data pulls the low and mid bands down markedly while the high band barely shifts. Once the data term is present the high band is the worst of the three in both families. The physics residual on its own does not substitute. In each family the pure-physics panel leaves the low and mid bands above what the same architecture reaches with data, and the high band in particular stays close to where an untrained network would leave it. Data moves the resolvable bands; the physics constraint does little.

The FNO panel is the one place where the wall is partly breached: it reaches the lowest low- and mid-band errors in the figure, and its high band ends several times lower than any other model's. Even so, its high band remains well above its low band. Read across all six panels, Figure 4.14 makes the general point that the operators compress spectral bias rather than remove it: even the plain FNO still leaves a high-frequency residual, and every other model leaves more.

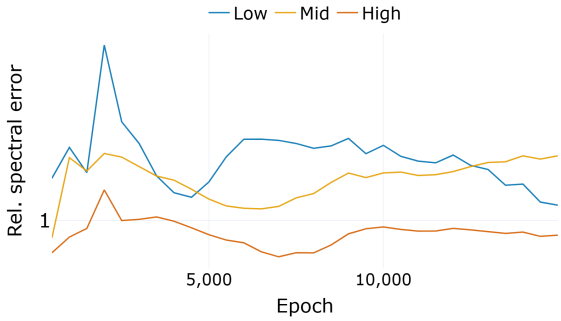
4.8 What the Experiments Show

The experiments do not identify a single best method. They map where each one succeeds and where it fails.

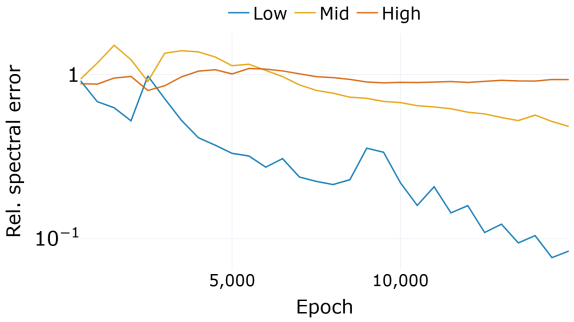
- Spectral bias appears in every model here, and its mechanism is measured rather



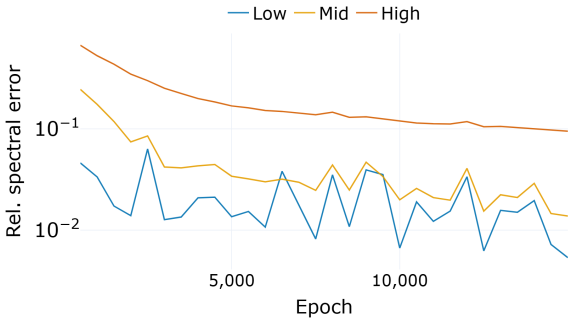
(a) MLP (pure data), at $\alpha = \beta = 3.21$.



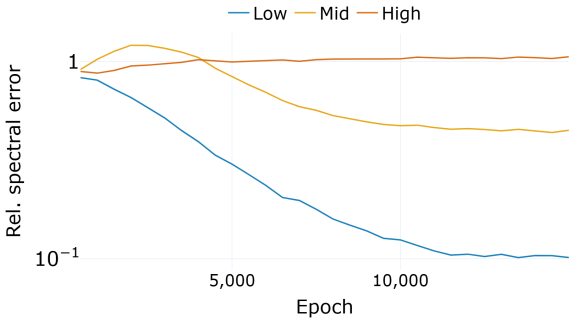
(b) PINN (pure physics), at $\alpha = \beta = 3.21$.



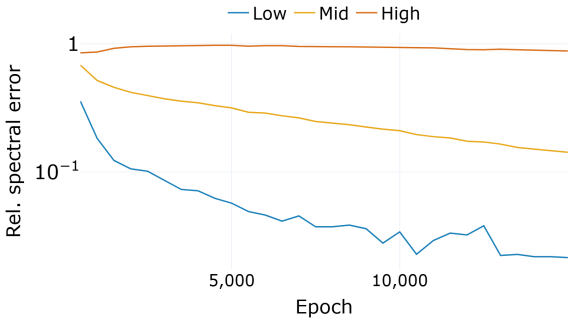
(c) PINN (data and physics), at $\alpha = \beta = 3.21$.



(d) FNO (pure data), at $\alpha = \beta = 3.21$.



(e) PINO (pure physics), at $\alpha = \beta = 3.21$.



(f) PINO (data and physics), at $\alpha = \beta = 3.21$.

Figure 4.14 – Relative spectral error by frequency band during training, every model tracked at the common parameter $\alpha = \beta = 3.21$. The low band (blue), mid band (yellow) and high band (red) follow (3.11). In every model but the PINN without data the three bands end in the order the theory predicts, the low band lowest and the high band highest. That PINN is the exception to where the worst error sits: it is the only model whose low band never falls below the magnitude of the reference, so its largest error stays at low wavenumbers rather than high. That gap between the bands is the signature of spectral bias.

Source: The author (2026).

than assumed. In the minimal experiment of Section 4.6 the per-eigendirection error decay follows the initial-kernel prediction for the reachable modes, and the kernel eigenvalues fall by many orders of magnitude within a dozen modes, so under a fixed kernel only a handful of smooth directions can be learned at all (Figures 4.11, 4.13). Every full architecture but the PINN without data then shows the same ordering, the low wavenumbers resolved first and the high wavenumbers last (Figure 4.14). The minimal experiment also shows how severe the effect is and where its limit lies: a narrow network asked for a single sharp profile stays trapped and recovers almost none of it, while wider networks escape only by leaving the lazy regime, and only after a run far longer than any used to train the models (Figure 4.12). Among the full architectures the high band is the wall: only the plain FNO brings it down by more than a modest fraction, and everywhere else it ends within a factor of two of the magnitude of the reference.

- Data is the main factor, and physics does not substitute for it. Within both the PINN and the PINO families, adding the supervised term reduces the low- and mid-band error, while the physics-only regimes stay close to the biased baseline (Figures 4.14, 4.9, 4.10).

- Operators are more accurate at low and mid wavenumbers. At the common parameter 3.21 the FNO and the PINO with data hold a smaller error across the low and mid spectrum than either PINN or the MLP (Section 4.4), and they cover the whole parameter family from one training, whereas each pointwise model is bound to a single parameter.

- Physics provides temporal stability. The FNO's error grows in time and spikes at the temporal boundary through the Gibbs artifact of its periodic time axis, and the PINO's residual suppresses the spike and keeps the late-time error bounded (Section 4.5). This was the main practical gain from the physics term in the study.

- Limitations remain. The relative error is non-monotone in nonlinearity, with the hardest parameter differing between the two operators (Section 4.2). Zero-shot accuracy is highest at the resolution the physics residual saw and lower away from it, rather than invariant (Section 4.3). The high-frequency band is not fully resolved by any of the methods.

The picture matches the one anticipated in Chapter 1: these methods complement the pseudospectral solver that sets the accuracy ceiling here rather than replacing it. Their value is in amortizing the cost of the parameter sweep and in the temporal regularization

that physics supplies, and it is smallest at the high wavenumbers, where the reference solver is unchallenged.

5 CONCLUSION

This work built a pseudospectral Boussinesq solver and three Scientific Machine Learning methods from first principles: Physics-Informed Neural Networks, Fourier Neural Operators, and Physics-Informed Neural Operators, with a data-only multilayer perceptron as the baseline. It then set them against one another on a single controlled testbed. That testbed was the one-dimensional Boussinesq system with a fixed solitary initial condition and a coupled coefficient $\alpha = \beta$, swept from the near-linear value to the more nonlinear end of the sampling range, with the pseudospectral solution as both training reference and accuracy ceiling. One result recurs throughout: spectral bias appeared in every architecture and every regime we tried. Supervised data reduced it more than the physics residual did. The physics residual mainly provided temporal stability, not raw accuracy, and the finest scales stayed accurate only in the classical solver. The sections below turn this finding into an assessment of each family and point to the work that should come next.

5.1 Assessment of Each Method Family

The models divide into two families by how a single trained network relates to the parameter axis. Reading them family by family also answers the four questions posed in Chapter 1.

The pointwise family: MLP and PINN

These networks represent one solution as a continuous map and are fixed to a single parameter, $\alpha = \beta = 3.21$. They show spectral bias in its simplest form. The data-only MLP, with no equation and no initial-condition term competing against the data fit, drove its low band down while its high band barely moved, ending as much the largest of the three. With nothing else in the objective, this is the bias in isolation, and it answers our question about how spectral bias manifests. Adding the equation, in the PINN, did not repair the high band. Both PINN regimes failed to represent the tail of the sech^2 spectrum even at $t = 0$, and they undershot the mid and high modes throughout. Within this family the data term mattered more than the physics residual, tightening the

low-wavenumber crests, while the physics residual on its own stayed close to the biased baseline. Measured against the pseudospectral baseline, the pointwise family did not close the gap in the mid and high bands, and it covers only the parameter it was trained on, so how it behaves as nonlinearity grows is left open.

The operator family: FNO and PINO

These operators learn the whole parameter family (3.2) from a single training and answer a new parameter with one forward pass. They invert the pointwise picture at low and mid wavenumbers, holding a smaller error across that part of the spectrum than either PINN or the MLP, and covering a parameter range the pointwise models cannot. On the role of data and physics, we can say more. Data remained the more effective intervention, since the no-data PINO gave up the low- and mid-band accuracy that the with-data PINO reached. The physics residual showed a narrower and more specific benefit: it suppressed the temporal-boundary Gibbs spike that degrades the plain FNO at the final time. The hybrid configuration, data and physics together, therefore combined the operator's spectral accuracy with the residual's temporal regularization, holding the field error steady across the trajectory where the plain operator jumps at the boundary.

The operators still show spectral bias at the highest wavenumbers, compressing it further than the pointwise models without removing it: the high-frequency band ended within a factor of two of the magnitude of the reference in every model except the plain FNO, which alone brought it down by more than a modest fraction.

One operator finding qualifies the promise of discretization invariance. Queried across the three evaluation grids, the trained PINO returned a coherent field at every resolution, so the invariance holds geometrically. Its accuracy was highest at resolution 256, the grid on which its physics residual was formed, and lower both below and above it. The invariance therefore does not carry over to accuracy, because a finer grid exposes fine scales that training never resolved.

The mechanism, common to both families

The last question asked whether Neural Tangent Kernel theory genuinely predicts the frequency imbalance or only redescribes it. The minimal experiment of Chapter 4 answers that it predicts, and it does so on the Boussinesq system rather than on a model built to agree. Three networks from $\mathbb{R}$ to $\mathbb{R}$, at widths 8, 64 and 512, were fit to the single reference profile $\eta(x, 15)$ by gradient descent. The measured error along each kernel eigendirection followed the predicted geometric decay for the reachable modes, and the eigendirections were spent in exact order of eigenvalue. The closed-form iteration count,

which contains no reference to width, placed the reachable modes of all three networks on one diagonal. But the agreement has a boundary. It holds while the kernel stays fixed, which is so for the narrowest network and for the leading modes of all of them; the wider networks drift out of that regime as training proceeds, their kernels reshaping until directions the fixed-kernel theory declares unreachable are learned after all. The initial-kernel picture holds where it applies, at onset and for the narrow network, and it is the departure from it that the wider networks exploit to escape.

The same experiment also shows how severe the consequence can be, and where it stops. The kernel spectrum falls by many orders of magnitude within a dozen modes, leaving only a few smooth directions reachable under a fixed kernel. Asked for a profile made of two narrow crests, the narrowest network returned a nearly flat curve and stayed at almost the full relative error of its starting point. The wall is set by the eigenvalue spectrum, not by patience: the stability bound caps the step size, and the smallest eigenvalues are too small for any admissible step to resolve in a feasible number of iterations. The wider networks did eventually recover the crests, but only by remaking their kernels over a run far longer than any spent on the trained models. The escape is real but not a usable remedy: it shows the trap to be a property of gradient descent in the lazy regime rather than of the network, and it points past brute force to the deliberate levers, adaptive and spectrum-aware optimizers, as the practical way to attack the same eigenvalue disparity.

That distinction makes the mechanism useful, not merely descriptive. It ties the frequency-band errors of every architecture to a concrete cause instead of a loose analogy, and it lets us read the compressed but persistent high-band error, in both families, as the same eigenvalue disparity at work, softened by architecture but never removed. It also identifies the lever: an optimizer that rescales the slow eigendirections attacks the disparity directly instead of working around it.

5.2 What This Study Cannot Claim

Several boundaries fence in these conclusions and should be weighed before carrying them further.

- One initial condition and a coupled parameter. Every experiment used a single solitary profile $\eta(x, 0) = \mathrm{sech}^2(x)$ and held $\alpha = \beta$. The operators therefore learned a one-parameter family, not the full four-parameter Boussinesq family, and the reported generalization is generalization along that single coordinate. Decoupling α and β and varying the initial data would test a substantially harder mapping.

- Pointwise models seen at one parameter. The MLP and the PINN were each trained and evaluated only at $\alpha = \beta = 3.21$. Their difficulty in the mid and high bands is a

statement about that regime, not evidence of how they would behave as nonlinearity grows, which would require training them across the axis.

- Time as a spectral axis. The operator implementation applies the Fourier transform along the temporal axis, treating time as periodic, which is the direct cause of the terminal-time Gibbs artifact of Section 4.5. A time-marching or autoregressive operator, as suggested by Li et al. (2021), would drop the periodicity assumption entirely.

- Fixed capacity and budget. Every operator kept only 16 Fourier modes per axis and trained for 15,000 iterations at a single learning rate. The persistent high-band error is consistent with this truncation, and a mode or capacity sweep would be needed to separate the architectural limit from the optimization limit.

- Relative-error normalization. The non-monotone dependence of the error on nonlinearity (Section 4.2) is partly an artifact of normalizing by the instantaneous solution norm. Absolute or energy-based metrics would round out the account of accuracy across the parameter range.

5.3 Directions Forward

The results point to several concrete continuations.

- Attack the shared failure mode directly. Spectral bias survives every regime, and its mechanism is pinned to the kernel eigenvalue disparity, so the levers that target that disparity are the ones to test on this same benchmark: adaptive loss balancing (WANG; TENG, et al., 2021) and the second-order and spectrum-aware optimizers of Khodakarami et al. (2026).

- Replace the temporal transform. A time-marching FNO would remove the temporal Gibbs artifact by construction, isolating how much of the PINO's late-time advantage is intrinsic to the physics term and how much is a repair of a periodicity assumption.

- Broaden the problem, decoupling α and β, varying the initial data, and extending to the two-dimensional Boussinesq system, to see whether the ordering of methods found here survives a genuinely higher-dimensional solution operator.

- Widen the field of comparison to DeepONet (LU et al., 2021) and to hybrid schemes that hand the high-frequency residual back to a classical corrector, which would probe whether the complementary role identified for these methods can become a solver that is at once fast and spectrally faithful.

For the nonlinear Boussinesq system, the value of these Scientific Machine Learning methods lies in amortizing a parameter sweep and, when physics is added, in gaining temporal stability. The fine scales that make dispersive waves hard remain the solver's alone.

REFERENCES

ARORA, S. et al. Fine-Grained Analysis of Optimization and Generalization for Overparameterized Two-Layer Neural Networks. In: PROCEEDINGS of the 36th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2019. v. 97, p. 322–332.

BAKER, N. et al. **Workshop Report on Basic Research Needs for Scientific Machine Learning: Core Technologies for Artificial Intelligence**. Washington, DC, 2019. DOI: 10.2172/1478744.

BIHLO, A.; POPOVYCH, R. O. Physics-informed neural networks for the shallow-water equations on the sphere. **Journal of Computational Physics**, 2022. Available from: <https://arxiv.org/abs/2104.00615>.

BONA, J. L.; CHEN, M.; SAUT, J.-C. Boussinesq equations and other systems for small-amplitude long waves in nonlinear dispersive media: I. Derivation and linear theory. **Journal of Nonlinear Science**, v. 12, p. 283–318, 2002. DOI: 10.1007/s00332-002-0466-4.

BONEV, B. et al. Spherical Fourier Neural Operators: Learning Stable Dynamics on the Sphere. In: INTERNATIONAL Conference on Machine Learning (ICML). [S.l.: s.n.], 2023. Available from: <https://arxiv.org/abs/2306.03838>.

BORDELON, B.; CANATAR, A.; PEHLEVAN, C. Spectrum Dependent Learning Curves in Kernel Regression and Wide Neural Networks. In: PROCEEDINGS of the 37th International Conference on Machine Learning (ICML). [S.l.: s.n.], 2020. Available from: <https://arxiv.org/abs/2002.02561>.

BOUSSINESQ, J. Théorie des ondes et des remous qui se propagent le long d'un canal rectangulaire horizontal. **Journal de Mathématiques Pures et Appliquées**, v. 17, p. 55–108, 1872.

BOYCE, W. E.; DIPRIMA, R. C. **Elementary Differential Equations and Boundary Value Problems**. 10. ed. Hoboken, NJ: John Wiley & Sons, 2012.

BRUNTON, S. L.; PROCTOR, J. L.; KUTZ, J. N. Discovering governing equations from data by sparse identification of nonlinear dynamical systems. **Proceedings of the National Academy of Sciences**, v. 113, n. 15, p. 3932–3937, 2016. DOI: 10.1073/pnas.1517384113.

BURDEN, R. L.; FAIRES, J. D. **Numerical Analysis**. 9. ed. Boston: Brooks/Cole, 2011.

CAO, Y. et al. Towards Understanding the Spectral Bias of Deep Learning. In: PROCEEDINGS of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/1912.01198>.

CHEN, R. T. Q. et al. Neural Ordinary Differential Equations. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2018. v. 31. Available from: <https://arxiv.org/abs/1806.07366>.

CHEN, T.; CHEN, H. Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems. **IEEE Transactions on Neural Networks**, v. 6, n. 4, p. 911–917, 1995.

CHIZAT, L.; OYALLON, E.; BACH, F. On Lazy Training in Differentiable Programming. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2019. v. 32. Available from: <https://arxiv.org/abs/1812.07956>.

CUOMO, S. et al. Physics-informed neural networks for data-driven simulation: Advantages, limitations, and opportunities. **Physica A: Statistical Mechanics and its Applications**, 2023. DOI: 10.1016/j.physa.2022.128415.

CYBENKO, G. Approximation by superpositions of a sigmoidal function. **Mathematics of Control, Signals and Systems**, v. 2, n. 4, p. 303–314, 1989.

DE PINA, I. et al. Investigating Steady Unconfined Groundwater Flow using Physics Informed Neural Networks. **Advances in Water Resources**, 2023. DOI: 10.1016/j.advwatres.2023.104445.

_____. Investigating Steady Unconfined Groundwater Flow Using Physics-Informed Neural Networks. **arXiv preprint arXiv:2112.13792**, 2021. Available from: <https://arxiv.org/abs/2112.13792>.

DEBNATH, L. **Nonlinear Partial Differential Equations for Scientists and Engineers**. 3. ed. New York: Birkhäuser, 2012.

GIBBS, J. Willard. Fourier's Series. **Nature**, v. 59, n. 1539, p. 606, 1899. DOI: 10.1038/059606a0.

GOLUB, G. H.; VAN LOAN, C. F. **Matrix Computations**. 4. ed. Baltimore: Johns Hopkins University Press, 2013.

GUPTA, G. et al. Multiwavelet-based Operator Learning for Differential Equations. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/2109.13459>.

HORNIK, K.; STINCHCOMBE, M.; WHITE, H. Multilayer feedforward networks are universal approximators. **Neural Networks**, v. 2, n. 5, p. 359–366, 1989.

JACOT, A.; GABRIEL, F.; HONGLER, C. Neural Tangent Kernel: Convergence and Generalization in Neural Networks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2018. v. 31. Available from: <https://arxiv.org/abs/1806.07572>.

JAGTAP, A. D. et al. Deep Learning of Inverse Water Waves Problems Using Multi-Fidelity Data: Application to Serre–Green–Naghdi Equations. **arXiv preprint arXiv:2202.02899**, 2022. Available from: <https://arxiv.org/abs/2202.02899>.

KARNIADAKIS, G. E. et al. Physics-informed machine learning. **Nature Reviews Physics**, v. 3, n. 6, p. 422–440, 2021. DOI: 10.1038/s42254-021-00314-5.

KHODAKARAMI, S. et al. Spectral bias in physics-informed and operator learning: analysis and mitigation guidelines. **arXiv preprint**, 2026. Available from: <https://arxiv.org/abs/2602.19265>.

KIM, B. J. et al. QuadNorm: Resolution-Robust Normalization for Neural Operators. **arXiv preprint arXiv:2605.07375**, 2026. Available from: <https://arxiv.org/abs/2605.07375>.

KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. **arXiv preprint arXiv:1412.6980**, 2014.

KOSSAIFI, J. et al. A Library for Learning Neural Operators. **arXiv preprint arXiv:2412.10354**, 2024. Available from: <https://arxiv.org/abs/2412.10354>.

KREYSZIG, E. **Introductory functional analysis with applications**. New York: John Wiley & Sons, 1978.

KRISHNAPRIYAN, A. S. et al. Characterizing Possible Failure Modes in Physics-Informed Neural Networks. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2021. v. 34, p. 26548–26560.

KUMAR, S.; DHAMA, S. Physics-informed neural networks for exploring soliton collisions in the non-integrable Schamel equation in plasmas. **Physics of Fluids**, v. 37, n. 1, 2025. DOI: 10.1063/5.0301334.

LECUN, Y.; BENGIO, Y.; HINTON, G. Deep learning. **Nature**, v. 521, n. 7553, p. 436–444, 2015. DOI: 10.1038/nature14539.

LEVEQUE, R. J. **Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems**. Philadelphia: Society for Industrial and Applied Mathematics, 2007. DOI: 10.1137/1.9780898717839.

LI, Z. et al. Physics-Informed Neural Operator for Learning Partial Differential Equations. **ACM/IMS Transactions on Data Science**, 2023. Available from: <https://arxiv.org/abs/2111.03794>.

LI, Z. et al. **PINO: Physics-Informed Neural Operator – Reference Implementation**. [S.l.: s.n.], 2023. GitHub repository. Available from: <https://github.com/neuraloperator/PINO>.

LI, Z. et al. Fourier Neural Operator for parametric partial differential equations. In: INTERNATIONAL Conference on Learning Representations (ICLR). [S.l.: s.n.], 2021. Available from: <https://arxiv.org/abs/2010.08895>.

LINNAINMAA, A. Taylor expansion of the accumulated rounding error. **BIT Numerical Mathematics**, v. 10, n. 1, p. 47–55, 1970.

LIU, H. et al. Prediction of phase transition and time-varying dynamics of the (2+1)-dimensional Boussinesq equation by parameter-integrated PINNs with phase domain decomposition. **Physical Review E**, v. 108, n. 4, p. 045303, 2023. DOI: 10.1103/PhysRevE.108.045303.

LKHAGVASUREN, B.; CHOI, B.-J.; JIN, H. S. Applications of the Fourier neural operator in a regional ocean modeling and prediction. **Frontiers in Marine Science**, v. 11, 2024. DOI: 10.3389/fmars.2024.1383997.

LU, L. et al. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. **Nature Machine Intelligence**, v. 3, p. 218–229, 2021. DOI: 10.1038/s42256-021-00302-5.

MARTINS, L. G. **Estudo Numérico do Modelo Boussinesq com Topografia e Pressão Variável no Espaço e no Tempo**. 2023. MA thesis – Universidade Federal do Paraná, Curitiba.

MARTINS, L. G.; FLAMARION, M. V.; RIBEIRO-JR, R. A forced Boussinesq model with a sponge layer. **Partial Differential Equations in Applied Mathematics**, v. 10, p. 100661, 2024. DOI: 10.1016/j.padiff.2024.100661.

MCCULLOCH, W. S.; PITTS, W. A logical calculus of the ideas immanent in nervous activity. **Bulletin of Mathematical Biophysics**, v. 5, n. 4, p. 115–133, 1943.

NOCEDAL, J. Updating quasi-Newton matrices with limited storage. **Mathematics of Computation**, v. 35, n. 151, p. 773–782, 1980.

PARIKH, N.; BOYD, S. Proximal Algorithms. **Foundations and Trends in Optimization**, v. 1, n. 3, p. 123–231, 2013. DOI: 10.1561/2400000003.

PASZKE, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: ANNUAL Conference on Neural Information Processing Systems (NeurIPS). Vancouver: [s.n.], 2019. p. 8024–8035. Available from: <https://arxiv.org/abs/1912.01703>.

PATEL, A. et al. A Neural Operator-Based Emulator for Regional Shallow Water Dynamics. **arXiv preprint**, 2025. Available from: <https://arxiv.org/abs/2502.14782>.

PEI, K. et al. Data-driven soliton mappings for integrable fractional nonlinear wave equations via deep learning with Fourier neural operator. **Chaos, Solitons & Fractals**, v. 165, 2022. DOI: 10.1016/j.chaos.2022.112848.

PEREGRINE, D. H. Long waves on a beach. **Journal of Fluid Mechanics**, v. 27, n. 4, p. 815–827, 1967. DOI: 10.1017/S0022112067002605.

RACKAUCKAS, C. et al. Universal Differential Equations for Scientific Machine Learning. **arXiv preprint arXiv:2001.04385**, 2020.

RAHAMAN, N. et al. On the spectral bias of neural networks. In: PMLR. INTERNATIONAL Conference on Machine Learning (ICML). Long Beach: [s.n.], 2019. p. 5301–5310.

RAISSI, M.; PERDIKARIS, P.; KARNIADAKIS, G. E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**, v. 378, p. 686–707, 2019.

ROSENBLATT, F. The perceptron: a probabilistic model for information storage and organization in the brain. **Psychological Review**, v. 65, n. 6, p. 386–408, 1958.

ROSOFSKY, S. et al. Applications of Physics-Informed Neural Operators (PINOs): Shallow Water Equations and Beyond. **arXiv preprint**, 2023. Available from: <https://github.com/shawnrosofsky/PINO_Applications>.

RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. **Nature**, v. 323, n. 6088, p. 533–536, 1986.

STEINMOELLER, D. T.; STASTNA, M.; LAMB, K. G. Fourier pseudospectral methods for 2D Boussinesq-type equations. **Ocean Modelling**, v. 52–53, p. 76–89, 2012. DOI: 10.1016/j.ocemod.2012.05.003.

STRANG, G. **Linear Algebra and Its Applications**. 4. ed. Belmont: Thomson Brooks/Cole, 2006.

SÜLI, E.; MAYERS, D. F. **An Introduction to Numerical Analysis**. Cambridge: Cambridge University Press, 2003.

TANCIK, M. et al. Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains. In: ADVANCES in Neural Information Processing Systems (NeurIPS). [S.l.: s.n.], 2020. v. 33.

TODO. A Review of Physics-Informed Machine Learning in Fluid Mechanics. **Energies**, v. 16, n. 5, 2023. DOI: 10.3390/en16052343.

_____. Modelo Numérico de Boussinesq Adaptado para Ondas de Embarcação. **Revista Brasileira de Recursos Hídricos**, v. 15, n. 4, p. 5–15, 2010. DOI: 10.21168/rbrh.v15n4.p5-15.

TREFETHEN, L. N. **Spectral Methods in MATLAB**. Philadelphia: Society for Industrial and Applied Mathematics, 2000. DOI: 10.1137/1.9780898719598.

UCHIDA, T. et al. Ocean Emulation With Fourier Neural Operators: Double Gyre. **Journal of Advances in Modeling Earth Systems**, v. 17, n. 1, 2025. DOI: 10.1029/2023MS004137.

WANG, L. et al. Explorations of nonlinear waves of the Boussinesq and Camassa-Holm equations using PINNs. **Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences**, v. 480, n. 2286, 2024. DOI: 10.1098/rspa.2023.0580.

WANG, S.; SANKARAN, S., et al. An Expert's Guide to Training Physics-informed Neural Networks. **arXiv preprint arXiv:2308.08468**, 2023. Available from: <https://arxiv.org/abs/2308.08468>.

WANG, S.; TENG, Y.; PERDIKARIS, P. Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks. **SIAM Journal on Scientific Computing**, v. 43, n. 5, a3055–a3081, 2021. DOI: 10.1137/20M1318043.

WANG, S.; YU, X.; PERDIKARIS, P. When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective. **Journal of Computational Physics**, v. 449, p. 110768, 2022. DOI: 10.1016/j.jcp.2021.110768.

WHITHAM, G. B. **Linear and Nonlinear Waves**. New York: Wiley-Interscience, 1974.

XU, Z.-Q. J. et al. Frequency Principle: Fourier Analysis Sheds Light on Deep Neural Networks. In: 5. COMMUNICATIONS in Computational Physics. [S.l.: s.n.], 2020. v. 28, p. 1746–1767. DOI: 10.4208/cicp.OA-2020-0085.

YU, J. et al. Spherical multigrid neural operator for improving autoregressive global weather forecasting. **Scientific Reports**, v. 15, n. 1, 2025. DOI: 10.1038/s41598-025-96208-y.

ZHANG, Y. et al. Influence of layer and neuron configurations on Boussinesq equation solutions via a bilinear neural network. **Nonlinear Dynamics**, v. 112, p. 12315–12333, 2024. DOI: 10.1007/s11071-024-09708-3.

ZHONG, M.; YAN, Z.; TIAN, S. Data-driven parametric soliton-rogon state transitions for nonlinear wave equations using deep learning with Fourier neural operator. **Communications in Theoretical Physics**, v. 75, n. 2, p. 025001, 2023. DOI: 10.1088/1572-9494/acab55.